Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

November 13, 2026

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Problem Statement . . . 4
- 1.3 Contributions . . . 4
- 1.4 Thesis Outline . . . 4

2 Regular Expressions **5**
- 2.1 Syntax . . . 5
- 2.2 Semantics . . . 6
- 2.3 Rust Representation . . . 7

3 Derivatives and Partial Derivatives **8**
- 3.1 Naive Matching . . . 8
- 3.2 Brzozowski Derivatives . . . 10
- 3.3 Antimirov Partial Derivatives . . . 16

4 Matching and Parsing **20**
- 4.1 Matching: A Boolean Answer . . . 20
- 4.2 Parsing: Parse Trees as Matching Evidence . . . 20

5 Derivative-Based Parser **23**
- 5.1 The Standard Derivative-Based Parser . . . 23
  - 5.1.1 Building the Empty-Word Tree . . . 24
  - 5.1.2 Reinserting Characters via Injection . . . 25
  - 5.1.3 Parser . . . 28
  - 5.1.4 Worked Example: Applying the Standard Parser . . . 29
- 5.2 The Bit-Coded Derivative-Based Parser . . . 30
  - 5.2.1 Annotated Expressions . . . 31
  - 5.2.2 Fusing Bit-Lists . . . 32
  - 5.2.3 Internalizing Expressions . . . 33
  - 5.2.4 Extracting Bits . . . 34
  - 5.2.5 Annotated Derivatives . . . 36
  - 5.2.6 Simplification . . . 37
  - 5.2.7 Decoding Bits . . . 39
  - 5.2.8 Parser . . . 41
  - 5.2.9 Worked Example: Applying the Bitcoded Parser . . . 42

6 Partial-Derivative-Based Parser **44**

6.1 The Standard Partial-Derivative Parser . . . 44
6.1.1 Partial Derivatives with Injections . . . 45
6.1.2 State Sets and Selection . . . 49
6.1.3 Parser . . . 51
6.1.4 Worked Example: Standard Partial-Derivative Parser . . . 51
6.2 The Bit-Coded Partial-Derivative Parser . . . 53
6.2.1 Bit-Coded Partial Derivatives . . . 54
6.2.2 Extracting Bits . . . 56
6.2.3 State Sets and Selection . . . 57
6.2.4 Parser . . . 58
6.2.5 Worked Example: Ambiguous Expression . . . 58

7 Translation **60**
7.1 Parsing: String to ExtPat . . . 60
7.2 Translate: ExtPat to Regex . . . 60
7.3 The Surface Syntax . . . 61
7.3.1 Accepted Constructs . . . 62
7.3.2 Rejected Constructs . . . 62
7.4 Translation Sizes . . . 63

8 Evaluation **64**
8.1 Candidates . . . 64
8.2 Evaluation Criteria . . . 65
8.2.1 Measurement Setup . . . 65
8.2.2 Timing . . . 65
8.2.3 Memory . . . 66
8.2.4 Translation . . . 66
8.2.5 Stack Depth . . . 67
8.2.6 Parse Agreement . . . 67
8.3 Datasets . . . 68
8.3.1 S2ELab . . . 68
8.3.2 Synthetic Stress Set . . . 69
8.4 Performance Analysis . . . 70
8.4.1 Stack Depth . . . 70
8.4.2 Timing . . . 71
8.4.3 Memory . . . 82
8.4.4 Translation . . . 85
8.4.5 Parse Agreement . . . 86
8.5 Discussion of Results . . . 88
8.5.1 Cost of the standard construction . . . 88
8.5.2 Cost of bit-coding . . . 89
8.5.3 Cost of partial derivatives . . . 89
8.5.4 Threats to validity . . . 90

9 Related Work and Conclusion **92**

1 Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the problem is regular expression matching. Given a regular expression `(a|ab)*` and an input string `"aba"`, determine whether the string belongs to the language denoted by the expression. This is the *membership problem*. For many applications, this Boolean answer is enough, such as when an input validator only needs to accept or reject a password or URL. Other applications require more information. A lexer, for example, must determine how the input decomposes into tokens, while capturing groups require identifying which parts of the input correspond to parts of the expression. In these cases, determining whether an input is a *match* or a *non-match* is not enough. The *structure* of the match must also be recovered.

Efficiently solving these problems remains an active research area. The choice of algorithm influences performance and memory consumption. Once matching becomes parsing, the way in which the parse tree is recovered also raises the question of how to resolve ambiguity when multiple valid parses exist.

Existing approaches include backtracking and finite-automata-based techniques, such as nondeterministic (NFA) and deterministic (DFA) automata. These approaches have different trade-offs in implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely used, but may exhibit exponential running time on constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require preprocessing or suffer from state-space explosion.

Regular expressions can also be processed using derivative-based algorithms, which transform the expression as input symbols are consumed. Brzozowski introduced regular expression derivatives as a way of transforming a regular expression after consuming an input symbol [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and are closely related to nondeterministic automata [Antimirov, 1996]. Each derivative represents the language remaining after an input symbol has been consumed. This relationship between an expression and its successive residuals provides an elegant foundation for regular expression matching and, with suitable extensions, structured parsing. However, derivative-based techniques are not commonly used in practical regular expression tooling. This work explores their implementation and practical application to regular expression parsing, moving “From Theory to Tools”.

1.2 Problem Statement

The work of [Sulzmann and Lu, 2014] developed a derivative-based algorithm for POSIX regular expression parsing. They also describe an optimized bit-coded variant that represents parse information as a compact bit sequence, which can be decoded after a successful match rather than maintaining an explicit parse tree throughout the computation. In an earlier work, [Sulzmann and Lu, 2012] also studied partial derivatives in the context of regular expression submatching. While submatching is not the focus of this thesis, their work provides relevant background for implementing partial-derivative-based parsers.

The authors have primarily studied these constructions in theoretical and functional programming settings, with less emphasis on practical considerations such as execution time and memory consumption. Haskell implementations of the relevant constructions are also presented in Sulzmann's *Programming Paradigms* teaching material [Sulzmann, 2026].

This thesis examines the practical aspects of derivative-based parsing by implementing derivative-based and partial-derivative-based parsers, including their bit-coded variants, in Rust. Rust's fine-grained control over memory allocation and data representation makes it suitable for evaluating their execution time and memory consumption.

1.3 Contributions

The thesis implements [Sulzmann and Lu, 2014]'s derivative-based and [Sulzmann and Lu, 2012]'s partial-derivative-based parsing constructions in Rust, including their direct and bit-coded variants. The implementations are based on the theory described in the literature and the Haskell reference. The derivative-based parsing function is further implemented in recursive and iterative forms, which are evaluated as separate candidates.

The five parser candidates are evaluated in terms of execution time and memory consumption using two datasets. The first is the *S2Elab* dataset [Siddiq et al., 2024], containing real-world regular expression patterns. The second is the *Synthetic Stress Set*, developed for this evaluation, with patterns designed to stress-test the parsers.

1.4 Thesis Outline

The chapters are structured to present the theoretical concepts and parsing constructions alongside their Rust implementations. *Chapter 2 – Regular Expressions* introduces regular expression syntax, semantics, with their Rust representation, followed by *Chapter 3 – Derivatives*, which covers Brzozowski's derivatives and Antimirov's partial derivatives matching algorithms. *Chapter 4 – Matching vs. Parsing* then introduces the distinction between matching and parsing and the associated disambiguation problem. The two parsing approaches are presented in *Chapter 5 – Derivative Parser* and *Chapter 6 – Partial-Derivative Parser*, including their direct and bit-coded Rust implementations. *Chapter 7 – Translation* describes the translation of regular expression strings into the internal representation. Finally, *Chapter 8 – Evaluation* examines the five parser candidates in terms of execution time, memory consumption, and correctness, while *Chapter 9 – Related Work and Conclusion* summarizes the findings, discusses their implications, and outlines directions for future work.

2 Regular Expressions

2.1 Syntax

Let Σ be an alphabet. Regular expressions over Σ are defined inductively by the grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \qquad (x \in \Sigma). \tag{2.1}$$

In the grammar, r and s range over regular expressions, while x ranges over the alphabet Σ and denotes an arbitrary symbol, such as a, b, or c. The constants ε and $\emptyset$ represent the *empty-word* and *empty-language* expressions. The operators $+$, $\cdot$, and * denote alternation, concatenation, and Kleene star, respectively.

Concatenation is written implicitly, so rs denotes $r \cdot s$. Given this notation, we adopt the precedence and associativity conventions. Unless parentheses are shown explicitly, Kleene star binds most tightly, followed by concatenation, followed by alternation. For example,

$$ab^*c = (a \cdot (b^*)) \cdot c,$$
$$a + bc^* = a + (b \cdot (c^*)).$$

For repeated operators we use left association, so

$$a + b + c = (a + b) + c,$$
$$abc = (ab)c.$$

The grammar above is the core syntax. Most regular expression libraries also offer one-or-more, optional, and bounded repetition (r^+, $r?$, $r^{\{m,n\}}$), but these add no expressive power since each can be defined by the core constructs: $r^+ = r \cdot r^*$, $r? = r + \varepsilon$, and

$$r^{\{m,n\}} = \underbrace{r \cdot \ldots \cdot r}_{m \text{ times}} \cdot \underbrace{(r + \varepsilon) \cdot \ldots \cdot (r + \varepsilon)}_{n-m \text{ times}}.$$

In the implementation, expressions using such operators are translated into the core constructs before they reach the parsing algorithms.

For example, consider the regular expression r_{01} over $\Sigma = \{0, 1\}$

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1. \tag{2.2}$$

By the conventions above, its syntax tree is $((0 + 1)^* \cdot 0) \cdot 1$. This tree represents the hierarchical structure of the expression: a Kleene star applied to the alternation $0 + 1$, followed by concatenation with the literals 0 and 1.

The syntactic structure must be given a semantic interpretation.

2.2 Semantics

A semantic interpretation assigns a language to each regular expression. A *word* over Σ is a finite sequence of symbols from Σ. The set of all words over Σ, including the empty word ε, is denoted by Σ^*. A *language* over Σ is a subset of Σ^*.

For a regular expression r, we write $L(r)$ for the language it denotes, where $L(r) \subseteq \Sigma^*$. The definition follows the structure of the syntax tree by *structural recursion*. Each case corresponds to one production of the grammar in (2.1):

$$\begin{aligned} L(x) &= \{x\}, \\ L(\varepsilon) &= \{\varepsilon\}, \\ L(\emptyset) &= \emptyset, \\ L(r + s) &= L(r) \cup L(s), \\ L(r \cdot s) &= \{vw \mid v \in L(r),\ w \in L(s)\}, \\ L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid n \geq 1,\ w_1, \ldots, w_n \in L(r)\}. \end{aligned} \tag{2.3}$$

Table 2.1 applies these definitions to small expressions. The literal a denotes the language containing only the word a. The expression ε denotes the language containing only the empty word, while $\emptyset$ denotes the language containing no words. The alternation $a + b$ combines the languages of a and b to give $\{a, b\}$, and concatenation ab combines words from the languages of a and b to give $\{ab\}$. The Kleene star a^* allows zero or more concatenations of words from the language of a, giving $\{\varepsilon, a, aa, aaa, \ldots\}$.

Expression r	Language $L(r)$
a	$\{a\}$
ε	$\{\varepsilon\}$
$\emptyset$	$\emptyset$
$a + b$	$\{a, b\}$
ab	$\{ab\}$
a^*	$\{\varepsilon, a, aa, aaa, \ldots\}$

Table 2.1: Languages denoted by basic expressions.

For example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2), we compute its language step by step:

$$\begin{aligned} L(0) &= \{0\}, \\ L(1) &= \{1\}, \\ L(0 + 1) &= L(0) \cup L(1) = \{0, 1\}, \\ L((0 + 1)^*) &= \{0, 1\}^*, \\ L(r_{01}) &= L((0 + 1)^*) \cdot L(0) \cdot L(1) = \{\, w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^* \,\}. \end{aligned} \tag{2.4}$$

This is the set of binary words ending in 01. For example, the words 01, 001, and 1101 belong to $L(r_{01})$, whereas ε, 0, and 010 do not.

This is the question of whether a word *matches* a regular expression: w *matches* r if $w \in L(r)$. Determining this is the *membership problem*.

2.3 Rust Representation

The grammar from (2.1) is represented directly as a Rust enum with one variant for each syntactic construct, as shown in Figure 2.1:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi, Eps, Lit(char),
    Seq(Box<Regex>, Box<Regex>),  // r . s
    Alt(Box<Regex>, Box<Regex>),  // r + s
    Star(Box<Regex>),              // r*
}

impl Regex {
    pub fn seq(r: Regex, s: Regex) -> Regex {
        Regex::Seq(Box::new(r), Box::new(s))
    }
    pub fn alt(r: Regex, s: Regex) -> Regex {
        Regex::Alt(Box::new(r), Box::new(s))
    }
    pub fn star(r: Regex) -> Regex { Regex::Star(Box::new(r)) }
    pub fn lit(c: char) -> Regex { Regex::Lit(c) }
}
```

Figure 2.1: The `Regex` enum and constructors. Derived traits support operations needed by parsing: `Clone` for duplication, `PartialEq/Eq` for comparison, `Hash` for collections.

The type carries no precedence or associativity information, as these are determined by the syntax tree. Rust requires explicit indirection because enum sizes must be known at compile time. Recursive variants therefore use `Box<Regex>`. A direct definition such as `Seq(Regex, Regex)` would have infinite size, whereas `Box` stores recursive values separately, providing finite size [The Rust Project Developers, 2026a,b]. This differs from Haskell's heap-allocated data types.

The example $r_{01} = (0 + 1)^* \cdot 0 \cdot 1$ from (2.2) is represented by the syntax tree:

$$\mathrm{Seq}\Big(\mathrm{Seq}(\mathrm{Star}(\mathrm{Alt}(0, 1)), 0), 1\Big). \tag{2.5}$$

In Rust, this is represented as shown in Figure 2.2, with the outermost `Seq` containing `Lit('1')` and `Seq(Star(Alt(0,1)), 0)` as its right and left children.

```
Regex::seq(
    Regex::seq(
        Regex::star(Regex::alt(Regex::lit('0'), Regex::lit('1'))),
        Regex::lit('0')),
    Regex::lit('1'))
```

Figure 2.2: Rust representation of r_{01}.

3 Derivatives and Partial Derivatives

Chapter 2 introduced the syntax and semantics of regular expressions, and defined the *membership problem*: Decide whether a word w belongs to $L(r)$. This chapter introduces three algorithms for solving it, each returning a Boolean answer: match or no match.

The first is a naive recursive matcher that follows the structure of the regular-expression semantics. The other two transform the regular expression as input symbols are consumed. Brzozowski derivatives transform it into a single residual expression, while Antimirov's partial derivatives represent the possible residual expressions as a set.

3.1 Naive Matching

A direct way to decide whether $w \in L(r)$ is to interpret the regular-expression semantics as recursive matching rules, by following the structure of the expression and searching for possible decompositions of the input. Let $w[i:j]$ denote the substring from position i to j. A recursive procedure $\mathrm{match}(w, i, j, r)$ determines whether $w[i:j] \in L(r)$:

$$\begin{aligned}
\mathrm{match}(w,i,j,\varepsilon) &\iff i = j,\\
\mathrm{match}(w,i,j,x) &\iff j = i+1 \wedge w[i] = x,\\
\mathrm{match}(w,i,j,r+s) &\iff \mathrm{match}(w,i,j,r) \vee \mathrm{match}(w,i,j,s),\\
\mathrm{match}(w,i,j,r\cdot s) &\iff \exists k \in [i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,s),\\
\mathrm{match}(w,i,j,r^*) &\iff i = j \vee \exists k \in (i,j].\ \mathrm{match}(w,i,k,r) \wedge \mathrm{match}(w,k,j,r^*).
\end{aligned} \tag{3.1}$$

Each rule corresponds directly to the semantics. Concatenation does not specify where to split, so the matcher must consider all possible split points k.

For matching $(a \cdot a) \cdot a$ against `aaa`, the split points $k \in \{0,1,2,3\}$ are:

$$\begin{aligned}
&\underbrace{\mathrm{match}(w,0,0,a\cdot a)}_{a\cdot a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\mathrm{match}(w,0,3,a)}_{a \text{ against } w[0:3]=\texttt{aaa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,1,a\cdot a)}_{a\cdot a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\mathrm{match}(w,1,3,a)}_{a \text{ against } w[1:3]=\texttt{aa}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,2,a\cdot a)}_{a\cdot a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\mathrm{match}(w,2,3,a)}_{a \text{ against } w[2:3]=\texttt{a}}\\
\vee\ &\underbrace{\mathrm{match}(w,0,3,a\cdot a)}_{a\cdot a \text{ against } w[0:3]=\texttt{aaa}} \wedge \underbrace{\mathrm{match}(w,3,3,a)}_{a \text{ against } w[3:3]=\epsilon}.
\end{aligned} \tag{3.2}$$

At $k = 2$, both operands match, triggering a recursive search on $(a \cdot a)$ against `aa` with split points $k \in \{0, 1, 2\}$, where $k = 1$ succeeds:

$$\begin{aligned} & \underbrace{\text{match}(w, 0, 0, a)}_{a \text{ against } w[0:0]=\epsilon} \wedge \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \\ \vee\ & \underbrace{\text{match}(w, 0, 1, a)}_{a \text{ against } w[0:1]=\texttt{a}} \wedge \underbrace{\text{match}(w, 1, 2, a)}_{a \text{ against } w[1:2]=\texttt{a}} \\ \vee\ & \underbrace{\text{match}(w, 0, 2, a)}_{a \text{ against } w[0:2]=\texttt{aa}} \wedge \underbrace{\text{match}(w, 2, 2, a)}_{a \text{ against } w[2:2]=\epsilon}. \end{aligned} \tag{3.3}$$

This completes the original match through the nested split points $k = 2$ and $k = 1$.

In Rust. The recursive rules translate directly into the corresponding Rust match arms, shown in Figure 3.1. Alternation tries both branches on the same input, while concatenation and star search possible split points. The star arm clones the body to construct the recursive `Star` expression. This is necessary because the recursive call needs to own the newly constructed expression. The range-based search short-circuits at the first successful split. Nested concatenations repeatedly search smaller substrings, exploring multiple paths before succeeding. This leads to exponential backtracking in the worst case.

```
Regex::Alt(r, s) => {
    match_naive_range(word, i, j, r)
        || match_naive_range(word, i, j, s)
}
Regex::Seq(r, s) => {
    (i..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j, s)
    })
}
Regex::Star(r) => {
    i == j || (i + 1..=j).any(|k| {
        match_naive_range(word, i, k, r)
            && match_naive_range(word, k, j,
                &Regex::star(*r.clone()))
    })
}
```

Figure 3.1: Alternation, concatenation, and star arms of the naive matcher.

Although inefficient, this matcher is useful as a reference implementation because it follows the regular-expression semantics directly without any intermediate transformation. The cost is visible in the worked examples: nested concatenations repeatedly search smaller substrings, each exploring all possible split points before finding a match.

The derivative approach takes a different approach. Rather than searching for a decomposition of w while keeping r fixed, it transforms r as each character of w is consumed. The resulting expression describes the language that can still match the remaining suffix.

3.2 Brzozowski Derivatives

Brzozowski's derivative of r with respect to a symbol x, written $r\backslash x$, is the expression describing exactly what remains of r's language after consuming the leading symbol x of a word $w \in L(r)$ [Brzozowski, 1964]:

$$L(r\backslash x) \;=\; \{\, w \mid xw \in L(r) \,\}. \tag{3.4}$$

For $w = x_1 x_2 \cdots x_n$, consume one symbol at a time by repeatedly taking derivatives:

$$r, \quad r\backslash x_1, \quad (r\backslash x_1)\backslash x_2, \quad \ldots$$

Nullability. After all symbols have been consumed, the remaining input is ε. Thus, the resulting expression must accept ε for the original word to belong to $L(r)$. This is the question of whether the expression is *nullable*. Let $n(r)$ be true exactly when $\varepsilon \in L(r)$:

$$\begin{array}{llll} n(\emptyset) = \text{false} & (\mathsf{N}_\emptyset) & n(r+s) = n(r) \vee n(s) & (\mathsf{N}_+) \\ n(\varepsilon) = \text{true} & (\mathsf{N}_\varepsilon) & n(r \cdot s) = n(r) \wedge n(s) & (\mathsf{N}_{\text{seq}}) \\ n(x) = \text{false} & (\mathsf{N}_{\text{lit}}) & n(r^*) = \text{true} & (\mathsf{N}_*) \end{array} \tag{3.5}$$

The empty language $\emptyset$ is not nullable because it contains no words, including ε, while a literal cannot match the empty word. ε and r^* always match ε. An alternation is nullable when either branch is nullable, and a concatenation only when both components are.

In Rust. The definition is transcribed one `Regex` variant at a time, using the enum of Figure 2.1. The case split of (3.5) becomes the arms of a `match`, shown in Figure 3.2.

```
pub fn nullable(r: &Regex) -> bool {
    match r {
        Regex::Phi => false,
        Regex::Eps => true,
        Regex::Lit(_) => false,
        Regex::Alt(r, s) => nullable(r) || nullable(s),
        Regex::Seq(r, s) => nullable(r) && nullable(s),
        Regex::Star(_) => true,
    }
}
```

Figure 3.2: `nullable` implementation and its six cases: $\mathsf{N}_\emptyset$, N_ε, N_{lit}, N_+, N_{seq} and N_*.

The expression is passed by reference because `nullable` only inspects r and does not modify or consume it. The underscores in N_{lit} and N_* indicate that the corresponding operands are irrelevant. A literal is never nullable, regardless of its symbol, while a Kleene-star expression is always nullable, regardless of its body. The short-circuiting operators `||` and `&&` avoid evaluating the second operand when the result is already determined, while preserving the corresponding logical semantics.

Derivative Rules. The rules specify the expression that remains after consuming x:

$$\begin{aligned}
&\emptyset\backslash x = \emptyset && (\mathsf{D}_\emptyset) && (r+s)\backslash x = (r\backslash x) + (s\backslash x) && (\mathsf{D}_+)\\
&\varepsilon\backslash x = \emptyset && (\mathsf{D}_\varepsilon) && r^*\backslash x = (r\backslash x)\cdot r^* && (\mathsf{D}_*)\\
&y\backslash x = \begin{cases} \varepsilon & x = y \\ \emptyset & x \neq y \end{cases} && (\mathsf{D}_{\text{lit}}) && (r\cdot s)\backslash x = \begin{cases} (r\backslash x)\cdot s + s\backslash x & n(r) = \text{true} \\ (r\backslash x)\cdot s & n(r) = \text{false} \end{cases} && \begin{matrix}(\mathsf{D}^n_{\text{seq}})\\(\mathsf{D}_{\text{seq}})\end{matrix}
\end{aligned} \tag{3.6}$$

The empty language and ε cannot consume x. A literal y succeeds exactly when $x = y$, leaving ε. Alternation retains both derivatives and for r^*, one occurrence of r consumes x while further occurrences remain available.

Concatenation is the only rule that depends on nullability. If r is not nullable, r must consume the leading x, leaving $r\backslash x$ as the remainder of r while s remains untouched. If r is nullable, it may either consume x giving $(r\backslash x)\cdot s$, or r matches ε and s consumes x leaving $s\backslash x$. Hence the nullable case produces the alternation $(r\backslash x)\cdot s + s\backslash x$.

In Rust. Figure 3.3 implements (3.6) similarly, one variant at a time.

```
pub fn deriv(r: &Regex, x: char) -> Regex {
    match r {
        Regex::Phi => Regex::Phi,
        Regex::Eps => Regex::Phi,
        Regex::Lit(c) => if *c == x { Regex::Eps } else { Regex::Phi },
        Regex::Alt(r1, r2) =>
            Regex::alt(deriv(r1, x), deriv(r2, x)),
        Regex::Seq(r1, r2) => {
            let dr1 = Regex::seq(deriv(r1, x), *r2.clone());
            if nullable(r1) {
                Regex::alt(dr1, deriv(r2, x))
            } else { dr1 }
        }
        Regex::Star(r1) =>
            Regex::seq(deriv(r1, x), Regex::star(*r1.clone())),
    }
}
```

Figure 3.3: `deriv` implementation with six cases.

`deriv` differs from `nullable` in that it borrows its input but returns an owned `Regex`, since a derivative is a new expression. The `if nullable(r1)` implements the case split of (3.6), with branches in order $\mathsf{D}^n_{\text{seq}}$ then D_{seq}. This is the only place where a derivative rule consults nullability and where the output can be larger than the input, since the nullable branch produces an alternation while the other produces a concatenation.

The cloning in `*r2.clone()` is necessary because s appears untouched in both branches of the concatenation case, and the star arm must preserve r^*'s body both derived and underived. This copying mechanism drives expression growth, which justifies simplification after every derivative step.

Example: Growth without simplification. Consider $(x+y)^*$ against the word $\mathtt{xx}$: The expression is a star, so D_* applies with body $r = x + y$:

$$\begin{aligned} (x+y)^*\backslash\mathtt{x} &= ((x+y)\backslash\mathtt{x}) \cdot (x+y)^* && \mathsf{D}_*,\ r = x+y \\ &= (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.7}$$

The expression is now the concatenation $(\varepsilon + \emptyset) \cdot (x+y)^*$, and deriving it consumes the second $\mathtt{x}$. The result is an alternation because $\mathsf{D}^n_{\text{seq}}$ applies with $r = \varepsilon + \emptyset$ and $s = (x+y)^*$, its nullable case being selected since $n(r) = n(\varepsilon + \emptyset) = \text{true}$ by N_+ and N_ε:

$$\begin{aligned} &((\varepsilon + \emptyset) \cdot (x+y)^*)\backslash\mathtt{x} \\ &= ((\varepsilon + \emptyset)\backslash\mathtt{x}) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}^n_{\text{seq}} \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (x+y)^*\backslash\mathtt{x} && \mathsf{D}_\varepsilon \\ &= (\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* && \mathsf{D}_*,\ \mathsf{D}_+,\ \mathsf{D}_{\text{lit}} \end{aligned} \tag{3.8}$$

After both symbols are consumed, the result is checked for nullability:

$$\begin{aligned} &n((\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^*) \\ &= n((\emptyset + \emptyset) \cdot (x+y)^*) \vee n((\varepsilon + \emptyset) \cdot (x+y)^*) && \mathsf{N}_+ \\ &= (n(\emptyset + \emptyset) \wedge n((x+y)^*)) \vee (n(\varepsilon + \emptyset) \wedge n((x+y)^*)) && \mathsf{N}_{\text{seq}} \\ &= (\text{false} \wedge \text{true}) \vee (\text{true} \wedge \text{true}) && \mathsf{N}_+,\ \mathsf{N}_\emptyset,\ \mathsf{N}_\varepsilon,\ \mathsf{N}_* \\ &= \text{true}. \end{aligned} \tag{3.9}$$

Thus $(x+y)^*$ matches $\mathtt{xx}$.

Simplification. Derivatives can grow unboundedly. Repeatedly deriving $(x+y)^*$ by the same symbol produces expressions of increasing size. This growth results from the nullable branch of $\mathsf{D}^n_{\text{seq}}$, which introduces an alternation as a redundant structure that is carried into subsequent derivatives. ε and $\emptyset$ accumulate without bound:

$$\begin{aligned} & (x+y)^* && 4 \\ \xrightarrow{\mathtt{x}}\ & (\varepsilon + \emptyset) \cdot (x+y)^* && 8 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^* && 17 \\ \xrightarrow{\mathtt{x}}\ & (\emptyset + \emptyset) \cdot (x+y)^* \ +\ \left((\emptyset + \emptyset) \cdot (x+y)^* \ +\ (\varepsilon + \emptyset) \cdot (x+y)^*\right) && 26 \end{aligned} \tag{3.10}$$

To counter the growth, expressions are simplified after each derivative step, using identities that preserve the denoted language:

$$\begin{aligned} \emptyset \cdot r &= \emptyset && (\mathsf{S}_1) \\ \varepsilon \cdot r &= r && (\mathsf{S}_2) \\ \emptyset + r &= r && (\mathsf{S}_3) \end{aligned} \tag{3.11}$$

S_1 eliminates dead concatenations with $\emptyset$, S_2 eliminates identity concatenations with ε, and S_3 eliminates $\emptyset$ from alternations. Each holds equally with the unit on the right.

Applying them to (3.8), after each step, returns the expression back to $(x+y)^*$, keeping the size bounded regardless of input length:

$$\begin{aligned}
&(\emptyset + \emptyset) \cdot (x+y)^* + (\varepsilon + \emptyset) \cdot (x+y)^* \\
&= \emptyset \cdot (x+y)^* + \varepsilon \cdot (x+y)^* && \mathsf{S}_3,\ \text{twice} \\
&= \emptyset + (x+y)^* && \mathsf{S}_1,\ \mathsf{S}_2 \\
&= (x+y)^* && \mathsf{S}_3
\end{aligned} \tag{3.12}$$

In Rust. The identities are implemented recursively, applying the rules bottom-up over the expression tree. Each arm of the inner match implements an identity from (3.11).

`simplify` takes ownership of its input and returns simplified subexpressions directly, unlike `deriv`, which borrows and constructs. This model is visible where the two compose:

```
current = simplify(deriv(&current, c));
```

`deriv` borrows `current`, constructs a new expression, and `simplify` takes ownership of that result to return it without additional copying. The recursive calls ensure that all dead code is eliminated before an expression is returned, keeping size bounded at each step. Together, derive-then-simplify forms the core loop of the matcher, transforming and cleaning the expression at every input symbol.

```
pub fn simplify(r: Regex) -> Regex {
    match r {
        Regex::Seq(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) | (_, Regex::Phi) => Regex::Phi,
                (Regex::Eps, _) => s,
                (_, Regex::Eps) => r,
                _ => Regex::seq(r, s),
            }
        }
        Regex::Alt(r, s) => {
            let r = simplify(*r);
            let s = simplify(*s);
            match (&r, &s) {
                (Regex::Phi, _) => s,
                (_, Regex::Phi) => r,
                _ if r == s => r,
                _ => Regex::alt(r, s),
            }
        }
        Regex::Star(r) => match simplify(*r) {
            Regex::Eps | Regex::Phi => Regex::Eps,
            r => Regex::star(r),
        },
        other => other,
    }
}
```

Figure 3.4: Simplification rules: S_1 and S_2 in concatenation, S_3 in both branches.

The matcher. The implementation can be summarized by the sequence of residual expressions produced while consuming the input. For $w = x_1 x_2 \cdots x_n$, the initial residual is $r_0 = r$, and each subsequent residual is obtained by deriving with respect to the next symbol and simplifying the result:

$$r_0 = r, \qquad r_i = \text{simplify}(r_{i-1} \backslash x_i) \quad (1 \le i \le n). \tag{3.13}$$

The residual r_i denotes exactly the set of suffixes that complete the consumed prefix $x_1 \cdots x_i$ to a word in $L(r)$. Since simplification preserves the represented language, each r_i is equivalent to the unsimplified derivative after consuming that prefix. After the final symbol has been consumed, the matcher accepts when the remaining residual is nullable:

$$w \in L(r) \iff n(r_n) = \text{true}.$$

Each step depends only on the current residual and the next symbol, so the matcher never revisits a prefix and never backtracks into an earlier residual. The cost of one step is the cost of `deriv` on the current residual, which is proportional to its size.

So $r_0 \xrightarrow{x_1} r_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} r_n$ can be viewed as a path through a lazily constructed DFA. Each residual expression represents a state, and $\xrightarrow{x}$ denotes the transition obtained by taking the derivative with respect to x. Unlike an explicitly constructed DFA, only the states reached by the input are constructed. A residual is an accepting state exactly when it is nullable, so acceptance is determined by whether the final residual is nullable.

In Rust. Figure 3.5 implements (3.13) by composing `deriv`, `simplify`, and `nullable`. The implementation follows the mathematical definition directly. The current residual is transformed at each step, and acceptance is determined after the complete input has been consumed.

```
pub fn match_deriv(input: &str, r: &Regex) -> bool {
    let mut current = r.clone();
    for c in input.chars() {
        current = simplify(deriv(&current, c));
    }
    nullable(&current)
}
```

Figure 3.5: `match_deriv`: the core matcher loop. For each input character, derive the current residual expression and simplify it in place. Accept if the final residual is nullable.

The loop implements the residual sequence. At each iteration, `current` contains the current residual, which is then derived and simplified before processing the next input symbol. Only the current residual is retained, since the previous residual is dropped when `current` is reassigned. In contrast to parsing, where intermediate results may be required to reconstruct a parse tree, Boolean matching requires only the final residual.

The derivative-based matcher avoids the exponential backtracking of naive matching by transforming the expression rather than searching for decompositions. Simplification keeps the resulting expressions compact, preventing the growth shown in (3.10).

Worked Example: Applying the derivative-based matcher Apply `match_deriv` to $r_0 = (a + ab)(b + \varepsilon)$ and $w = \mathtt{ab}$ with the expression written as:

$$r_0 = \mathit{Seq}(s,\, t), \qquad s = a + ab, \qquad t = b + \varepsilon,$$

Derive by `a`. The first factor s is not nullable, so D_{seq} applies to the sequence r_0:

$$r_1 = r_0 \backslash \mathtt{a} = (s \backslash \mathtt{a}) \cdot t \qquad (\mathsf{D}_{\text{seq}}).$$

Since s cannot match the empty word, the second factor t is not touched by this derivative and appears unchanged. To derive s, the alternation rule D_{+} is applied to give:

$$s \backslash \mathtt{a} = (a \backslash \mathtt{a}) + (ab \backslash \mathtt{a}) \qquad (\mathsf{D}_{+}).$$

The two summands are derived separately, each by its own rule:

$$\begin{aligned} a \backslash \mathtt{a} &= \varepsilon && (\mathsf{D}_{\text{lit}}), \\ ab \backslash \mathtt{a} &= (a \backslash \mathtt{a}) \cdot b = \varepsilon \cdot b && (\mathsf{D}_{\text{seq}},\ \mathsf{D}_{\text{lit}}). \end{aligned}$$

Substituting both summands into the alternation and the result into the outer sequence:

$$\begin{aligned} s \backslash \mathtt{a} &= \varepsilon + \varepsilon \cdot b && (\mathsf{D}_{+}), \\ r_0 \backslash \mathtt{a} &= (\varepsilon + \varepsilon \cdot b) \cdot t && (\mathsf{D}_{\text{seq}}). \end{aligned}$$

Then, applying the simplification rule S_2 to the occurrences of $\varepsilon \cdot b$, gives the residual:

$$r_1 = (\varepsilon + b) \cdot t \qquad (\mathsf{S}_2).$$

Derive by `b`. The residual r_1 is again a sequence, but now the first factor $\varepsilon + b$ is nullable, so the nullable sequence rule $\mathsf{D}^{n}_{\text{seq}}$ applies when deriving by b:

$$r_2 = r_1 \backslash \mathtt{b} = \Big((\varepsilon + b) \backslash \mathtt{b}\Big) \cdot t + (t \backslash \mathtt{b}) \qquad (\mathsf{D}^{n}_{\text{seq}}).$$

Since the first factor $\varepsilon + b$ is nullable, it can either consume the character itself, leaving t untouched, or match ε and let t consume it. Both are possible, so $\mathsf{D}^{n}_{\text{seq}}$ produces one summand for each, and D_{+} derives the first factor and t in their respective summands:

$$\begin{aligned} (\varepsilon + b) \backslash \mathtt{b} &= (\varepsilon \backslash \mathtt{b}) + (b \backslash \mathtt{b}) = \emptyset + \varepsilon && (\mathsf{D}_{+},\ \mathsf{D}_{\varepsilon},\ \mathsf{D}_{\text{lit}}), \\ t \backslash \mathtt{b} = (b + \varepsilon) \backslash \mathtt{b} &= (b \backslash \mathtt{b}) + (\varepsilon \backslash \mathtt{b}) = \varepsilon + \emptyset && (\mathsf{D}_{+},\ \mathsf{D}_{\text{lit}},\ \mathsf{D}_{\varepsilon}). \end{aligned}$$

Substituting both summands into the two-summand expression:

$$r_1 \backslash \mathtt{b} = (\emptyset + \varepsilon) \cdot t + (\varepsilon + \emptyset).$$

Applying the simplification rules S_3 and S_2 to the first summand and S_3 to the second, gives the final residual after deriving the second character:

$$r_2 = t + \varepsilon \qquad (\mathsf{S}_3,\ \mathsf{S}_2,\ \mathsf{S}_3).$$

Acceptance. The residual $r_2 = (b + \varepsilon) + \varepsilon$ is nullable by N_{+} and N_{ε}, so $\mathtt{ab} \in L(r_0)$.

3.3 Antimirov Partial Derivatives

The derivative of Section 3.2 combines alternative residuals into a single expression using $+$. Antimirov's partial derivatives instead keep them as a *set* of expressions, one per strand of nondeterminism [Antimirov, 1996]. Writing $pd(r, x)$ for this set:

$$
\begin{aligned}
pd(\emptyset, x) &= \{\} && && (\mathsf{P}_\emptyset)\\
pd(\varepsilon, x) &= \{\} && && (\mathsf{P}_\varepsilon)\\
pd(y, x) &= \begin{cases} \{\varepsilon\} & x = y \\ \{\} & x \neq y \end{cases} && && (\mathsf{P}_{\text{lit}})\\
pd(r + s, x) &= pd(r, x) \cup pd(s, x) && && (\mathsf{P}_+)\\
pd(r^*, x) &= \{\, smartC(r', r^*) \mid r' \in pd(r, x) \,\} && && (\mathsf{P}_*)\\
pd(r \cdot s, x) &= \begin{cases} \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} \cup pd(s, x) & n(r) = \text{true} & (\mathsf{P}^n_{\text{seq}}) \\ \{\, smartC(r', s) \mid r' \in pd(r, x) \,\} & n(r) = \text{false} & (\mathsf{P}_{\text{seq}}) \end{cases}
\end{aligned}
\tag{3.14}
$$

The empty language and ε produce the empty set, since neither has a residual with respect to x: where $\mathsf{D}_\emptyset$ and D_ε returned $\emptyset$ as a node in the result, $\mathsf{P}_\emptyset$ and P_ε return no residual at all. A literal y gives $\{\varepsilon\}$ when $x = y$ and $\{\}$ otherwise. Alternation unions the two residual sets, and for r^* one iteration of r consumes x, with each resulting residual placed in front of r^* so the star may continue. None of these alternatives are combined with $+$; each stays a separate element of the set.

Concatenation depends on nullability for the same reason as D_{seq}. If r is not nullable it must consume x, and each of its residuals is concatenated with the untouched s. If r is nullable, r may instead match ε and let s consume x, in which case $pd(s, x)$ joins the set. Where $\mathsf{D}^n_{\text{seq}}$ combined the two cases with $+$ inside one expression, $\mathsf{P}^n_{\text{seq}}$ keeps them as two elements of one set.

The smart constructor. The concatenation in P_{seq} uses an auxiliary constructor *smartC*. Since each element of $pd(r, x)$ is a separate residual rather than a branch of a sum, *smartC* is applied to each independently. The constructor avoids building $\varepsilon \cdot s$:

$$
smartC(r', s) = \begin{cases} s & r' = \varepsilon \\ r' \cdot s & \text{otherwise.} \end{cases}
\tag{3.15}
$$

In Rust. Figure 3.6 implements *smartC* directly as `smart_seq`. Matching on `r` implements the case split of (3.15): the `Eps` arm returns a clone of `s` and never builds $\varepsilon \cdot s$, while the second arm wraps `r` in front of a cloned `s`

```
pub fn smart_seq(r: Regex, s: &Regex) -> Regex {
    match r {
        Regex::Eps => s.clone(),
        r => Regex::seq(r, s.clone()),
    }
}
```

Figure 3.6: `smart_seq`, the smart constructor *smartC*

Figure 3.7 presents the implementation of (3.14), with each `match` arm corresponding to one case. The `Phi`, `Eps`, and `Lit` variants implement $\mathsf{P}_{\emptyset}$, P_{ε}, and P_{lit} directly, returning `Vec::new()` where the equation gives $\{\}$. `Alt` concatenates the two child vectors, implementing the union of P_{+}.

`Star` maps `smart_seq` over r's own residuals, matching P_{*}. The star's body is derived, and each resulting residual is placed in front of a fresh `Regex::star` so the star may continue. `Regex::star(*r.clone())` clones because `Regex::star` needs an owned `Regex` and `r` is only borrowed, once per residual since the closure runs once for each.

`Seq` implements the case split directly. The `map` applies `smart_seq` to each residual of the left operand, matching P_{seq}, and the guarded `out.extend(pderiv(s, x))` adds $pd(s, x)$ for $\mathsf{P}^{n}_{\text{seq}}$. Although the mathematical definition uses sets, the implementation uses `Vec<Regex>` throughout the recursion and defers deduplication to a single `HashSet` insertion at the top, rather than deduplicating at every `Alt` or `Seq` node on the way down.

Every union in `pderiv` is therefore a vector `extend`, not a set operation. Duplicate residuals reached by different strands are only merged once inserted into the `HashSet` used by the matcher below to keep the residual generation separate from deduplication

```
fn pderiv(r: &Regex, x: char) -> Vec<Regex> {
    match r {
        Regex::Phi => Vec::new(),
        Regex::Eps => Vec::new(),
        Regex::Lit(c) => {
            if *c == x { vec![Regex::Eps] } else { Vec::new() }
        }
        Regex::Alt(r, s) => {
            let mut out = pderiv(r, x);
            out.extend(pderiv(s, x));
            out
        }
        Regex::Seq(r, s) => {
            let mut out: Vec<Regex> = pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, s))
                .collect();
            if nullable(r) {
                out.extend(pderiv(s, x));
            }
            out
        }
        Regex::Star(r) => {
            pderiv(r, x)
                .into_iter()
                .map(|r_prime| smart_seq(r_prime, &Regex::star(*r.clone())))
                .collect()
        }
    }
}
```

Figure 3.7: `pderiv`, implementing (3.14) one arm per case of the equation.

Example: No growth under *pd*. Consider $(x+y)^*$ against $\mathtt{xx}$ under pd.

P_* applies with star body $r = x+y$:

$$\begin{aligned} pd((x+y)^*, \mathtt{x}) &= \{\, smartC(r', (x+y)^*) \mid r' \in pd(x+y, \mathtt{x}) \,\} && \mathsf{P}_* \\ &= \{\, smartC(r', (x+y)^*) \mid r' \in \{\varepsilon\} \,\} && \mathsf{P}_+,\ \mathsf{P}_{\text{lit}} \text{ twice} \\ &= \{(x+y)^*\} && \mathsf{SC},\ r' = \varepsilon. \end{aligned} \tag{3.16}$$

The set is $\{(x+y)^*\}$, so deriving by the second $\mathtt{x}$ repeats the computation, and the final partial derivative accepts ε since $(x+y)^*$ is nullable by N_*. So $\mathtt{xx} \in L((x+y)^*)$.

Equation (3.17) traces the derivative and the partial derivative on the same expression:

$$\begin{aligned} \text{Derivative (unsimplified):}\quad & \underbrace{(x+y)^*}_{4} \xrightarrow{\mathtt{x}} \underbrace{(\varepsilon+\emptyset)(x+y)^*}_{8} \xrightarrow{\mathtt{x}} \underbrace{(\emptyset+\emptyset)(x+y)^*+(\varepsilon+\emptyset)(x+y)^*}_{17} \\ \text{Partial derivative:}\quad & \underbrace{\{(x+y)^*\}}_{4} \xrightarrow{\mathtt{x}} \underbrace{\{(x+y)^*\}}_{4} \xrightarrow{\mathtt{x}} \underbrace{\{(x+y)^*\}}_{4} \end{aligned} \tag{3.17}$$

The unsimplified derivative grows at every step, while the partial derivative stays at one element because $\mathsf{P}_\emptyset$ and P_ε contribute no residual, so there is nothing to simplify.

Finiteness. For a regular expression r, the number of distinct partial derivatives reachable by any input is bounded by $\|r\| + 1$ [Antimirov, 1996, Theorem 3.4, p. 309]. This is a ceiling, not a guarantee that all $\|r\| + 1$ are reached. For $(x+y)^*$, $\|r\| = 2$ the ceiling is 3, yet only $(x+y)^*$ is reachable, since deriving by either $\mathtt{x}$ or $\mathtt{y}$ returns it unchanged.

Ordinary derivatives need equivalence classes to be finite [Brzozowski, 1964, Theorem 4.3], giving $d_R \leq 2^{d_P} - 1$. Simplification removes units and redundant $\emptyset$, but not all equivalent forms. In contrast the partial-derivative bound is linear without simplification.

The matcher. The implementation can be summarized by the sequence of state sets produced while consuming the input. For $w = x_1 x_2 \cdots x_n$, the initial state set is $S_0 = \{r\}$, and each subsequent set is obtained by taking pd of every residual in the previous set and unioning the results:

$$S_0 = \{r\}, \qquad S_i = \bigcup_{r' \in S_{i-1}} pd(r', x_i) \quad (1 \leq i \leq n). \tag{3.18}$$

The state set S_i collects every residual reachable after consuming the prefix $x_1 \cdots x_i$. Unlike the derivative matcher's single residual r_i, no element of S_i grows with the input. The number of elements is bounded by $\|r\| + 1$ as shown above. After the final symbol has been consumed, the matcher accepts when some residual in S_n is nullable:

$$w \in L(r) \iff \exists r_n \in S_n.\ n(r_n) = \text{true}.$$

So $S_0 \xrightarrow{x_1} S_1 \xrightarrow{x_2} \cdots \xrightarrow{x_n} S_n$ can be viewed as a path through a lazily constructed NFA. Each state set represents the set of NFA states reachable after reading the corresponding prefix, and $\xrightarrow{x}$ denotes the transition obtained by taking pd with respect to x and unioning. Unlike an explicitly constructed NFA, only the states reached by the input are constructed. A state set is accepting exactly when it contains at least one nullable residual, so acceptance is determined by whether the final state set contains a nullable residual.

In Rust. Figure 3.8 implements (3.18) with a `HashSet<Regex>`, whose `Hash` and `Eq` implementations collapse residuals reached by different paths. The early return when the set empties follows from construction: an empty set has no states to derive. The final `any(nullable)` checks whether some residual is nullable. Where the derivative matcher holds one growing `Regex`, this holds a set of small residuals that never need simplification.

```
pub fn match_pderiv(input: &str, r: &Regex) -> bool {
    let mut states: HashSet<Regex> = HashSet::new();
    states.insert(r.clone());
    for c in input.chars() {
        let mut next: HashSet<Regex> = HashSet::new();
        for state in &states {
            next.extend(pderiv(state, c));
        }
        if next.is_empty() { return false; }
        states = next;
    }
    states.iter().any(nullable)
}
```

Figure 3.8: The partial-derivative matcher.

Worked Example: Applying the partial-derivative-based matcher Applying `match_pderiv` to $r_0 = (a + ab)(b + \varepsilon)$ and $w = \mathtt{ab}$ with the expression written as:

$$r_0 = \mathit{Seq}(s,\, t), \qquad s = a + ab, \qquad t = b + \varepsilon.$$

Derive by `a`. Since s is not nullable, t is untouched and P_{seq} applies to r_0:

$$S_1 = \{\, \mathit{smartC}(r', t) \mid r' \in \mathit{pd}(s, \mathtt{a}) \,\} \qquad (\mathsf{P}_{\text{seq}}).$$

The alternation rule gives the two branches of s separately:

$$\begin{aligned} \mathit{pd}(a, \mathtt{a}) &= \{\varepsilon\} && (\mathsf{P}_{\text{lit}}),\\ \mathit{pd}(ab, \mathtt{a}) &= \{\mathit{smartC}(\varepsilon, b)\} = \{b\} && (\mathsf{P}_{\text{seq}}), \end{aligned}$$

So $\mathit{pd}(s, \mathtt{a}) = \{\varepsilon, b\}$. Applying smartC to each element against t prepends that element to t, with ε absorbed so that $\varepsilon \cdot t$ is never built:

$$S_1 = \{\, t,\ b \cdot t \,\} = \{\, b + \varepsilon,\ \ b\,(b + \varepsilon) \,\}.$$

Derive by `b`. Each state in S_1 is derived separately and the results unioned.

For the first state $t = b + \varepsilon$:

$$\mathit{pd}(b + \varepsilon, \mathtt{b}) = \mathit{pd}(b, \mathtt{b}) \cup \mathit{pd}(\varepsilon, \mathtt{b}) = \{\varepsilon\} \cup \{\} = \{\varepsilon\} \qquad (\mathsf{P}_{+},\ \mathsf{P}_{\text{lit}},\ \mathsf{P}_{\varepsilon}).$$

For the second state $b \cdot t = b\,(b + \varepsilon)$:

$$\mathit{pd}\big(b\,(b + \varepsilon), \mathtt{b}\big) = \{\, \mathit{smartC}(\varepsilon,\, b + \varepsilon) \,\} = \{b + \varepsilon\} \qquad (\mathsf{P}_{\text{seq}}).$$

Only the left factor b is derived, as it is a non-nullable literal. The union gives:

$$S_2 = \{\varepsilon\} \cup \{b + \varepsilon\} = \{\, \varepsilon,\ \ b + \varepsilon \,\}.$$

Acceptance. Both states in S_2 are nullable, ε by N_{ε} and $b + \varepsilon$ by N_{+} and N_{ε}, so `match_pderiv("ab",` r_0`)` returns `true` and $\mathtt{ab} \in L(r_0)$, agreeing with Section 3.2.

4 Matching and Parsing

Chapter 3 established matching with a Boolean answer: $w \in L(r)$. A matcher only answers whether the input is accepted by the regular expression. Parsing identifies the evidence for the match and *parse trees* act as their structured witnesses.

4.1 Matching: A Boolean Answer

Consider $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$. The matcher can determine $w \in L(r)$ using the algorithms of Chapter 3. There are, however, two ways in which r can match w:

$$\underbrace{a}_{a+ab}\ \underbrace{b}_{b+\varepsilon} \quad \text{and} \quad \underbrace{\texttt{ab}}_{a+ab}\ \underbrace{\varepsilon}_{b+\varepsilon} \tag{4.1}$$

Both choices establish the same membership result, but assign the input differently to the subexpressions of r. The Boolean result alone cannot distinguish these cases, so the matcher cannot reveal which decomposition was chosen.

Parsing makes this additional structure explicit, by showing which alternatives were chosen and which parts of the input were consumed by the corresponding subexpressions.

4.2 Parsing: Parse Trees as Matching Evidence

Parse Tree. A *parse tree* provides a structured witness for such a derivation. Sulzmann and Lu view regular expressions as types and *parse trees* as their values [Sulzmann and Lu, 2014]. The judgment $\vdash v : r$ states that v is a valid parse tree for r, recording which alternatives were chosen and how each part of the input was consumed during matching:

$$\frac{}{\vdash () : \varepsilon}\ (\mathsf{T}_\varepsilon) \qquad \frac{\vdash v_1 : r_1}{\vdash \mathit{Left}\ v_1 : r_1 + r_2}\ (\mathsf{T}^L_+)$$

$$\frac{l \in \Sigma}{\vdash l : l}\ (\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash v_2 : r_2}{\vdash \mathit{Right}\ v_2 : r_1 + r_2}\ (\mathsf{T}^R_+) \tag{4.2}$$

$$\frac{\vdash v_1 : r_1 \qquad \vdash v_2 : r_2}{\vdash (v_1, v_2) : r_1 r_2}\ (\mathsf{T}_{\text{seq}}) \qquad \frac{}{\vdash [\,] : r^*}\ (\mathsf{T}^0_*)$$

$$\frac{\vdash v : r \qquad \vdash vs : r^*}{\vdash (v : vs) : r^*}\ (\mathsf{T}_*)$$

In Rust. Figure 4.1 shows the parse-tree type, with one variant for each constructor.

```
#[derive(Debug, Clone, PartialEq, Eq)]
pub enum ParseTree {
    Empty,                                  // ()
    Char(char),                             // l
    Pair(Box<ParseTree>, Box<ParseTree>),   // (v1, v2)
    Left(Box<ParseTree>),                   // Left v
    Right(Box<ParseTree>),                  // Right v
    Star(Vec<ParseTree>),                   // [v1, ..., vn]
}
```

Figure 4.1: The parse tree type, representing the structure of a parse.

The definitions above translate directly to Rust, except for the star case. $\mathsf{T}\varepsilon$ corresponds to `Empty` for the empty parse, T_{lit} to `Char` for a literal, T_{seq} to `Pair` for a concatenation of parse trees, and T_{+}^{L} and T_{+}^{R} to `Left` and `Right`, recording the chosen branch of an alternation. Both star rules are represented by `Star(vec![])`.

Flattening. The typing judgment describes the structure of a match, but a parse tree must also be related to the word that it matches. For this purpose, they define a *flattening* function $|v|$ that maps a parse tree to the word represented by its structure:

$$\begin{aligned} |()| &= \varepsilon, & |(v_1, v_2)| &= |v_1||v_2|, \\ |l| &= l, & |\mathit{Left}\ v| &= |v|, \\ |[]| &= \varepsilon & |\mathit{Right}\ v| &= |v|, \\ & & |v : vs| &= |v||vs|. \end{aligned} \tag{4.3}$$

The flattening function directly connects parse trees to $L(r)$, and thus validates that a parse tree witnesses membership in $L(r)$, serving as an correctness check:

$$L(r) = \{\, |v| \mid\mid\!\vdash v : r \,\}. \tag{4.4}$$

In Rust. Figure 4.2 shows how `flatten` recursively traverses the parse tree and concatenates its leaf characters. The function borrows the tree to allow subsequent inspection and comparison. The `Pair` case concatenates the results of flattening both children, while `Left` and `Right` delegate to their child, and `Star` concatenates the results for each element.

```
pub fn flatten(v: &ParseTree) -> String {
    match v {
        ParseTree::Empty        => String::new(),
        ParseTree::Char(c)      => c.to_string(),
        ParseTree::Pair(v1, v2) => format!("{}{}", flatten(v1), flatten(v2)),
        ParseTree::Left(v)      => flatten(v),
        ParseTree::Right(v)     => flatten(v),
        ParseTree::Star(vs)     => vs.iter().map(flatten).collect(),
    }
}
```

Figure 4.2: The `flatten` function extracts the matched word from a parse tree.

Unambiguous example. Consider $r = ab$ and $w = \texttt{ab}$. Since each subexpression is a literal, the rule T_{lit} gives each literal a unique witness. Then T_{seq} requires the parse tree for a to be followed by the parse tree for b, combining them into a parse tree for ab:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{\vdash a : a \qquad \vdash b : b}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}}) \tag{4.5}$$

The parse tree $v = (a, b)$ flattens to $|(a, b)| = \texttt{ab}$.

Ambiguous Example. Recall $r = (a + ab)(b + \varepsilon)$ and $w = \texttt{ab}$ from Section 4.1. The word admits two distinct decompositions, corresponding to different parse trees.

The first decomposition uses the right alternatives of both subexpressions:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})$$

$$\frac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}\,(\mathsf{T}_{\text{seq}})$$

$$\underbrace{\frac{\vdash (a, b) : ab}{\vdash \mathit{Right}(a, b) : a + ab}\,(\mathsf{T}^R_+)}_{a + ab} \qquad \underbrace{\frac{\frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}}) \qquad \vdash () : \varepsilon\,(\mathsf{T}_\varepsilon)}{\vdash \mathit{Right}() : b + \varepsilon}\,(\mathsf{T}^R_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the parse tree:

$$\frac{\dfrac{\vdash a : a \quad \vdash b : b}{\vdash (a, b) : ab}}{\vdash \mathit{Right}(a, b) : a + ab} \qquad \frac{\vdash () : \varepsilon}{\vdash \mathit{Right}() : b + \varepsilon} \tag{4.6}$$

$$v_1 = \Big(\mathit{Right}(a, b), \mathit{Right}()\Big), \qquad |v_1| = \texttt{ab}.$$

The second decomposition uses the left alternatives of both subexpressions:

$$\frac{a \in \Sigma}{\vdash a : a}\,(\mathsf{T}_{\text{lit}}) \qquad \frac{b \in \Sigma}{\vdash b : b}\,(\mathsf{T}_{\text{lit}})$$

$$\underbrace{\frac{\vdash a : a}{\vdash \mathit{Left}\, a : a + ab}\,(\mathsf{T}^L_+)}_{a + ab} \qquad \underbrace{\frac{\vdash b : b}{\vdash \mathit{Left}\, b : b + \varepsilon}\,(\mathsf{T}^L_+)}_{b + \varepsilon}$$

By T_{seq}, these derivations combine into the second parse tree:

$$\frac{\vdash a : a}{\vdash \mathit{Left}\, a : a + ab} \qquad \frac{\vdash b : b}{\vdash \mathit{Left}\, b : b + \varepsilon} \tag{4.7}$$

$$v_2 = \Big(\mathit{Left}\, a, \mathit{Left}\, b\Big), \qquad |v_2| = \texttt{ab}.$$

Both trees are valid parse trees for the same expression and word, yet they differ in structure. The typing judgment alone does not determine which parse tree should be selected. A *disambiguation policy* specifies this choice.

5 Derivative-Based Parser

Chapter 3 showed how a Brzozowski-derivative matcher determines whether $w \in L(r)$ and Chapter 4 showed that a matching word may admit multiple valid parse trees. A parser must therefore select a single parse tree according to a disambiguation policy.

Following Sulzmann and Lu [Sulzmann and Lu, 2014], this chapter presents two derivative-based methods for constructing POSIX parse trees. The direct method performs a forward derivative pass followed by a backward pass for tree reconstruction. The bit-coded method records parse-tree information during the forward pass, avoiding a separate backward pass.

5.1 The Standard Derivative-Based Parser

Consider the example $r = (a + ab)(b + \varepsilon)$ with input $w = \texttt{ab}$. Chapter 4 showed that it is ambiguous, as the word admits two parse trees. The question here is whether a parse tree can be recovered from the final derivative:

$$\begin{aligned} r &\xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow{b} (\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon) + (\varepsilon + \emptyset). \end{aligned} \tag{5.1}$$

The derivation was already computed in Section 3.2, shown here unsimplified. The final derivative is nullable, so $w \in L(r)$ and both disjuncts accept the empty word. Its first disjunct, $(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)$, reflects $ab \cdot \varepsilon$, and its second, $(\varepsilon + \emptyset)$, reflects $a \cdot b$.

Since a derivative step only records the remaining input language and not how consumed characters were matched, the final derivative alone does not show which decomposition was selected, and cannot be used to recover the parse tree. However, the derivation sequence itself contains this information. Each step consumed one character and is invertible given that character, so reversing the steps recovers the tree, as shown in Figure 5.1:

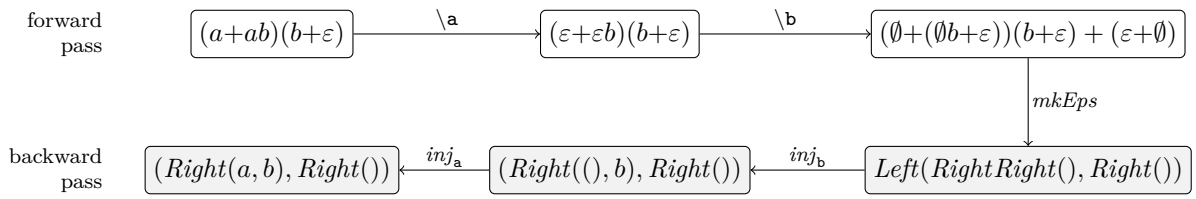


Figure 5.1: Overview of the standard derivative-based parser on the ambiguous example. The forward pass computes derivatives, while the backward pass uses *mkEps* to construct a parse tree for the final nullable derivative and *inj* to reconstruct the original parse tree.

The two-pass pipeline. The direct construction uses two passes. The forward pass applies `deriv` (Figure 3.3) and `nullable` (Figure 3.2) to process the input.

The backward pass applies two new functions to walk the derivation sequence in reverse and reconstruct the parse tree. First *mkEps* builds a parse tree for the empty word from the final nullable expression. Then injection steps are applied, with *inj* reinserting one consumed character into the parse tree, moving in w from right to left. Each injection step takes the current parse tree, the corresponding derivative step, and its consumed character, and returns the parse tree for the preceding expression.

As shown in Figure 5.1 after the first injection of b, the tree is $(\mathit{Right}((), b), \mathit{Right}())$, where $()$ marks the position still to be filled. After the second injection of a, that position is replaced by the subtree $\mathit{Right}(a, b)$, giving the complete parse tree $(\mathit{Right}(a, b), \mathit{Right}())$.

In the following the operations for the standard derivative parser are shown in more detail.

5.1.1 Building the Empty-Word Tree

The backward pass begins with *mkEps*, which reads a parse tree for the empty word off the final nullable expression. The function follows [Sulzmann and Lu, 2014, Fig. 3] and is defined recursively:

$$\begin{aligned}
\mathit{mkEps}_{\varepsilon} &= () && (\mathsf{E}_{\varepsilon}) \\
\mathit{mkEps}_{r^*} &= [\,] && (\mathsf{E}_{*}) \\
\mathit{mkEps}_{r_1 r_2} &= (\mathit{mkEps}_{r_1}, \mathit{mkEps}_{r_2}) && (\mathsf{E}_{\mathrm{seq}}) \\
\mathit{mkEps}_{r_1+r_2} &= \begin{cases} \mathit{Left}\ \mathit{mkEps}_{r_1} & \text{if } n(r_1) \quad (\mathsf{E}^L_+) \\ \mathit{Right}\ \mathit{mkEps}_{r_2} & \text{otherwise} \quad (\mathsf{E}^R_+) \end{cases}
\end{aligned} \tag{5.2}$$

The first three rules follow from the structure of the expression: $(\mathsf{E}_{\varepsilon})$ gives ε an empty parse tree, (E_{*}) treats a star matching the empty word as zero iterations, and $(\mathsf{E}_{\mathrm{seq}})$ matches a concatenation against ε only when both factors do. No rules are needed for $\emptyset$ or literals, since neither can match the empty word. The alternation rule is where disambiguation occurs. If exactly one of r_1 and r_2 is nullable, that branch is the only possible witness. If both are, (E^L_+) breaks the tie by preferring the left one, following POSIX's leftmost-longest policy [Sulzmann and Lu, 2014]. In the example, the final expression is:

$$(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon) + (\varepsilon + \emptyset).$$

At top-level $+$, both summands are nullable, so (E^L_+) applies and picks the left one. Within the left summand's first factor $\emptyset + (\emptyset b + \varepsilon)$, only the right alternative is nullable, so (E^R_+) applies, and within it only the right sub-alternative ε is nullable, so (E^R_+) applies again, contributing $\mathit{RightRight}()$. The second factor $b + \varepsilon$ contributes $\mathit{Right}()$ via (E^R_+), giving:

$$\mathit{Left}(\mathit{RightRight}(), \mathit{Right}()).$$

Since *mkEps* handles disambiguation, the forward pass cannot simplify. It must keep the branch that consumed a, or injection cannot recover $\mathit{Right}(a, b)$. Simplifying gives:

$$r \xrightarrow{a} (\varepsilon + \varepsilon b)(b + \varepsilon) \xrightarrow{\text{simp}} (\varepsilon + b)(b + \varepsilon) \xrightarrow{b} (\emptyset + \varepsilon)(b + \varepsilon) + (\varepsilon + \emptyset) \xrightarrow{\text{simp}} (b + \varepsilon) + \varepsilon,$$

This still witnesses $w \in L(r)$, but $(\emptyset + \varepsilon) \to \varepsilon$ and $(\varepsilon + \emptyset) \to \varepsilon$ erase the pair structure holding both characters. *mkEps* on $(b + \varepsilon) + \varepsilon$ gives $\mathit{Left}(\mathit{Right}(()))$ and injecting b fills that one empty leaf, leaving none for a, so $\mathit{Right}(a, b)$ is unrecoverable.

In Rust. The recursive definition of *mkEps* (5.2) is implemented directly in Rust, as shown in Figure 5.2, following the same cases for each expression form.

```
pub fn mk_eps(r: &Regex) -> ParseTree {
    match r {
        Regex::Eps          => ParseTree::Empty,
        Regex::Star(_)      => ParseTree::Star(Vec::new()),
        Regex::Seq(r1, r2) => ParseTree::Pair(
                               Box::new(mk_eps(r1)), Box::new(mk_eps(r2))),
        Regex::Alt(r1, r2) =>
            if nullable(r1)   {ParseTree::Left(Box::new(mk_eps(r1)))}
            else              {ParseTree::Right(Box::new(mk_eps(r2)))},
        Regex::Lit(c)       => panic!("mk_eps called on Lit('{}')", c),
        Regex::Phi          => panic!("mk_eps called on Phi"),
    }
}
```

Figure 5.2: `mk_eps`: construct a parse tree for the empty word.

The expression r is borrowed rather than consumed because the backward pass needs it again during injection. The alternation arm encodes disambiguation as a conditional, examining `r2` only when `nullable(r1)` is false. The panic arms enforce the precondition that *mkEps* applies only to nullable expressions, rejecting the non-nullable $\emptyset$ and literals.

5.1.2 Reinserting Characters via Injection

After *mkEps* constructed the parse tree v for the empty input, the reconstruction starts from this initial tree and proceeds backwards through the derivation sequence. Each application of *inj* takes the current tree, the corresponding derivative step, and its consumed character, returning a tree for the preceding expression. *inj* recursively follows the structure of the residual until it reaches the subexpression containing the literal that consumed the character. It reinserts the character there and then unwinds to rebuild the surrounding tree. After all characters have been reinserted, the parse tree for the original expression and input is recovered. The injection function is defined by the following rules [Sulzmann and Lu, 2014, Fig. 3]:

$$
\begin{aligned}
inj_{x\backslash x}() &= x && (\mathsf{I}_{\text{lit}}) \\
inj_{(r_1+r_2)\backslash x}(\mathit{Left}\ v_1) &= \mathit{Left}\ (inj_{r_1\backslash x}\ v_1) && (\mathsf{I}^L_+) \\
inj_{(r_1+r_2)\backslash x}(\mathit{Right}\ v_2) &= \mathit{Right}\ (inj_{r_2\backslash x}\ v_2) && (\mathsf{I}^R_+) \\
inj_{r^*\backslash x}(v, vs) &= (inj_{r\backslash x}\ v) : vs && (\mathsf{I}_*) \\
inj_{(r_1 r_2)\backslash x}(v_1, v_2) &= (inj_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}_{\text{seq}}),\ n(r_1) = \text{false} \\
inj_{(r_1 r_2)\backslash x}(\mathit{Left}(v_1, v_2)) &= (inj_{r_1\backslash x}\ v_1,\ v_2) && (\mathsf{I}^L_{\text{seq}}),\ n(r_1) = \text{true} \\
inj_{(r_1 r_2)\backslash x}(\mathit{Right}\ v_2) &= (mkEps_{r_1},\ inj_{r_2\backslash x}\ v_2) && (\mathsf{I}^R_{\text{seq}}),\ n(r_1) = \text{true}
\end{aligned}
\tag{5.3}
$$

Rules $(\mathsf{I}_{\text{lit}})$ and (I_*) invert their derivative rules directly, reconstructing the consumed character and the current iteration, the latter prepended to the remaining ones. Since the consumed character lies in exactly one branch of an alternation, the incoming tree is either *Left* v_1 or *Right* v_2. Rule (I^L_+) recurses into v_1 and rebuilds the *Left* node, and (I^R_+) recurses

into v_2 and rebuilds the *Right* node. The concatenation rules cover the three cases of the sequence derivative. If $n(r_1) = \text{false}$, r_1 cannot match ε and must consume x, so $(\mathsf{I}_{\text{seq}})$ recurses into v_1. If $n(r_1) = \text{true}$, the derivative recorded which component consumed x as a tag: $(\mathsf{I}^L_{\text{seq}})$ handles an incoming $\mathit{Left}(v_1, v_2)$ tag and recurses into v_1, while $(\mathsf{I}^R_{\text{seq}})$ handles a *Right* v_2 tag by reconstructing r_1's empty match with mkEps_{r_1} and recursing into v_2. Since $\emptyset$ consumes no characters, *inj* is never applied with $\emptyset$ as the residual.

```
pub fn inject(r: &Regex, l: char, v: ParseTree) -> ParseTree {
    match r {
        Regex::Lit(c) => { assert!(*c == l);
            ParseTree::Char(l) }
        Regex::Alt(r1, r2) => match v {
            ParseTree::Left(v1) =>
                ParseTree::Left(Box::new(inject(r1, l, *v1))),
            ParseTree::Right(v2) =>
                ParseTree::Right(Box::new(inject(r2, l, *v2))),
            _ => panic!("inject/Alt: want Left|Right"),
        },
        Regex::Star(r1) => match v {
            ParseTree::Pair(v1, vs) => {
                let v1 = inject(r1, l, *v1);
                match *vs {
                    ParseTree::Star(mut rest) => {
                        rest.push(v1); ParseTree::Star(rest)
                    }
                    o => panic!("inject/Star: want Star, got {:?}", o),
                }
            }
            _ => panic!("inject/Star: want Pair"),
        },
        Regex::Seq(r1, r2) => if !nullable(r1) {
            match v {
                ParseTree::Pair(v1, v2) => ParseTree::Pair(
                    Box::new(inject(r1, l, *v1)), v2),
                _ => panic!("inject/Seq: want Pair"),
            }
        } else {
            match v {
                ParseTree::Left(vp) => match *vp {
                    ParseTree::Pair(v1, v2) => ParseTree::Pair(
                        Box::new(inject(r1, l, *v1)), v2),
                    o => panic!("inject/SeqL: want Pair, got {:?}", o),
                },
                ParseTree::Right(v2) => ParseTree::Pair(
                    Box::new(mk_eps(r1)),
                    Box::new(inject(r2, l, *v2))),
                _ => panic!("inject/Seq: want Left|Right"),
            }
        },
        Regex::Eps | Regex::Phi => panic!("inject called on Eps/Phi"),
    }
}
```

Figure 5.3: `inject`: reinsert a character into a parse tree.

In Rust. `inject(r, l, v)` takes three arguments because *inj* is indexed by both an expression and a character. Figure 5.3 shows the complete implementation.

The function matches on the expression `r` first and on the tree `v` second, following the

order in which (5.3) indexes its rules. This order matters because a tree tag is interpreted relative to the expression that produced it. Under an alternation, `Left` selects the left disjunct, while under a nullable concatenation it indicates that the derivative continued inside the left operand. The `Lit` arm reconstructs a leaf after asserting that the consumed character matches, and the `Star` arm inverts (I_*) by reinserting the character into the current iteration and appending it to the tail built so far.

The expression `r` is borrowed and read only, while the tree `v` is consumed and decomposed. In the concatenation arms where the right operand was not derived, `v2` moves directly into the new pair, since (5.3) leaves it unchanged. The `Right` arm of concatenation is the only concatenation case that synthesizes content, calling `mk_eps(r1)` because the derivative skipped `r1` and the forward pass produced no tree for that half of the pair.

Calling `nullable(r1)` in the `Seq` arm recomputes a decision the forward pass already made when it chose between D_{seq} and $\mathsf{D}^n_{\text{seq}}$. However, carrying this as a flag at every step would require additional storage, while `nullable` is cheap to evaluate on demand.

Appending rather than prepending makes each injection step cost $O(1)$ amortized instead of copying the tail's current length, but it leaves a `Star`'s iterations in the reverse of their chronological order until a single corrective pass restores it. `reverse_stars` makes this pass once, after the whole backward walk completes, walking the finished tree and reversing every `Star`'s items exactly once; both parser entry points apply it to their result before returning. This keeps the whole backward pass $O(n)$: prepending at every step, as (5.3) suggests literally, would instead copy the already-built tail on each call, the same $O(n^2)$ cost the persistent `Bits` representation avoids for the bit-coded parser's annotations (§5.2.1).

5.1.3 Parser

The standard parser has two passes. The forward pass stores each residual and the backward pass applies *mkEps* then *inj* in reverse. The split is needed because *inj* requires the pre-character residual, which only the stored sequence provides:

$$\begin{aligned} r_0 &= r, & r_i &= r_{i-1}\backslash x_i & (1 \le i \le n),\\ v_n &= \mathit{mkEps}(r_n), & v_i &= \mathit{inj}_{r_{i-1}\backslash x_i}(x_i,\ v_{i+1}) & (0 \le i < n). \end{aligned} \tag{5.4}$$

Every r_i is kept because each injection step needs r_{i-1}; re-deriving it would duplicate the forward pass. The vector holds the entire sequence $r_0, \ldots, r_n$, not just the final residual.

In Rust. Figure 5.4 implements (5.4). The forward loop fills the `Vec expressions` with $r_0, \ldots, r_n$, the nullability check returns early if r_n is not nullable, and the backward loop walks the vector in reverse, applying `inject` to each residual and its character, before `reverse_stars` restores left-to-right order once (§5.1.2).

```
pub fn parse_deriv_std_loop(input: &str, r: &Regex) -> Option<ParseTree>{
    let chars: Vec<char> = input.chars().collect();
    let mut expressions = Vec::with_capacity(chars.len() + 1);
    expressions.push(r.clone());
    for &c in &chars {
        expressions.push(deriv(expressions.last().unwrap(), c));
    }
    if !nullable(expressions.last().unwrap()) { return None; }
    let mut tree = mk_eps(expressions.last().unwrap());
    for i in (0..chars.len()).rev() {
        tree = inject(&expressions[i], chars[i], tree);
    }
    Some(reverse_stars(tree))
}
```

Figure 5.4: `parse_deriv_std_loop`: the standard parser.

5.1.4 Worked Example: Applying the Standard Parser

Apply the standard parser to the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. There are two derivative steps, one per input symbol. After deriving `a`, the left factor $\varepsilon + \varepsilon b$ is nullable, so the second derivation on `b` uses $\mathsf{D}^n_{\text{seq}}$ rather than D_{seq}:

$$\begin{aligned} r_0 = (a + ab)(b + \varepsilon) &\xrightarrow[\mathsf{D}_{\text{seq}}]{a} r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon) \\ &\xrightarrow[\mathsf{D}^n_{\text{seq}}]{b} r_2 = \underbrace{(\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)}_{X} + \underbrace{(\varepsilon + \emptyset)}_{Y}. \end{aligned}$$

The final expression $r_2 = X + Y$ is nullable, and both disjuncts accept the empty word. X records $ab \cdot \varepsilon$ (the first factor matched ab, the second ε); Y records $a \cdot b$ (the first matched a, the second b).

Backward pass: mkEps. The backward pass begins by building the parse tree for the empty word, $mkEps(r_2)$. Since $r_2 = X + Y$ and both disjuncts are nullable, E^L_+ selects the left one:

$$mkEps(r_2) = Left\Big(mkEps(X)\Big) \qquad (\mathsf{E}^L_+),$$

where $X = (\emptyset + (\emptyset b + \varepsilon))(b + \varepsilon)$. Since X is a concatenation, E_{seq} combines the trees for its two factors:

$$mkEps(X) \xrightarrow{\mathsf{E}_{\text{seq}}} \Big(mkEps(\emptyset + (\emptyset b + \varepsilon)),\ mkEps(b + \varepsilon)\Big).$$

The first factor selects ε by applying E^R_+ twice, since $\emptyset$ and $\emptyset b$ are not nullable; the second factor selects ε by E^R_+, since b is not nullable. These give

$$\begin{aligned} mkEps(\emptyset + (\emptyset b + \varepsilon)) &\xrightarrow{\mathsf{E}^R_+} Right\ mkEps(\emptyset b + \varepsilon) \\ &\xrightarrow{\mathsf{E}^R_+} Right\ Right\ mkEps(\varepsilon) \\ &\xrightarrow{\mathsf{E}_\varepsilon} Right\ Right\ () \end{aligned} \qquad \begin{aligned} mkEps(b + \varepsilon) &\xrightarrow{\mathsf{E}^R_+} Right\ mkEps(\varepsilon) \\ &\xrightarrow{\mathsf{E}_\varepsilon} Right\ () \end{aligned}$$

and hence

$$v_2 = mkEps(r_2) = Left\Big(Right\ Right\ (), Right\ ()\Big).$$

Backward pass: Inj. Reinsertion begins with $\mathtt{b}$ and maps v_2 to a tree for $r_1 = (\varepsilon + \varepsilon b)(b + \varepsilon)$. The *Left* tag of v_2 sends $\mathsf{I}^L_{\text{seq}}$ into the first factor $\varepsilon + \varepsilon b$, leaving the second factor's tree $\mathit{Right}\,()$ unchanged. Inside $\varepsilon + \varepsilon b$, I^R_+ selects the ab branch; since ε is nullable, $\mathsf{I}^R_{\text{seq}}$ supplies the empty tree for ε before I_{lit} reinserts $\mathtt{b}$:

$$
\begin{aligned}
\mathit{inj}_{r_1\backslash \mathtt{b}}(v_2) &= \mathit{inj}_{r_1\backslash \mathtt{b}}\Big(\underbrace{\mathit{Left}(\mathit{Right}\,\mathit{Right}\,(),\ \mathit{Right}\,())}_{v_2}\Big) \\
&\xrightarrow{\mathsf{I}^L_{\text{seq}}} \Big(\mathit{inj}_{(\varepsilon+\varepsilon b)\backslash \mathtt{b}}(\mathit{Right}\,\mathit{Right}\,()),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}^R_+} \Big(\mathit{Right}\,\mathit{inj}_{\varepsilon b\backslash \mathtt{b}}(\mathit{Right}\,()),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}^R_{\text{seq}}} \Big(\mathit{Right}\,(\mathit{mkEps}(\varepsilon),\ \mathit{inj}_{b\backslash \mathtt{b}}()),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{E}_\varepsilon,\ \mathsf{I}_{\text{lit}}} \underbrace{\Big(\mathit{Right}((),b),\ \mathit{Right}\,()\Big)}_{v_1}.
\end{aligned}
$$

Reinsertion of $\mathtt{a}$ maps v_1 to a tree for $r_0 = (a + ab)(b + \varepsilon)$. The outer sequence's first factor $a + ab$ is not nullable, so I_{seq} fires directly on v_1's first component, leaving the second component $\mathit{Right}\,()$ untouched. That first component, $\mathit{Right}((), b)$, carries a *Right* tag, so I^R_+ selects ab. Inside $ab = \mathit{Seq}(a, b)$ the first factor a is again not nullable, so I_{seq} fires once more and recurses into a, where I_{lit} reinserts the character:

$$
\begin{aligned}
\mathit{inj}_{r_0\backslash \mathtt{a}}(v_1) &= \mathit{inj}_{r_0\backslash \mathtt{a}}\Big(\underbrace{(\mathit{Right}((),b),\ \mathit{Right}\,())}_{v_1}\Big) \\
&\xrightarrow{\mathsf{I}_{\text{seq}}} \Big(\mathit{inj}_{(a+ab)\backslash \mathtt{a}}(\mathit{Right}((),b)),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}^R_+} \Big(\mathit{Right}\,\mathit{inj}_{ab\backslash \mathtt{a}}(((),b)),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}_{\text{seq}}} \Big(\mathit{Right}\,(\mathit{inj}_{a\backslash \mathtt{a}}(()),\ b),\ \mathit{Right}\,()\Big) \\
&\xrightarrow{\mathsf{I}_{\text{lit}}} \underbrace{\Big(\mathit{Right}(a,b),\ \mathit{Right}\,()\Big)}_{v_0}.
\end{aligned}
$$

The implementation stores r_0, r_1, and r_2 in its vector. One entry per character, plus the initial expression, kept for the backward pass to reconstruct from. The tree it returns is $(\mathit{Right}(a, b), \mathit{Right}())$, which flattens to $\mathtt{ab}$ and matches the input.

5.2 The Bit-Coded Derivative-Based Parser

The direct construction of Sulzmann and Lu [Sulzmann and Lu, 2014] reconstructs the tree only after the full input, since each injection step needs the preceding expression. Their bit-coded optimization records this information during a single pass instead.

Each subexpression carries the bit-sequence it has committed to so far, written $bs@r$. P

and Q denote the annotated factors for $r_0 = (a + ab)(b + \varepsilon)$. Deriving $w = \texttt{ab}$ gives:

$$\begin{aligned} ri_0 &= [\,]@\Big(\underbrace{[0]@a \oplus [1]@(a\,b)}_{P} \cdot \underbrace{[0]@b \oplus [1]@\varepsilon}_{Q}\Big) \\ &\xrightarrow{a} ri_1 = [\,]@\Big(\underbrace{[0]@\varepsilon \oplus [1]@b}_{P'} \cdot Q\Big) \\ &\xrightarrow{b} ri_2 = [\,]@\Big([1]@Q \;\oplus\; [0,0]@\varepsilon\Big). \end{aligned} \tag{5.5}$$

Figure 5.5 shows this derivation together with the extraction step it feeds into:

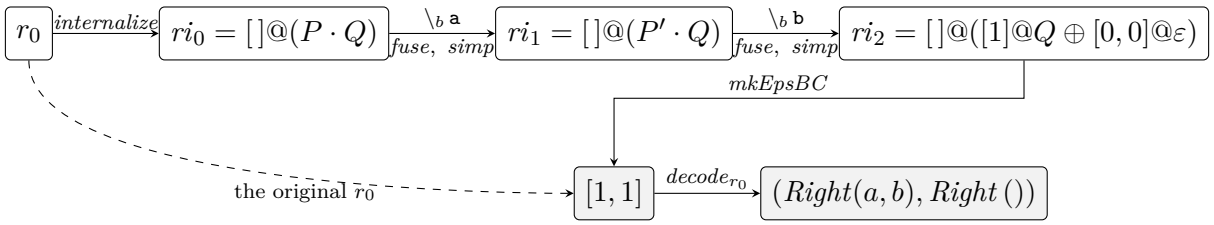


Figure 5.5: The bit-coded parser on the ambiguous example. *internalize* introduces the bits, derivative threads them forward, and *mkEpsBC* and $decode_{r_0}$ extract and read them against r_0.

The bit-coded pipeline. *internalize* tags each alternation branch with a bit that records its preference, producing $ri_0 = [\,]@(P \cdot Q)$ with $P = [0]@a \oplus [1]@(a\,b)$ and $Q = [0]@b \oplus [1]@\varepsilon$, where 0 selects the left branch and 1 the right.

Deriving by `a` extracts P's leading a, giving P' while Q is untouched; deriving by `b` consumes the rest. Since P''s ε branch is nullable, both decompositions survive: in the first, the derivative continues inside P', which finishes and folds its bit 1 onto Q; in the second, Q matches `b` instead, and P''s completion bit fuses with Q's match into $[0,0]@\varepsilon$. This yields $ri_2 = [\,]@([1]@Q \oplus [0,0]@\varepsilon)$.

Unlike the standard construction, the bit-coded construction requires simplification to prevent unbounded growth of annotated expressions. The function *simp*, from [Sulzmann and Lu, 2014, Fig. 6], discards dead branches and collapses finished subexpressions, while *fuse* preserves bits on surviving nodes, as when P collapses into P' and then onto Q.

Once `ab` is exhausted, *mkEpsBC* walks ri_2 the way *mkEps* walks a plain expression, taking the leftmost nullable branch at each remaining choice: $[1]@Q$ is nullable, so it contributes its 1, and inside Q the ε branch is nullable, contributing a second 1, giving $bits = [1,1]$. $decode_{r_0}$ then replays these two bits against the original r_0. The first 1 selects $a+ab$'s right branch and the second selects $b + \varepsilon$'s right branch, giving the tree $(Right(a,b), Right\,())$.

This shows that the bit-coded construction reconstructs the parse tree in a single step, while the direct construction reverses residuals step by step.

5.2.1 Annotated Expressions

Building parse-tree information during the forward pass extends the derivative computation with bit-sequence annotations, propagated by the bit-threaded derivative $ri\backslash_b l$. An

annotated expression ri extends r by associating each non-$\emptyset$ node with a bit-sequence bs recording the reconstruction choices made during the forward pass.

$$ri ::= \emptyset \mid (bs@\varepsilon) \mid (bs@x) \mid (bs@ri_1 \oplus ri_2) \mid (bs@ri_1\, ri_2) \mid (bs@ri^*). \tag{5.6}$$

In Rust. Figure 5.6 defines `ARegex`, the annotated counterpart of `Regex`. Every variant except `Phi` carries a leading `Bits` field (cf. (5.6)). `Bits` is a persistent, `Rc`-backed cons list, not a `Vec<bool>`: prepending a prefix costs only the prefix, independent of the accumulated history. A `Vec<bool>` would copy the entire prefix on every derivative step, making an n-character run cost $O(n^2)$. the persistent representation keeps it $O(n)$.

```
pub enum ARegex {
    Phi,
    Eps(Bits), Lit(Bits, char),
    Alt(Bits, Box<ARegex>, Box<ARegex>),
    Seq(Bits, Box<ARegex>, Box<ARegex>),
    Star(Bits, Box<ARegex>),
}
```

Figure 5.6: The annotated-expression type extends `Regex` with a leading bit-sequence, `Bits`.

All following operations act on this `ARegex`. Since derivative and simplification steps remove nodes while the bits attached to them must survive, each operation needs a way to transfer those bits onto the node that remains. The helper *fuse* provides this transfer.

5.2.2 Fusing Bit-Lists

fuse, defined in [Sulzmann and Lu, 2014, Fig. 5], transfers bits from a node that is about to be removed onto the node that survives in its place. It prepends bs to the top node's annotation bs' without descending into its children, so outer reconstruction choices precede those already recorded in the subtree.

$$\begin{aligned}
\mathit{fuse}(bs, \emptyset) &= \emptyset \\
\mathit{fuse}(bs, (bs'@\varepsilon)) &= (bs +\!\!+ bs')@\varepsilon \\
\mathit{fuse}(bs, (bs'@x)) &= (bs +\!\!+ bs')@x \\
\mathit{fuse}(bs, (bs'@ri_1 \oplus ri_2)) &= (bs +\!\!+ bs')@(ri_1 \oplus ri_2) \\
\mathit{fuse}(bs, (bs'@ri_1\, ri_2)) &= (bs +\!\!+ bs')@(ri_1\, ri_2) \\
\mathit{fuse}(bs, (bs'@ri^*)) &= (bs +\!\!+ bs')@ri^*.
\end{aligned} \tag{5.7}$$

In Rust. Figure 5.7 implements the six rules of (5.7), one per variant. The helper `Bits::prepend` corresponds to the concatenation $+\!\!+$ and places `bs` before the node's existing bits:

```
pub fn fuse(bs: &[bool], ri: ARegex) -> ARegex {
    if bs.is_empty() { return ri; }
    match ri {
        ARegex::Phi => ARegex::Phi,
        ARegex::Eps(p) => ARegex::Eps(p.prepend(bs)),
        ARegex::Lit(p, c) => ARegex::Lit(p.prepend(bs), c),
        ARegex::Alt(p, r1, r2) => ARegex::Alt(p.prepend(bs), r1, r2),
        ARegex::Seq(p, r1, r2) => ARegex::Seq(p.prepend(bs), r1, r2),
        ARegex::Star(p, r) => ARegex::Star(p.prepend(bs), r),
    }
}

impl Bits {
    // prefix ++ self, O(prefix.len()) regardless of how long self
    // already is -- the existing chain is shared, not copied.
    pub fn prepend(&self, prefix: &[bool]) -> Bits {
        if prefix.is_empty() { return self.clone(); }
        Bits(Rc::new(Node::Prepend(Rc::from(prefix), self.0.clone())))
    }
}
```

Figure 5.7: `fuse`: prepend a bit-list to the annotation of the top node.

The `Phi` arm is the only case that returns without calling `prepend`, because `Phi` represents the empty language and carries no annotation slot, so there is no bit-list for `fuse` to prepend to. Every other arm rebuilds the top node with a combined annotation and moves its two children, or single child in the star case, unchanged. Since `fuse` prepends `bs` only to the top node's annotation, the bits carried by its children are not touched, and the operation rewrites exactly one node regardless of how large the subtree below it is.

The function is therefore non-recursive, and no subtree needs to be copied. The implementation realizes this directly, since `ri` is consumed by value and its untouched children move into the rebuilt node without a clone. `Bits::prepend` wraps the node's existing chain in a new `Rc`, sharing rather than copying it: the allocation is proportional only to `bs`'s length, independent of how many bits the node already carries. This is why `Bits` replaces a plain `Vec<bool>` here: with a `Vec`, `prepend` would have to copy the existing content on every call, a cost proportional to the combined length of the two bit-lists – the node's own history can grow with every character consumed so far, which would make a single `fuse` call as expensive as the whole parse up to that point. An empty `bs` is the one case that still skips any work, since prepending an empty prefix returns the node's existing chain unchanged.

5.2.3 Internalizing Expressions

The forward pass operates on annotated expressions, but the input is a plain expression, so r must first be converted into its annotated counterpart ri. The function *internalize*, following [Sulzmann and Lu, 2014, Fig. 5], performs this conversion recursively, and it is

the only step that introduces new bits:

$$\begin{aligned}
\mathit{internalize}(\emptyset) &= \emptyset \\
\mathit{internalize}(\varepsilon) &= [\,]@\varepsilon \\
\mathit{internalize}(x) &= [\,]@x \\
\mathit{internalize}(r_1 + r_2) &= [\,]@\Big(\mathit{fuse}([0], \mathit{internalize}(r_1)) \oplus \mathit{fuse}([1], \mathit{internalize}(r_2))\Big) \\
\mathit{internalize}(r_1\, r_2) &= [\,]@\Big(\mathit{internalize}(r_1)\; \mathit{internalize}(r_2)\Big) \\
\mathit{internalize}(r^*) &= [\,]@\mathit{internalize}(r)^*.
\end{aligned} \tag{5.8}$$

The tags [0] and [1] on the two branches of $r_1 + r_2$ mark the left and right branch respectively, so the decoder can later recover which one was taken. Every other case introduces no new bit and uses only the empty list $[\,]$. The two calls to *fuse* in (5.8) are the only points in the construction where new bits are introduced.

In Rust. Figure 5.8 directly maps the constructors of `Regex` to their annotated counterparts. Each recursive call applies `internalize` to the corresponding subexpression, preserving the structure of the original expression in the annotated representation.

```
pub fn internalize(r: &Regex) -> ARegex {
    match r {
        Regex::Phi => ARegex::Phi,
        Regex::Eps => ARegex::Eps(Bits::empty()),
        Regex::Lit(c) => ARegex::Lit(Bits::empty(), *c),
        Regex::Alt(r1, r2) => {
            let ri1 = fuse(&[false], internalize(r1));
            let ri2 = fuse(&[true], internalize(r2));
            ARegex::Alt(Bits::empty(), Box::new(ri1), Box::new(ri2))
        }
        Regex::Seq(r1, r2) => ARegex::Seq(Bits::empty(),
            Box::new(internalize(r1)), Box::new(internalize(r2))),
        Regex::Star(r1) => ARegex::Star(Bits::empty(),
            Box::new(internalize(r1))),
    }
}
```

Figure 5.8: `internalize` converts a `Regex` into an `ARegex`. Each alternation branch receives a tag recording its side, and the two `fuse` calls are the only points where new bits are introduced.

The `Alt` case calls `fuse` with a non-empty bit-list, tagging the left operand with `[false]` and the right with `[true]` before wrapping them in a `Alt` with an empty outer bit-list. Every later bit originates from one of these two tags and is propagated during derivation.

The `Phi`, `Eps`, `Lit`, `Seq`, and `Star` cases use an empty bit-list, since they introduce no disambiguation choice at this stage. Star iteration choices are instead determined during derivative computation.

5.2.4 Extracting Bits

mkEpsBC, from [Sulzmann and Lu, 2014, Fig. 5], is the bit-list counterpart of *mkEps*. It keeps the left preference and star closing bit, but extracts bits instead of building a tree.

Each rule prepends the current node's bits to those returned recursively.

$$
\begin{aligned}
mkEpsBC_{(bs@\varepsilon)} &= bs \\
mkEpsBC_{(bs@ri^*)} &= bs +\!\!+ [1] \\
mkEpsBC_{(bs@ri_1\, ri_2)} &= bs +\!\!+ mkEpsBC_{ri_1} +\!\!+ mkEpsBC_{ri_2} \\
mkEpsBC_{(bs@ri_1 \oplus ri_2)} &= bs +\!\!+ \begin{cases} mkEpsBC_{ri_1} & \varepsilon \in L(ri_1) \\ mkEpsBC_{ri_2} & \text{otherwise.} \end{cases}
\end{aligned}
\tag{5.9}
$$

As with *mkEps*, the rules cover only the four nullable forms. $\emptyset$ and literals cannot match the empty word, so *mkEpsBC* is called only on nullable expressions and both are handled as preconditions.

In Rust. Figure 5.9 implements `mk_eps_bc` alongside `mk_eps` (Figure 5.2), with one difference. Where `mk_eps` builds a `ParseTree`, `mk_eps_bc` concatenates bit-lists, calling `bs.to_vec()` where `mk_eps` would borrow or clone: this is the point where a node's `Bits` chain (Section 5.2.1) is finally walked and materialized into a concrete `Vec<bool>`, something `fuse` deliberately avoids doing on every derivative step. The `Star` arm also does not recurse, since a star matching the empty word uses zero iterations, so it only appends the closing `true` and never examines the child expression:

```
pub fn mk_eps_bc(ri: &ARegex) -> Vec<bool> {
    match ri {
        ARegex::Phi => panic!("mk_eps_bc called on Phi"),
        ARegex::Lit(_, c) => panic!("mk_eps_bc called on Lit('{}')", c),
        ARegex::Eps(bs) => bs.to_vec(),
        ARegex::Alt(bs, r1, r2) => {
            let mut result = bs.to_vec();
            if nullable_bc(r1) {
                result.extend(mk_eps_bc(r1));
            } else {
                result.extend(mk_eps_bc(r2));
            }
            result
        }
        ARegex::Seq(bs, r1, r2) => {
            let mut result = bs.to_vec();
            result.extend(mk_eps_bc(r1));
            result.extend(mk_eps_bc(r2));
            result
        }
        ARegex::Star(bs, _) => {
            let mut result = bs.to_vec();
            result.push(true);
            result
        }
    }
}
```

Figure 5.9: `mk_eps_bc` implements (5.9). `Phi` and `Lit` panic, since neither can be nullable.

5.2.5 Annotated Derivatives

$ri\backslash_b l$ has the same case structure as the plain derivative (3.6), but extended with bits threaded through the rules. Each rule corresponds to its unannotated counterpart:

$$
\begin{aligned}
\emptyset\backslash_b x &= \emptyset && (\mathsf{B}_\emptyset)\\
(bs@\varepsilon)\backslash_b x &= \emptyset && (\mathsf{B}_\varepsilon)\\
(bs@c)\backslash_b x &= \begin{cases} bs@\varepsilon & c = x \\ \emptyset & \text{otherwise} \end{cases} && (\mathsf{B}_{\text{lit}})\\
(bs@ri_1 \oplus ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x) \oplus (ri_2\backslash_b x)) && (\mathsf{B}_+)\\
(bs@ri_1\, ri_2)\backslash_b x &= (bs@(ri_1\backslash_b x)\, ri_2) && (\mathsf{B}_{\text{seq}}),\ \varepsilon \notin L(ri_1)\\
(bs@ri_1\, ri_2)\backslash_b x &= bs@\Big(([\,]@(ri_1\backslash_b x)\, ri_2) \oplus \mathit{fuse}(\mathit{mkEpsBC}_{ri_1}, ri_2\backslash_b x)\Big) && (\mathsf{B}^n_{\text{seq}}),\ \varepsilon \in L(ri_1)\\
(bs@ri^*)\backslash_b x &= (bs@\mathit{fuse}([0], ri\backslash_b x)\, ([\,]@ri^*)) && (\mathsf{B}_*).
\end{aligned}
\tag{5.10}
$$

The rules B_+, B_{seq}, and B_* preserve the outer bs, since derivation changes the residual but not the choices already recorded above it.

The nullable sequence case $\mathsf{B}^n_{\text{seq}}$ is the exception. Since ri_1 is nullable, x may be matched by either factor, so the rule introduces a new alternation. Its left branch continues into ri_1, while its right branch recurses into ri_2, with the empty match's bits extracted by $\mathit{mkEpsBC}$ and fused onto ri_2's derivative, so the choice inside ri_1 survives. This performs during the forward pass what $\mathsf{I}^R_{\text{seq}}$ performs during injection. The outer bs attaches to the new alternation, since the alternation as a whole is what $ri_1\, ri_2$ derived into.

In Rust. Figure 5.10 implements (5.10), following the case structure of plain `deriv` with bits threaded through the constructors. The base cases `Phi`, `Eps`, and `Lit` match plain `deriv`, except that `Lit` carries its bit-list onto the resulting `Eps`.

The `Alt` arm derives both operands and rebuilds the alternation with the outer `bs` on the wrapper, not pushed into either branch. This matches B_+, where the recorded choice belongs to the alternation itself and derivation does not change it, so the bit stays on the outer constructor until the decoder reads it.

The `Seq` arm reflects B_{seq} and $\mathsf{B}^n_{\text{seq}}$. In the non-nullable case, `r1` must consume the character, so only its derivative is needed and `r2` moves directly into the rebuilt sequence. In the nullable case, both alternatives are possible. The bits of `r1`'s empty match are extracted by `mk_eps_bc` *before* `r1` is consumed by `deriv_bc`, since after that call the original `r1` is gone. Those bits are then fused onto the derivative of `r2`, so the choice made inside `r1` reaches the surviving branch. The left branch is wrapped in a fresh `Seq` with an empty bit-list, since the outer `bs` moves to the new `Alt` wrapper, matching $\mathsf{B}^n_{\text{seq}}$. Two clones are therefore needed. `r2.clone()` provides one copy for the left branch and one for its own derivative, and `nullable_bc` borrows `r1` before `deriv_bc` consumes it.

The `Star` arm follows the same pattern as the nullable case of `Seq`. The body is needed both for its derivative and for the reconstructed star, so `r.clone()` provides the second copy, and the reconstructed star is wrapped in a fresh `Star` with an empty bit-list for the same reason as the left branch above. The leading `[false]` prepended to the derivative marks one more iteration, matching the outermost bit in B_*

```
pub fn deriv_bc(ri: ARegex, l: char) -> ARegex {
    match ri {
        // ...
        ARegex::Alt(bs, r1, r2) => {
            let d1 = deriv_bc(*r1, l);
            let d2 = deriv_bc(*r2, l);
            ARegex::Alt(bs, Box::new(d1), Box::new(d2))
        }
        ARegex::Seq(bs, r1, r2) => {
            if nullable_bc(&r1) {
                let eps_bits = mk_eps_bc(&r1);
                let d1 = deriv_bc(*r1, l);
                let d2 = deriv_bc(*r2.clone(), l);
                let left_branch = ARegex::Seq(Bits::empty(), Box::new(d1), r2);
                let right_branch = fuse(&eps_bits, d2);
                ARegex::Alt(bs, Box::new(left_branch), Box::new(right_branch))
            } else {
                let d1 = deriv_bc(*r1, l);
                ARegex::Seq(bs, Box::new(d1), r2)
            }
        }
        ARegex::Star(bs, r) => {
            let d = deriv_bc(*r.clone(), l);
            let fused = fuse(&[false], d);
            let new_star = ARegex::Star(Bits::empty(), r);
            ARegex::Seq(bs, Box::new(fused), Box::new(new_star))
        }
    }
}
```

Figure 5.10: Annotated derivative `deriv_bc`, implementing $\mathsf{B}_{\emptyset}$, B_{ε}, B_{lit}, B_{+}, $\mathsf{B}_{\text{seq}}/\mathsf{B}^{n}_{\text{seq}}$, B_{*}.

5.2.6 Simplification

simp, from [Sulzmann and Lu, 2014, Fig. 6], keeps the annotated expression's size bounded as derivation proceeds: it discards dead ($\emptyset$) branches, collapses an ε operand out of a concatenation, and removes duplicate alternation branches, fusing (§5.2.2) each removed node's bits onto whatever replaces it. A single bottom-up pass does not always reach a fixpoint – collapsing a branch at one level can expose a new collapse at the level above – so *simp* re-applies itself until the shape stops changing.

In Rust. Figure 5.11 shows `simp_once`'s `Seq` and `Alt` cases, the two that do real work; `Phi`, `Eps`, and `Lit` are already simplified, and `Star` only recurses into its body. `simp` is the fixpoint loop around it.

```
pub fn simp(ri: ARegex) -> ARegex {
    let mut cur = ri;
    loop {
        let next = simp_once(cur.clone());
        if same_shape(&next, &cur) { return next; }
        cur = next;
    }
}

fn simp_once(ri: ARegex) -> ARegex {
    match ri {
        ARegex::Phi | ARegex::Eps(_) | ARegex::Lit(_, _) => ri,

        ARegex::Seq(bs, r1, r2) => {
            let (r1s, r2s) = (simp_once(*r1), simp_once(*r2));
            if is_phi(&r1s) || is_phi(&r2s) { return ARegex::Phi; }
            if let ARegex::Eps(bs2) = r1s {
                return fuse_bits(&bs, fuse_bits(&bs2, r2s));  // eps.r -> r
            }
            if let ARegex::Eps(bs2) = &r2s {
                if bs2.is_empty() { return fuse_bits(&bs, r1s); } // r.eps -> r
            }
            ARegex::Seq(bs, Box::new(r1s), Box::new(r2s))
        }

        ARegex::Alt(bs, r1, r2) => {
            // flatten nested Alt, drop dead branches, dedup by shape
            let mut branches: Vec<ARegex> = collect_alt_branches(*r1, *r2)
                .into_iter().map(simp_once).filter(|b| !is_phi(b)).collect();
            dedup_first(&mut branches);
            match branches.len() {
                0 => ARegex::Phi,
                1 => fuse_bits(&bs, branches.remove(0)),
                _ => fuse_bits(&bs, fold_alt(branches)),
            }
        }

        ARegex::Star(bs, r) => ARegex::Star(bs, Box::new(simp_once(*r))),
    }
}
```

Figure 5.11: `simp` and `simp_once`: the fixpoint loop and its two non-trivial cases. `collect_alt_branches`, `dedup_first`, and `fold_alt` implement flattening, deduplication, and rebuilding an n-ary alternation, omitted here for space; each does no more than its name says. `fuse_bits` is `fuse`'s (§5.2.2) counterpart for the native `Bits` type, avoiding a conversion back to a slice that would force the entire accumulated history to be materialized.

The `Seq` case mirrors $\varepsilon \cdot r = r$ and $r \cdot \varepsilon = r$, but asymmetrically: the right-hand collapse only fires when r_2's own bits are empty, since *mkEpsBC* concatenates bits left to right, so fusing r_2's bits onto r_1 the same way the left case does would place them before r_1's own bits instead of after. `same_shape`, used by both the fixpoint check and `dedup_first`,

compares expressions while ignoring their bit annotations: two branches denoting the same future language collapse into one even if they recorded different choices so far, the same exclusion the reference's *nub2* makes and that §6.2.3 relies on for the bit-coded partial-derivative parser. Comparing by shape alone also keeps the fixpoint check itself cheap: the derived structural equality would otherwise force a full bit-list materialization at every node on every iteration, reintroducing the $O(n^2)$ cost `Bits` (§5.2.1) exists to avoid.

5.2.7 Decoding Bits

$decode_r$ reconstructs a `ParseTree` from the bit sequence of $mkEpsBC$, walking r with one bit per alternation to choose a branch. Its worker $decodeInner$ returns the current tree and the remaining bits, so the recursion continues at the correct position:

$$
\begin{aligned}
decode_r(bs) &= v \quad \text{if } decodeInner_r(bs) = (v, [\,]) \\
\\
decodeInner_\varepsilon(bs) &= (Empty,\ bs) \\
decodeInner_x(bs) &= (Char(x),\ bs) \\
decodeInner_{r_1+r_2}(0{:}bs) &= (Left(v),\ bs_1) \quad \text{if } decodeInner_{r_1}(bs) = (v, bs_1) \\
decodeInner_{r_1+r_2}(1{:}bs) &= (Right(v),\ bs_1) \quad \text{if } decodeInner_{r_2}(bs) = (v, bs_1) \\
decodeInner_{r_1 r_2}(bs) &= (Pair(v_1, v_2),\ bs_2) \\
&\qquad \text{if } decodeInner_{r_1}(bs) = (v_1, bs_1) \text{ and } decodeInner_{r_2}(bs_1) = (v_2, bs_2) \\
decodeInner_{r^*}(1{:}bs) &= (Star([\,]),\ bs) \\
decodeInner_{r^*}(0{:}bs) &= (Star(v{:}vs),\ bs_2) \\
&\qquad \text{if } decodeInner_r(bs) = (v, bs_1) \text{ and } decodeInner_{r^*}(bs_1) = (Star(vs), bs_2)
\end{aligned}
\tag{5.11}
$$

In Rust. Figure 5.12 shows `decode_inner`, the worker behind `decode`. The public entry point `decode` wraps it and asserts that no bits remain. This assertion is the side condition under which $decode_r$ is defined. The worker must consume the entire bit-string.

The worker consumes the `&[bool]` one bit per step using Rust's slice patterns: `[false, rest @ ..]` matches a bit-string whose first element is `false` and binds the remainder to `rest`, and `[true, rest @ ..]` does the same for `true`.

```
fn decode_inner<'a>(r: &Regex, bs: &'a [bool]) -> (ParseTree, &'a [bool]) {
    match r {
        Regex::Eps => (ParseTree::Empty, bs),
        Regex::Lit(c) => (ParseTree::Char(*c), bs),
        Regex::Alt(r1, r2) => match bs {
            [false, rest @ ..] => {
                let (v, remaining) = decode_inner(r1, rest);
                (ParseTree::Left(Box::new(v)), remaining)
            }
            [true, rest @ ..] => {
                let (v, remaining) = decode_inner(r2, rest);
                (ParseTree::Right(Box::new(v)), remaining)
            }
            [] => panic!("decode: unexpected end of bits at Alt"),
        },
        Regex::Seq(r1, r2) => {
            let (v1, after_r1) = decode_inner(r1, bs);
            let (v2, after_r2) = decode_inner(r2, after_r1);
            (ParseTree::Pair(Box::new(v1), Box::new(v2)), after_r2)
        }
        Regex::Star(r1) => {
            let mut iterations = Vec::new();
            let mut remaining = bs;
            loop {
                match remaining {
                    [true, rest @ ..] => {
                        remaining = rest;
                        break;
                    }
                    [false, rest @ ..] => {
                        let (v, after) = decode_inner(r1, rest);
                        iterations.push(v);
                        remaining = after;
                    }
                    [] => panic!("decode: unexpected end of bits inside Star"),
                }
            }
            (ParseTree::Star(iterations), remaining)
        }
        Regex::Phi => panic!("decode: called on Phi"),
    }
}
```

Figure 5.12: `decode_inner`: the worker behind `decode`, one arm per expression form.

The `Alt` arm matches the leading bit with `[false, rest @ ..]` or `[true, rest @ ..]` and recurses into the branch it names, returning `rest` as the remaining bits, since only the leading bit belongs to the alternation.

The `Seq` arm shows why the worker returns a pair. It recurses into the first factor, receives both a tree and the bits the first factor left unconsumed, and passes those bits to the second factor. The second factor then returns its own tree and the bits that follow

it, which are the bits for any enclosing expression. Returning only the tree would leave the second recursion with no way to know where to start.

The `Star` arm loops, since the number of iterations is not known in advance and is itself encoded in the bits. Each leading `false` is matched by `[false, rest @ ..]` and corresponds to one iteration, so the arm recurses into the body and continues the loop from whatever bits the body left behind. The loop ends at the leading `true` matched by `[true, rest @ ..]`, which `mk_eps_bc` appended.

5.2.8 Parser

The bit-coded parser performs three steps on the input. First, *internalize* converts the pattern r into an annotated expression ri. Then, for each input character c, the parser takes the annotated derivative $ri\backslash_b c$ and simplifies it, replacing ri with $simp(ri\backslash_b c)$. After the last character, the parser checks whether the final ri is nullable. If it is, the parser extracts the bits with $mkEpsBC$ and decodes them against the original r:

$$\begin{aligned} ri_0 &= internalize(r), \\ ri_i &= simp(ri_{i-1}\backslash_b x_i) \quad (1 \le i \le n), \\ result &= \begin{cases} decode_r(mkEpsBC(ri_n)) & n(ri_n) \\ \text{no match} & \text{otherwise.} \end{cases} \end{aligned} \tag{5.12}$$

The loop holds only the current ri. Unlike the standard derivative parser, which keeps a growing vector of residuals for the backward reconstruction, the bit-coded parser needs no history. The bits record everything needed to decode, so the forward pass can discard each intermediate expression as soon as the next one is computed.

In Rust. Figure 5.13 implements (5.12).

```
fn derive_and_simplify(ri: ARegex, chars: &[char]) -> ARegex {
    let mut cur = ri;
    for &l in chars { cur = simp(deriv_bc(cur, l)); }
    cur
}

pub fn parse_bitcoded_loop(input: &str, r: &Regex) -> Option<ParseTree> {
    let ri = internalize(r);
    let chars: Vec<char> = input.chars().collect();
    let final_ri = derive_and_simplify(ri, &chars);
    if !nullable_bc(&final_ri) { return None; }
    Some(decode(r, &mk_eps_bc(&final_ri)))
}
```

Figure 5.13: `parse_bitcoded_loop`: the bit-coded parser's driver. `derive_and_simplify` is factored out because the recursive entry point (§5.1.3's counterpart for this parser) shares the same per-character stepping and differs only in how it reaches `ri` in the first place.

The loop matches the three cases of (5.12) one for one. `internalize` initialises `ri`, `derive_and_simplify` applies `simp(deriv_bc(ri, l))` to each character and the nullability check gates the final `decode(mk_eps_bc(ri))`.

5.2.9 Worked Example: Applying the Bitcoded Parser

Consider the same input ab and regular expression $r_0 = (a + ab)(b + \varepsilon)$ as in Section 5.1.4.

Internalize. Before consuming input, *internalize* tags each alternative with [0] for the left branch and [1] for the right branch:

$$\mathit{internalize}(r_0) = ri_0 = [\,]@\Big([\,]@([0]@a \oplus [1]@(a\,b)) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big). \tag{5.13}$$

Bit-Threaded Derivatives. The two input symbols are consumed by successive bit-threaded derivatives with *simp* applied after each step. The first derivative uses B_{seq}. After consuming a, the first factor becomes nullable, so the second derivative uses $\mathsf{B}^n_{\text{seq}}$:

$$\begin{aligned}
ri_0 &\xrightarrow[\mathsf{B}_{\text{seq}}]{a} ri_1 = [\,]@\Big([\,]@([0]@\varepsilon \oplus [1]@b) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)\Big) \\
ri_1 &\xrightarrow[\mathsf{B}^n_{\text{seq}}]{b} ri_2' = [\,]@\Big(\underbrace{[\,]@(\emptyset \oplus [1]@\varepsilon) \cdot [\,]@([0]@b \oplus [1]@\varepsilon)}_{X} \oplus \underbrace{[0]@\Big([0]@\varepsilon \oplus \emptyset\Big)}_{Y} \Big) \\
&\xrightarrow{\mathit{simp}} ri_2 = [\,]@\Big([1]@([0]@b \oplus [1]@\varepsilon) \oplus [0,0]@\varepsilon\Big).
\end{aligned} \tag{5.14}$$

Each disjunct of ri_2' has a dead branch. In X, the first factor's derivative by b is $\emptyset \oplus [1]@\varepsilon$: the [0] side cannot consume b, the [1] side owes one. In Y, the second factor's derivative by b is $[0]@\varepsilon \oplus \emptyset$: the [0] side matches, the [1] side cannot. The outer [0] on Y is the bit that $\mathsf{B}^n_{\text{seq}}$ fused on from the first factor's empty match.

simp drops the dead branches. From X, the survivor is $[1]@\varepsilon$, and since ε contributes nothing to the concatenation, the [1] is carried onto the second factor, giving $[1]@([0]@b \oplus [1]@\varepsilon)$. From Y, the survivor is $[0]@\varepsilon$; the outer [0] is merged with the inner [0] and the resulting $[0,0]$ is carried onto the ε, giving $[0,0]@\varepsilon$. The two results are the disjuncts of ri_2, so both bits are recorded there and the tree can later be decoded from the bits alone.

Extracting the bits. Both disjuncts of ri_2 are nullable, so EB^L_+ selects the left one. Inside, b is not nullable while ε is, so EB^R_+ contributes its [1] tag:

$$\mathit{mkEpsBC}(ri_2) \xrightarrow{\mathsf{EB}^L_+} \mathit{mkEpsBC}\Big([1]@([0]@b \oplus [1]@\varepsilon)\Big) \xrightarrow{\mathsf{EB}^R_+} [1,1]. \tag{5.15}$$

Unlike *mkEps*, which constructs a partial parse tree, *mkEpsBC* extracts only the committed choices as a bit-string.

Decoding the bits. The decoder interprets $[1,1]$ against the original r_0: the first bit selects the right branch of $a + ab$ (producing ab), and the second selects the right branch of $b + \varepsilon$:

$$\mathit{decode}_{r_0}([1,1]) = \Big(\mathit{Right}(a,b), \mathit{Right}(\,)\Big) = v_0. \tag{5.16}$$

The decoder consumes the bits left to right, one per alternation it encounters, in the same order that *internalize* introduced them. Because *mkEpsBC* preserved that order when it fused and prepended bits, the first bit always belongs to the outermost alternation, and so on down.

Flattening gives $\mathit{flatten}(v_0) = a \cdot b \cdot \varepsilon = \texttt{ab}$, confirming the parse. The bit-coded construction produces the same tree as the direct construction, but stores only ri_2, a single expression bounded by the pattern instead of one residual per character. That bound is on the live expression's size, not on total memory: each node's bit-sequence is itself built by the persistent, `Rc`-backed cons list of Section 5.2.1, so holding one bounded expression across an n-character run still allocates cumulatively in n, it just never holds more than one expression's worth at once. Chapter 8's *total* and *max peak* columns for `deriv_bc` separate exactly this: a peak bounded by the pattern next to a total that still grows with how much input it consumed.

6 Partial-Derivative-Based Parser

Chapter 5 presented two derivative-based methods for constructing POSIX parse trees. The standard parser stores the derivation sequence and reconstructs the tree in a backward pass using *mkEps* and the injections. The bit-coded parser records the choices as bits during the forward pass and decodes them against the original expression at the end.

This chapter presents the partial-derivative alternative of Sulzmann and Lu [Sulzmann and Lu, 2012]. Instead of one residual per step, the partial derivative produces a set of states, each pairing a residual expression with the reconstruction information for one path through the original expression. The parser maintains this set as an NFA state set, and at the end walks it in generation order, taking the first accepting state. Because partial derivatives break apart the structure of the original expression, this walk yields the leftmost alternative at every choice rather than the longest one, and therefore a non-POSIX parse tree.

As in Chapter 5, two variants are presented. The standard parser attaches an injection closure to each state. The bit-coded parser replaces these closures with bit-sequences. Both compute the same parse tree.

6.1 The Standard Partial-Derivative Parser

Consider again $r_0 = (a + ab)(b + \varepsilon)$ with input $w = \mathtt{ab}$. Where *deriv* reduces r_0 to a single residual at each step, Antimirov's partial derivative *pderiv* reduces it to a *set* of states, one per way the next character can be consumed. Each state pairs a residual expression with an injection that rebuilds a parse tree for r_0 from a parse tree for that residual. The injection is attached to its state as soon as the state is created, rather than recovered afterwards from a stored sequence the way *inj* was for the standard derivative parser in Section 5.1.

Deriving r_0 by $\mathtt{a}$ produces two states, one for each way $\mathtt{a}$ can be consumed:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}) = \big\{ &(\ \varepsilon \cdot (b + \varepsilon), \ \lambda(_, v_2).\ (\mathit{Left}\, a, \ v_2) \ ), \\ &(\ (\varepsilon \cdot b)\,(b + \varepsilon), \ \lambda((v_1, v_2), v_2').\ (\mathit{Right}(a, v_2), \ v_2') \ ) \big\}. \end{aligned} \tag{6.1}$$

A state is a pair: a residual expression describing what is left to match, and an injection rebuilding a parse tree of the original expression from a parse tree of the residual. The two states above correspond to the two branches of the alternation $a + ab$ in the first factor. The second factor $b + \varepsilon$ is untouched in both and appears as the trailing component of each residual.

Each residual has the shape $r' \cdot (b + \varepsilon)$: the derivative of the first factor, followed by the untouched second factor, never simplified. The leading ε in each r' records that the consumed character matched a leading literal and left nothing behind in that position; it is not removed, because the injection below pattern-matches on exactly this pair shape.

The first state's residual $\varepsilon \cdot (b + \varepsilon)$ is a sequence, so its tree is a pair (v_1, v_2): v_1 is the tree of the leading ε, which is necessarily *Empty*, and v_2 is the tree of the untouched second factor $b + \varepsilon$. The closure returns $(\mathit{Left}(a), v_2)$. It discards v_1 because the tree of ε carries no information, and replaces the first position with $\mathit{Left}(a)$ to record that the left branch of the alternation $a + ab$ was taken. The second factor's tree v_2 is passed through unchanged.

The second state's residual $(\varepsilon \cdot b)\,(b + \varepsilon)$ is a sequence whose first component is itself a sequence, so its tree is a pair $((v_1, v_2), v_2')$: the inner pair (v_1, v_2) is the tree of $\varepsilon \cdot b$, with v_1 the tree of the leading ε and v_2 the tree of the trailing b; the outer v_2' is the tree of the untouched second factor. The closure returns $(\mathit{Right}(a, v_2), v_2')$. It rebuilds the consumed a (which the leading ε records) and the still-owed b (which v_2 records) as the pair (a, v_2), places that pair inside a right injection to record that the right branch ab of the alternation was taken, and passes the second factor's tree v_2' through unchanged. The binder v_1 is bound but not used in the body, for the same reason as the first state's `_`: the tree of ε carries no information.

Transitioning both states on the second character `b` gives a new state set:

$$\begin{aligned} \mathit{pderiv}(r_0, \mathtt{a}, \mathtt{b}) = \big\{&(\varepsilon,\ \lambda_.\ (\mathit{Left}\, a,\ \mathit{Left}\, b)\ ),\\ &(\varepsilon \cdot (b + \varepsilon),\ \lambda(_, v_2).\ (\mathit{Right}(a, b),\ v_2)\ )\big\}. \end{aligned} \tag{6.2}$$

The first state now has residual ε, meaning every input character has been consumed, and is accepting; the second has residual $\varepsilon \cdot (b + \varepsilon)$ – the second factor is untouched by `b`, since it was the first factor's leftover b that matched – which is also accepting. Selection walks the state set in generation order and returns the first accepting state. Applying its injection to $\mathit{mkEps}_{\varepsilon} = ()$ yields $(\mathit{Left}\, a, \mathit{Left}\, b)$, the leftmost alternative at every choice (Figure 6.1).

Two properties of the state set determine which tree is selected. First, the order of states in the set encodes the preference. When a state produces several successors, they are appended in the order the rules generate them, with the left alternative first, so the leftmost path appears earliest. Second, when two states share a residual, *nub* keeps the first and discards the later one with its injection, so the earlier path wins ties – the frontier-wide deduplication rule stated as Definition 8 of [Sulzmann and Lu, 2012]. Both mechanisms favour the leftmost alternative at every choice over the longest one, which is what distinguishes the tree this parser returns from the POSIX tree of Chapter 5.

6.1.1 Partial Derivatives with Injections

A partial derivative of an expression r with respect to a character x is not a single expression but a set of pairs. Each pair consists of a residual expression r' whose language is one component of the left quotient, together with an injection closure ι that maps a parse tree for r' back to a parse tree for r. Stripped of the injection, the residual side of this definition is exactly *pd*, the Antimirov partial derivative already used by the matcher of

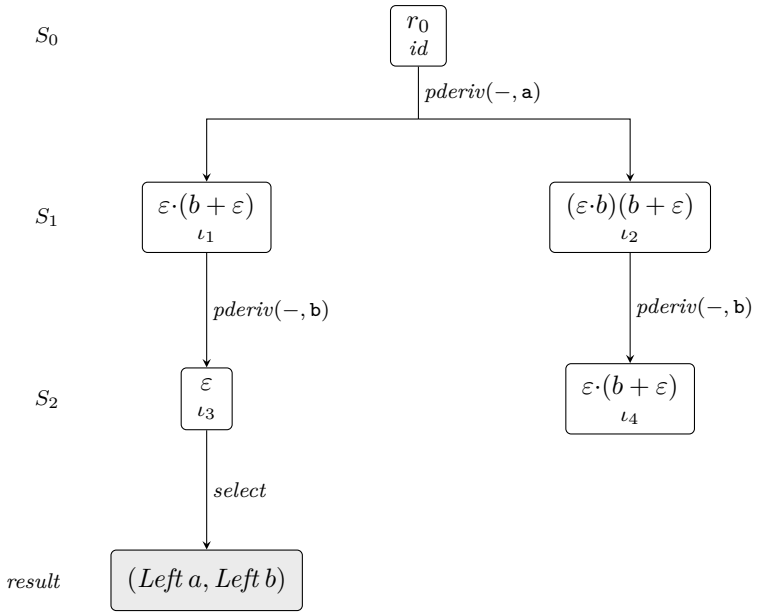


Figure 6.1: The standard partial-derivative parser on $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a state (r, ι): a residual and an injection, both shown without any simplification. $pderiv(-, x)$ advances every state by one character; *select* takes the first accepting state and applies its injection to *mkEps* of its residual.

Section 3.3 [Antimirov, 1996], so *pderiv* reuses its rule names $\mathsf{P}_\emptyset, \mathsf{P}_\varepsilon, \mathsf{P}_{\text{lit}}, \mathsf{P}_+, \mathsf{P}_{\text{seq}}, \mathsf{P}^n_{\text{seq}}, \mathsf{P}_*$ from Equation 3.14, each now additionally threading an injection alongside its residual. The idea of pairing a residual with a reconstruction function is, in turn, in the style of Sulzmann and Lu's pattern partial derivative [Sulzmann and Lu, 2012, Fig. 11], which pairs a residual *pattern* with a function building a variable-binding environment; here the same idea builds a *ParseTree* instead.

The rules that do not depend on whether their operand is nullable are:

$$
\begin{aligned}
pderiv(\emptyset, x) &= \{\} && (\mathsf{P}_\emptyset)\\
pderiv(\varepsilon, x) &= \{\} && (\mathsf{P}_\varepsilon)\\
pderiv(y, x) &= \begin{cases} \{(\varepsilon,\ \lambda_.\, y)\} & x = y \\ \{\} & x \neq y \end{cases} && (\mathsf{P}_{\text{lit}})\\
pderiv(r + s, x) &= \{\, (r',\ \lambda v.\, Left\,(\iota\, v)) \mid (r', \iota) \in pderiv(r, x) \,\}\\
&\quad \cup\ \{\, (s',\ \lambda v.\, Right\,(\iota\, v)) \mid (s', \iota) \in pderiv(s, x) \,\} && (\mathsf{P}_+)\\
pderiv(r^*, x) &= \{\, (r'\, r^*,\ \lambda(v, Star(vs)).\, Star([\iota\, v] +\!\!+ vs)) \mid (r', \iota) \in pderiv(r, x) \,\} && (\mathsf{P}_*)
\end{aligned}
\tag{6.3}
$$

exactly as in *pd*, with the recursive call's injection wrapped by *Left*, *Right*, or prepended to the iteration list. No rule is needed for $\emptyset$ or ε beyond their empty results, since neither can consume a character.

Concatenation alone must branch on $n(r)$, so it is shown separately:

$$
pderiv(r \cdot s, x) = \begin{cases}
\{\, (r'\, s,\ \lambda(v_1, v_2).\, (\iota\, v_1,\ v_2)) \mid (r', \iota) \in pderiv(r, x) \,\} & n(r) \quad (\mathsf{P}^n_{\text{seq}})\\
\cup\ \{\, (s',\ \lambda v.\, (mkEps_r,\ \iota\, v)) \mid (s', \iota) \in pderiv(s, x) \,\} & \\
\{\, (r'\, s,\ \lambda(v_1, v_2).\, (\iota\, v_1,\ v_2)) \mid (r', \iota) \in pderiv(r, x) \,\} & \neg\, n(r) \quad (\mathsf{P}_{\text{seq}})
\end{cases}
\tag{6.4}
$$

After expansion, each rule's result is deduplicated by residual, keeping the first occurrence; this is *nub*, applied wherever a transition can produce two states with the same residual, namely P_+, $\mathsf{P}^n_{\text{seq}}$, and P_*. P_{seq} has only one source of states and needs no deduplication of its own, since its single recursive call is already duplicate-free. This is the step that breaks apart the original expression: where the derivative of $r + s$ produces a single alternation, the partial derivative produces separate states for each branch, and the outer alternation disappears from the residual.

Rule (P_{seq}) – the only one present regardless of nullability – passes the outer pair through unchanged, with the recursive injection applied to the first component only. When r is nullable, rule ($\mathsf{P}^n_{\text{seq}}$) additionally unions in the states that skip r entirely, using $mkEps_r$ to synthesise its empty match for the first component. Rule (P_+) wraps the recursive injection in a *Left* or *Right* node, reinserting the alternation choice. Rule (P_*) reconstructs the current iteration by applying the recursive injection and prepending it to the remaining iterations.

Unlike *pd*, which applies the smart constructor *smartC* in P_{seq} and P_* to avoid building $\varepsilon \cdot s$ (Equation 3.15), *pderiv* never simplifies: $r' \cdot s$ above is built with the plain pairing constructor, even when $r' = \varepsilon$. A simplified residual would collapse exactly the pair-shaped node that the injection above pattern-matches on, so *pderiv* trades the smaller residuals *pd* produces for residuals whose shape the injection can always follow. This is the same tradeoff the standard derivative parser makes by never simplifying (Section 5.1), for the same reason: *inj* there pattern-matches on the derivative's structure just as ι does here.

In Rust. The recursive definition in Equations 6.3 and 6.4 is implemented directly in Rust, with two departures from a literal reading of the equations, both for performance rather than semantics.

First, `pderiv_tree` itself builds only *one* transition's closure, type `InjStep`, a reference-counted closure exactly as before. The accumulated injection a state carries across many characters – what the equations call ι once composition has happened several times – is a separate type, `Inj`, described in Section 6.1.2: a persistent list of `InjStep`s rather than a nested closure, so that applying or dropping a long-lived `Inj` does not recurse one native stack frame per character.

Second, the `Seq` and `Star` arms use `smart_seq` – the Rust counterpart of *smartC* used by `pderiv` in Figure 3.7 – wherever the equations build $\varepsilon \cdot s$, including `Seq`'s nullable branch's continue-strand. Collapsing $\varepsilon \cdot s$ to s changes the shape the next transition's closure must match against, so each arm that uses `smart_seq` branches on whether the residual collapsed (`r1_ == Regex::Eps`) and builds the closure for that shape specifically, rather than assuming a `Pair` is always present. Without this, two strands that independently derive to the same residual but reached it through different wrapper shapes would not dedup against each other, and the frontier can grow unboundedly on an ambiguous pattern even though the number of distinct residuals stays bounded.

```
pub(crate) type InjStep = Rc<dyn Fn(ParseTree) -> ParseTree>;

pub(crate) fn pderiv_tree(r: &Regex, x: char) -> Vec<(Regex, InjStep)> {
    match r {
        Regex::Eps | Regex::Phi => vec![],

        Regex::Lit(c) => {
            if *c == x {
                vec![(Regex::Eps,
                      Rc::new(move |_| ParseTree::Char(x))) as InjStep]
            } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, InjStep)> = pderiv_tree(r1, x)
                .into_iter()
                .map(|(r1_, inj1)| {
                    let f: InjStep = Rc::new(move |v| {
                        ParseTree::Left(Box::new(inj1(v)))
                    });
                    (r1_, f)
                })
                .collect();
            out.extend(pderiv_tree(r2, x).into_iter().map(|(r2_, inj2)| {
                let f: InjStep = Rc::new(move |v| {
                    ParseTree::Right(Box::new(inj2(v)))
                });
                (r2_, f)
            }));
            nub_tree(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, InjStep)> = pderiv_tree(r1, x)
                    .into_iter()
                    .map(|(r1_, inj1)| {
                        let r2c = (**r2).clone();
                        // smart_seq collapse, same split as below.
                        if r1_ == Regex::Eps {
                            let f: InjStep = Rc::new(move |v2| {
                                let v1 = inj1(ParseTree::Empty);
                                ParseTree::Pair(Box::new(v1), Box::new(v2))
                            });
                            (r2c, f)
                        } else {
                            let f: InjStep = Rc::new(move |v| match v {
                                ParseTree::Pair(v1, v2) =>
                                    ParseTree::Pair(Box::new(inj1(*v1)), v2),
                                _ => panic!("pderiv_tree/Seq: want Pair"),
                            });
                            (smart_seq(r1_, &r2c), f)
                        }
                    })
                    .collect();

                let v1_eps = mk_eps(r1);
```

The `Phi`, `Eps`, and `Lit` arms implement $\mathsf{P}_\emptyset$, P_ε, and P_{lit} directly, matching the same arms of `pderiv` in Figure 3.7, except that each returns a closure alongside its residual instead of nothing. The `Alt` arm implements P_+ and is where the partial derivative visibly breaks apart the expression: each branch produces its own states, tagged by *Left* or *Right* in the injection, then deduplicated by `nub_tree`.

The `Seq` arm's `if nullable(r1)` matches the case split between $\mathsf{P}^n_{\text{seq}}$ and P_{seq}. The nullable branch's continue-strand is built exactly as the equations describe, with the plain pairing constructor, since the skip-strand alongside it needs `r1`'s own closure to still expect a `Pair`; the nullable branch additionally computes `r1`'s empty tree once with `mk_eps` and captures a clone of it in every skip-strand closure, then deduplicates the combined list. The non-nullable branch, by contrast, does use `smart_seq`, with the closure split on whether the residual collapsed to `Eps`, as explained above.

The `Star` arm implements P_*, with the closure appending the reconstructed iteration to the star's list (left-to-right order, reversed once at the end by `select` rather than paying `insert(0, ..)`'s cost on every iteration), again split on whether `smart_seq` collapsed the residual.

6.1.2 State Sets and Selection

The parser maintains a state set of (residual, injection) configurations. For each input character, every configuration in the state set is transitioned with `pderiv_tree`, and the resulting configurations are collected into a new state set, replacing the old one – unlike the standard derivative parser, nothing from earlier state sets is retained once the next one has been computed. A configuration's parent injection is composed with the transition injection, so that at the end the state set's injections map residuals all the way back to the original expression.

Conceptually the composition is direct: when a parent configuration (r, inj_p) transitions to (r', inj_s), the new configuration's injection is $\lambda v.\, inj_p(inj_s(v))$. Composing this way as nested closures, however, makes both calling the result and dropping it recurse one native stack frame per character – fine at tens of thousands of characters, a real stack-overflow risk beyond that. The `Inj` type instead stores a configuration's accumulated steps as a persistent cons-list and applies them in an explicit loop:

```
enum InjNode { Identity, Step(InjStep, Rc<InjNode>) }
pub(crate) struct Inj(Rc<InjNode>);

impl Inj {
    pub(crate) fn then(&self, step: InjStep) -> Self {
        Inj(Rc::new(InjNode::Step(step, self.0.clone())))
    }
    pub(crate) fn apply(&self, v: ParseTree) -> ParseTree {
        let mut v = v;
        let mut cur = &self.0;
        loop {
            match &**cur {
                InjNode::Identity => return v,
                InjNode::Step(step, rest) => { v = step(v); cur = rest; }
```

```
            }
        }
    }
}
```

`then` adds one more character's step in O(1), just wrapping the existing chain in a new node, the same way `Bits::prepend` (Section 5.2.2) wraps an existing bit chain; `apply` walks the whole chain in one iterative pass, applying each step in the order it was added, with no recursive call regardless of how long the chain is. `Inj` also has its own iterative `Drop`, for the same reason `Bits`'s does: the derived recursive drop glue would walk the chain one stack frame per character too.

```
pub(super) fn step_frontier(
    frontier: Vec<(Regex, Inj)>, x: char
) -> Vec<(Regex, Inj)> {
    let mut out = Vec::new();
    for (r, inj_parent) in frontier {
        for (r_next, inj_step) in pderiv_tree(&r, x) {
            out.push((r_next, inj_parent.then(inj_step)));
        }
    }
    nub_tree(out)
}
```

Figure 6.3: `step_frontier`: one transition of the NFA state set on a single character. The function name retains the older "frontier" terminology; it computes the successor state set $\delta(S, x)$. The final `nub_tree` deduplicates *across* parent configurations, not just within one parent's own expansion (which `pderiv_tree` already does on its own output) – without it, two strands that independently derive to the same residual both survive, and the state set can double every character on an ambiguous pattern even though the number of distinct residuals stays bounded by the pattern's own size.

After the input is exhausted, selection walks the state set in priority order and returns the tree for the first accepting configuration:

```
pub(super) fn select(frontier: &[(Regex, Inj)]) -> Option<ParseTree> {
    for (r, inj) in frontier {
        if nullable(r) {
            return Some(reverse_stars(inj.apply(mk_eps(r))));
        }
    }
    None
}
```

Figure 6.4: `select`: return the tree for the first accepting configuration in the state set. `reverse_stars` undoes, in one O(total output size) pass, the left-to-right-via-push construction order `pderiv_tree`'s `Star` arm uses (Figure 6.2).

Because the state set lists configurations in the order the rules generated them, and because `nub_tree` keeps the first of each duplicate residual, selection favours the configuration that committed earliest to the leftmost alternative. This is the non-POSIX leftmost

policy. Unlike the standard derivative parser, there is no separate backward pass here: by the time `select` runs, every configuration's injection is already the full composition back to the original expression, so selection only has to apply it once.

6.1.3 Parser

The parser has a single pass: every input character advances the whole state set at once via *step_frontier*, and the final state set is handed directly to *select*:

$$S_0 = \{(r, id)\}, \qquad S_i = step_frontier(S_{i-1}, x_i) \;\; (1 \le i \le n), \qquad result = select(S_n). \tag{6.5}$$

Unlike the standard derivative parser's two-pass driver (Equation 5.4), no backward pass is needed here: each configuration in S_i already carries its injection composed all the way back to r_0 (Section 6.1.2), so *select* only ever applies one injection, to the single accepting configuration it finds in S_n.

In Rust. Figure 6.5 implements (6.5).

```
pub fn parse_pderiv_std(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier: Vec<(Regex, Inj)> =
        vec![(r.clone(), Inj::identity())];
    for c in input.chars() {
        frontier = step_frontier(frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    select(&frontier)
}
```

Figure 6.5: The standard partial-derivative parser.

The loop computes $S_0, \ldots, S_n$ in place, overwriting `frontier` on every iteration rather than appending to a growing `Vec` the way `parse_deriv_std_loop` (Figure 5.4) does – nothing from an earlier state set is retained once the next one exists. The early return on an empty frontier corresponds to $S_i = \{\}$: no configuration can consume the current character, so the input cannot be matched. The final `select` returns `None` if no configuration in S_n is accepting, and otherwise the tree for the first one.

6.1.4 Worked Example: Standard Partial-Derivative Parser

Recall the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration:

$$S_0 = \Big[(r_0,\ id)\Big].$$

Transitioning with respect to `a`:

$$pderiv(r_0, \mathtt{a}) = pderiv\Big(Alt(a, ab), \mathtt{a}\Big) \cdot Alt(b, \varepsilon).$$

Inside the first factor:

$$\begin{aligned} \mathit{pderiv}(a, \mathtt{a}) &= \Big[(\varepsilon,\ \lambda_.\, a)\Big] \\ \mathit{pderiv}(ab, \mathtt{a}) &= \Big[(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, (a, v_2))\Big]. \end{aligned}$$

After the alternation rule wraps each state with *Left* or *Right*:

$$\begin{aligned} \mathit{pderiv}(a + ab, \mathtt{a}) = \Big[&(\varepsilon,\ \lambda v.\, \mathit{Left}\,(\lambda_.\, a)(v)), \\ &(\mathit{Seq}(\varepsilon, b),\ \lambda(v_1, v_2).\, \mathit{Right}\,(a, v_2))\Big]. \end{aligned}$$

The outer sequence rule combines this with the untouched second factor:

$$\begin{aligned} S_1 = \Big[&(\mathit{Seq}(\varepsilon, b + \varepsilon),\ \lambda(v_1, v_2).\, (\mathit{Left}\, a,\ v_2)), \\ &(\mathit{Seq}(\mathit{Seq}(\varepsilon, b), b + \varepsilon),\ \lambda((v_1, v_2), v_2').\, (\mathit{Right}(a, v_2),\ v_2'))\Big]. \end{aligned}$$

Transitioning with respect to `b` steps both configurations. For the first, the left factor ε is nullable, so the sequence rule produces the state that jumps directly into the second factor:

$$\mathit{pderiv}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ \lambda v.\, \mathit{Left}\, b)\Big].$$

The second factor's injection inserts the character at the leftmost position of the pair:

$$\mathit{inj}_1(\mathit{Left}\, b) = (\mathit{Left}\, a,\ \mathit{Left}\, b).$$

For the second configuration, the leading $\mathit{Seq}(\varepsilon, b)$ is not nullable (ε is, but b is not), so rule (P_{seq}) fires and the derivative continues inside b:

$$\mathit{pderiv}(\mathit{Seq}(\varepsilon, b), \mathtt{b}) = \Big[(\varepsilon,\ \lambda_.\, ((), b))\Big],$$

and the outer sequence rule rebuilds the pair *without* simplifying the leading ε away, giving the second configuration's residual $\varepsilon \cdot (b + \varepsilon)$, not $b + \varepsilon$. The two resulting state-set entries are:

$$\begin{aligned} S_2 = \Big[&(\varepsilon,\ \lambda_.\, (\mathit{Left}\, a,\ \mathit{Left}\, b)), \\ &(\varepsilon \cdot (b + \varepsilon),\ \lambda(_, v_2).\, (\mathit{Right}(a, b),\ v_2))\Big]. \end{aligned}$$

Selection. The first configuration's residual is ε, which is nullable, and selection returns immediately:

$$\mathit{select}(S_2) = (\lambda_.\, (\mathit{Left}\, a, \mathit{Left}\, b))(\mathit{mkEps}_{\varepsilon}) = (\mathit{Left}\, a, \mathit{Left}\, b).$$

Flattening confirms the result:

$$\mathit{flatten}\Big((\mathit{Left}\, a, \mathit{Left}\, b)\Big) = \mathtt{ab}.$$

The second configuration is never inspected: its residual $\varepsilon \cdot (b + \varepsilon)$ is also nullable, and would likewise decode to a valid tree, but generation order already placed the first configuration ahead of it, and *select* stops at the first match.

This is the non-POSIX parse tree. The derivative-based parser from Chapter 5 returns $(\mathit{Right}(a, b), \mathit{Right}())$ for the same input, the POSIX tree. The two trees agree on which word they spell out, but differ on how the input is decomposed: the non-POSIX tree commits to the shorter first alternative a in the first factor, while the POSIX tree commits to the longer alternative ab.

6.2 The Bit-Coded Partial-Derivative Parser

The standard partial-derivative parser carries a closure per configuration, which accumulates the reconstruction path during the forward pass. The bit-coded variant replaces closures with bit-sequences: each configuration's annotation records the choices made so far, and decoding the final bit-sequence against the original expression yields the parse tree.

Unlike the bit-coded *derivative* parser of Chapter 5, this construction has no `internalize` step and builds no annotated `ARegex` type. A partial derivative already returns one residual per surviving strand, so each strand's bits thread alongside its plain `Regex` residual as a separate value (`AppendBits`, §6.2.3) rather than being embedded in the expression tree itself. There is accordingly no *fuse* step either: *fuse* exists to carry bits from a node a shared annotated tree is about to lose onto the node that replaces it, and no such tree is ever built here.

Transitioning $w = \texttt{ab}$ from $r_0 = (a + ab)(b + \varepsilon)$ gives:

$$\begin{aligned} S_0 &= \big[(r_0,\ [\,])\big] \\ &\xrightarrow{\texttt{a}} S_1 = \big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\big] \\ &\xrightarrow{\texttt{b}} S_2 = \big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\big]. \end{aligned} \tag{6.6}$$

The first configuration of S_1 records that the character `a` was matched by the left branch of the first factor's alternation. The second configuration records that it was matched by the right branch, leaving the residual b to be consumed later. After the second character, only the first configuration is accepting, and its bits $[\mathit{false}, \mathit{false}]$ decode to $(\mathit{Left}\, a, \mathit{Left}\, b)$.

Figure 6.6 shows this transition together with the selection and decode steps.

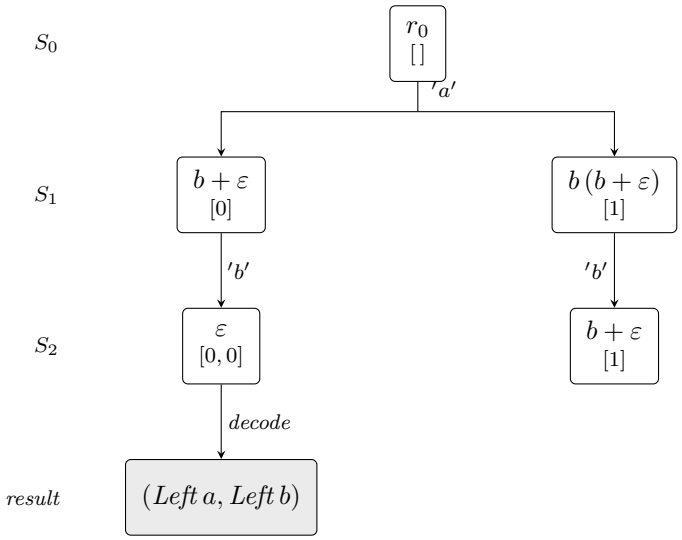


Figure 6.6: Overview of the bit-coded partial-derivative parser on the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ with input `ab`. Each box is a configuration: a residual expression and a bit-sequence, written 0 for *false* and 1 for *true*. Selection takes the first accepting configuration in S_2 and appends *mkEpsBC* of its residual, yielding the bit-string $[0, 0]$, which decode_{r_0} reads against the original expression.

The bit-coded parser keeps the same state-set structure as the standard one. Only the configuration's annotation changes, from a closure to a sequence of bits, and the final step changes from applying an injection to calling *decode*.

6.2.1 Bit-Coded Partial Derivatives

The bit-coded partial derivative $pderivBC(r, x)$ returns a list of (residual, bit-sequence) pairs. It follows the same case structure as (**??**), with one difference: where the standard version builds an injection closure, the bit-coded version prepends a fixed bit-sequence to the configuration's existing bits.

The bit conventions are inherited from Chapter 5.2. Each alternation contributes a leading *false* for the left branch or a leading *true* for the right branch. Each star iteration contributes a leading *false*, and each star termination contributes a *true*. Concatenation contributes no bits of its own.

$$
\begin{aligned}
pderivBC(\phi, x) &= \emptyset \\
pderivBC(\varepsilon, x) &= \emptyset \\
pderivBC(c, x) &= \begin{cases} \{(\varepsilon,\, [\,])\} & c = x \\ \emptyset & \text{otherwise} \end{cases} \\
pderivBC(r_1 + r_2, x) &= \{\, (r_1',\, [false] +\!\!+\, bs) \mid (r_1', bs) \in pderivBC(r_1, x) \,\} \\
&\quad \cup \{\, (r_2',\, [true] +\!\!+\, bs) \mid (r_2', bs) \in pderivBC(r_2, x) \,\} \\
pderivBC(r_1\, r_2, x) &= \{\, (seq(r_1', r_2),\, bs) \mid (r_1', bs) \in pderivBC(r_1, x) \,\} \\
&\quad \cup \begin{cases} \{\, (r_2',\, mkEpsBC_{r_1} +\!\!+\, bs) \mid (r_2', bs) \in pderivBC(r_2, x) \,\} & n(r_1) \\ \emptyset & \text{otherwise} \end{cases} \\
pderivBC(r^*, x) &= \{\, (seq(r', r^*),\, [false] +\!\!+\, bs) \mid (r', bs) \in pderivBC(r, x) \,\}.
\end{aligned}
\tag{6.7}
$$

As in the standard version, the result is deduplicated by residual, keeping the first occurrence.

The bit-coded sequence rule is the counterpart of the injection rule in the standard version. When the first factor is nullable, the bits recorded inside its empty match are extracted with $mkEpsBC_{r_1}$ and prepended to the second factor's bits, so the choice made inside the first factor survives in the surviving configuration.

The reference teaching material's own derivation keeps a trace of why this prefix is necessary: an earlier draft of this clause, kept only as a comment, used the jump strand's residual with no bits prepended, annotated *bug spotted by clause*; the live version was corrected to the $mkEpsBC_{r_1}$-prefixed form given here [Sulzmann, 2026], which is what (6.7) and Figure 6.7 below implement.

In Rust. Figure 6.7 implements (6.7) directly. The base cases and the alternation case mirror the standard version with bit-lists in place of closures. The sequence case inlines both the non-nullable and nullable paths. The star case prepends the leading *false* that marks one more iteration.

```
pub fn pderiv_bc(r: &Regex, x: char) -> Vec<(Regex, Vec<bool>)> {
    match r {
        Regex::Phi | Regex::Eps => vec![],

        Regex::Lit(y) => {
            if *y == x { vec![(Regex::Eps, vec![])] } else { vec![] }
        }

        Regex::Alt(r1, r2) => {
            let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (r1_, b)
                })
                .collect();
            out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, mut bs)| {
                let mut b = vec![true]; b.append(&mut bs);
                (r2_, b)
            }));
            nub2(out)
        }

        Regex::Seq(r1, r2) => {
            if nullable(r1) {
                let mut out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
                    .collect();

                let eps_bits = mk_eps_bits(r1);
                out.extend(pderiv_bc(r2, x).into_iter().map(|(r2_, bs)| {
                    let mut b = eps_bits.clone(); b.extend(bs);
                    (r2_, b)
                }));
                nub2(out)
            } else {
                pderiv_bc(r1, x)
                    .into_iter()
                    .map(|(r1_, bs)| (smart_seq(r1_, r2), bs))
                    .collect()
            }
        }

        Regex::Star(r1) => {
            let out: Vec<(Regex, Vec<bool>)> = pderiv_bc(r1, x)
                .into_iter()
                .map(|(r1_, mut bs)| {
                    let mut b = vec![false]; b.append(&mut bs);
                    (smart_seq(r1_, &Regex::star(*r1.clone())), b)
                })
                .collect();
            nub2(out)
        }
    }
}
```


Figure 6.7: `pderiv_bc`: one bit-coded partial-derivative transition.

The `Seq` arm uses `smart_seq` in both the nullable and the non-nullable case: r_1's own derivative can be ε regardless of whether r_1 itself is nullable – a literal's derivative is ε the moment it matches, even though the literal itself never is – so both branches need the same collapse. Unlike `pderiv_tree` (Section 6.1.1), no companion adjustment is needed here: bits ride alongside the residual as a separate value rather than being stored on it, so collapsing $\varepsilon \cdot s$ to s leaves `bs` untouched.

6.2.2 Extracting Bits

When a configuration's residual is nullable, its accumulated bits must be extended with the bits of the residual's empty match. This is the role of *mkEpsBC*, which mirrors *mkEps* from Chapter 5 but produces a bit-sequence instead of a parse tree.

$$\begin{aligned} mkEpsBC_{\varepsilon} &= [\,] \\ mkEpsBC_{r^*} &= [true] \\ mkEpsBC_{r_1 r_2} &= mkEpsBC_{r_1} +\!\!+ mkEpsBC_{r_2} \\ mkEpsBC_{r_1+r_2} &= \begin{cases} [false] +\!\!+ mkEpsBC_{r_1} & n(r_1) \\ [true] +\!\!+ mkEpsBC_{r_2} & \text{otherwise.} \end{cases} \end{aligned} \tag{6.8}$$

The alternation rule reproduces the left preference of *mkEps*, emitting *false* when the left branch is nullable and *true* otherwise. The star rule emits a single terminating *true*, since a star matching the empty word uses zero iterations.

```
pub fn mk_eps_bits(r: &Regex) -> Vec<bool> {
    match r {
        Regex::Phi => panic!("mk_eps_bits called on Phi"),
        Regex::Lit(c) => panic!("mk_eps_bits called on Lit('{}')", c),
        Regex::Eps => Vec::new(),
        Regex::Alt(r1, r2) => {
            if nullable(r1) {
                let mut bits = vec![false];
                bits.extend(mk_eps_bits(r1));
                bits
            } else {
                let mut bits = vec![true];
                bits.extend(mk_eps_bits(r2));
                bits
            }
        }
        Regex::Seq(r1, r2) => {
            let mut bits = mk_eps_bits(r1);
            bits.extend(mk_eps_bits(r2));
            bits
        }
        Regex::Star(_) => vec![true],
    }
}
```

Figure 6.8: `mk_eps_bits`: the bit-string analogue of `mk_eps` for plain `Regex`.

The `Phi` and `Lit` arms panic, matching the precondition that *mkEpsBC* is applied only to nullable expressions. Neither empty language nor a literal can match the empty word.

6.2.3 State Sets and Selection

State-set transition for the bit-coded parser mirrors the standard version, with bit-list concatenation replacing closure composition. Each configuration's bits accumulate across the whole input so far, so a plain `Vec<bool>` clone-then-extend on every transition is O(current length) per step, $O(n^2)$ over n characters – the same cost `Bits`/`Inj` avoid elsewhere in this and the previous chapter. `AppendBits` is the append-optimized mirror of `Bits` (Section 5.2.1): it stores the reverse of the real sequence internally, so appending a suffix becomes prepending its reverse, reusing `Bits::prepend`'s O(suffix length) guarantee, and materializes the sequence back into a plain `Vec<bool>` only once, when the bits are actually consumed:

```
pub(super) fn step_frontier(
    frontier: &[(Regex, AppendBits)], x: char
) -> Vec<(Regex, AppendBits)> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            out.push((r_next, bits.append(&extra)));
        }
    }
    nub2(out)
}
```

Figure 6.9: `step_frontier` (bit-coded): as with the standard parser's version, `nub2` deduplicates across the whole combined state set, not just within one configuration's own expansion (which `pderiv_bc` already does via its own `nub2` call).

Selection walks the state set in priority order and returns the bits of the first accepting configuration, extended with *mkEpsBC* of the residual:

```
pub(super) fn select_bits(
    frontier: &[(Regex, AppendBits)]
) -> Option<Vec<bool>> {
    for (r, bits) in frontier {
        if nullable(r) {
            return Some(bits.append(&mk_eps_bits(r)).to_vec());
        }
    }
    None
}
```

The resulting bit-sequence is interpreted by the shared `decode` function from Chapter 5.2, which walks the original expression and consumes one bit per alternation and per star iteration.

6.2.4 Parser

The parser composes `step_frontier` over the input, checks that the state set is non-empty, and calls `select_bits` followed by `decode`.

```
pub fn parse_pderiv_bc(input: &str, r: &Regex) -> Option<ParseTree> {
    let mut frontier = vec![(r.clone(), AppendBits::empty())];
    for c in input.chars() {
        frontier = step_frontier(&frontier, c);
        if frontier.is_empty() {
            return None;
        }
    }
    let bits = select_bits(&frontier)?;
    Some(decode(r, &bits))
}
```

Figure 6.10: The bit-coded partial-derivative parser.

The driver has the same structure as the standard one. Only the configuration type and the final step differ: bits and `decode` in place of closures and injection application.

6.2.5 Worked Example: Ambiguous Expression

Consider again the ambiguous example $r_0 = (a + ab)(b + \varepsilon)$ on input `ab`.

Forward pass. The initial state set holds a single configuration with empty bits:

$$S_0 = \Big[(r_0,\ [\,])\Big].$$

Transitioning by `a`, the first factor is not nullable, so *pderivBC* recurses into it:

$$\begin{aligned} \mathit{pderivBC}(a + ab, \mathtt{a}) &= \mathit{pderivBC}(a, \mathtt{a}).\mathit{map}(+[\mathit{false}]) \cup \mathit{pderivBC}(ab, \mathtt{a}).\mathit{map}(+[\mathit{true}]) \\ &= \Big[(\varepsilon,\ [\mathit{false}]),\ (b,\ [\mathit{true}])\Big]. \end{aligned}$$

The sequence rule then places the second factor after each derivative:

$$S_1 = \Big[(b + \varepsilon,\ [\mathit{false}]),\ (b\,(b + \varepsilon),\ [\mathit{true}])\Big].$$

Transitioning by `b`, both configurations are stepped. The first configuration reaches the second factor directly:

$$\mathit{pderivBC}(b + \varepsilon, \mathtt{b}) = \Big[(\varepsilon,\ [\mathit{false}])\Big],$$

giving bits $[\mathit{false}, \mathit{false}]$. The second configuration's derivative comes from b inside the leading sequence:

$$\mathit{pderivBC}(b, \mathtt{b}) = \Big[(\varepsilon,\ [\,])\Big],$$

and the outer sequence rule reconstructs the residual $b + \varepsilon$ with the bits unchanged:

$$S_2 = \Big[(\varepsilon,\ [\mathit{false}, \mathit{false}]),\ (b + \varepsilon,\ [\mathit{true}])\Big].$$

Selection and decoding. The first configuration is accepting, so *selectBits* returns its bits extended with $mkEpsBC_{\varepsilon} = [\,]$:

$$selectBits(S_2) = [false, false].$$

Decoding these bits against r_0:

$$decode_{r_0}\Big([false, false]\Big) = \Big(Left(a), Left(b)\Big).$$

The first *false* selects the left branch of the first factor's alternation, committing to the shorter alternative a. The second *false* selects the left branch of the second factor's alternation, committing to the shorter alternative b. The resulting tree is the non-POSIX parse tree, identical to the one produced by the standard partial-derivative parser.

Flattening validates the result:

$$flatten\Big((Left(a), Left(b))\Big) = \texttt{ab}.$$

Both partial-derivative parsers therefore produce the same tree on this example, and both differ from the derivative-based parsers of Chapter 5, which commit to the longer first alternative and return $(Right(a, b), Right())$.

7 Translation

The five parsers of Chapters 5 and 6 operate on the six-constructor `Regex` of Chapter 2. Real patterns may be more complex and contain shorthand character classes, bounded repetition, alternation over ranges, and a PCRE flag suffix, none of which the `Regex` can represent. This chapter introduces `ExtPat`, an AST that represents a surface syntax that allows for such richer patterns, and shows how pattern strings are first parsed into this `ExtPat` and subsequently translated into the internal `Regex`.

7.1 Parsing: String to ExtPat

Take the pattern string `[a-c]+\d{2}`. A recursive-descent parser turns it into the intermediate tree `ExtPat`, as shown in Figure 7.1. Precedence is fixed by the call chain, from loosest to tightest, with alternation and concatenation being left-associated and postfix quantifiers binding most tightly. Concatenation splits the pattern into `([a-c]+)(\d{2})`, and the postfix turns them into `Plus` over a class and a bounded repetition of a shorthand:

```
Concat [
    Plus(Any(['a', 'b', 'c'])),
    Bound(Escape('d'), 2, Some(2))
]
```

Figure 7.1: The `ExtPat` produced from `[a-c]+\d{2}`.

7.2 Translate: ExtPat to Regex

`translate` then turns the `ExtPat` into the `Regex` value, that can be consumed by the parsing algorithms of the previous chapters:

- `Any(['a','b','c'])` expands over the working alphabet to a 3-leaf `Alt` chain.
- `Plus` is rewritten structurally as `Seq(r, Star(r))`.
- `Escape('d')` expands to a 10-leaf `Alt` chain, with one leaf for each digit.
- The two `Concat` layers are translated into nested `Seq` nodes, with the resulting structure folded into a single `Seq` chain.

The resulting `Regex` has 54 nodes (Figure 7.2). The two digit expansions account for 38 of them, with 20 digit leaves and 18 `Alt` nodes, here abbreviated as `D`:

```
D = Alt(Alt(Alt(Alt(Alt(Alt(Alt(Alt(Alt(0,1),2),3),4),5),6),7),8),9)
```

The remaining 16 come from the two `[a-c]` expansions, the `Plus` rewrite, the unrolling, and the surrounding `Seq` nodes.

```
Seq(
   Seq(
      Alt(Alt(a, b), c),
      Star(Alt(Alt(a, b), c))
   ),
   Seq(
      Seq(Eps, D),
      D
   )
)
```

Figure 7.2: The full `Regex` produced by translating the `ExtPat`. `D` abbreviates the nine-deep, left-nested `Alt` chain over the digits `0`–`9`.

7.3 The Surface Syntax

Rejected constructs: The example pattern is accepted, but not all are. A construct is rejected whenever it cannot be translated without changing what the pattern denotes.

Full-string matching: A translated pattern matches the whole input or nothing. There is no search padding, no substring search, and no anchor handling.

A 98-character working alphabet: Shorthand and negated classes enumerate a fixed set of 98 characters, the 95 printable ASCII characters `0x20`–`0x7E` together with tab, newline, and carriage return. The shorthands expand to `\d` (digit) `0`–`9`, `\w` (word) `a`–`z`, `A`–`Z`, `0`–`9`, and underscore, and `\s` (whitespace) space, tab, newline, and carriage return.

PCRE flags: Rule sets commonly encode matching options in a `/PATTERN/FLAGS` wrapper such as `/abc/i`, so the suffix must be interpreted alongside the pattern. Each flag falls into one of three groups (Table 7.1).

Flag	Group	Effect
`i`	honoured	case-fold literal letters
`s`	honoured	dot matches newline
`m`	rejected	anchors match at newlines
`x`	rejected	whitespace insignificant
`R`	ignored	search positioning
`A`	ignored	anchor selection
`E`	ignored	dollar tuning
`U`, `H`, `P`, `C`, ...	ignored	buffer or normalisation selection

Table 7.1: PCRE flags by group. Honoured flags are part of the pattern's meaning and are reproduced by a structural rewrite on `ExtPat`. Some flags are rejected outright. Ignored flags choose a buffer region or search position and do not affect the denoted language.

7.3.1 Accepted Constructs

Table 7.2 summarizes the surface syntax handled by the frontend, its representation in `ExtPat`, and the corresponding `Regex` translation for supported constructs.

Surface syntax	`ExtPat`	Translates to	Note
Structural			
ε, empty branch of `a|/(|b)`	`Eps`, `Empty`	`Eps`	
$\emptyset$	`Never`	`Phi`	
literal `a`	`Char`	`Lit('a')`	full Unicode `char`
`ab`	`Concat`	`Seq(a, b)`	left-folded
`a|b|c`	`Or`	`Alt(Alt(a, b), c)`	left-folded
`(...)`, `(?:...)`, `(?<n>...)`, `(?P<n>...)`	`Group`, `GroupNonMarking`	body	wrapper name dropped
`\n`, `\t`, `\r`	`Char`	`Lit(ch)`	
`\xHH`	`Char`	`Lit(ch)`	one byte
`\0` outside a class	`Char`	`Lit(ch)`	octal, clamped to 255
`\0`–`\7` inside `[...]`	class member	`Any`/`NoneOf` member	octal, clamped to 255
Quantifier			
`r?`	`Opt`	`Alt(r, Eps)`	
`r+`	`Plus`	`Seq(r, Star(r))`	
`r*`	`Star`	`Star(r)`	
`r{m,n}`	`Bound`	`m` copies and `n-m` optional	$m,n \leq 1000$
`r{m,}`	`Bound`	`m` copies and a `Star`	$m \leq 1000$
`r*?`, `r+?`, `r??`, `r{m,n}?`	as above	as above	lazy marker dropped
Class (alphabet expansion)			
`[abc]`, `[a-z]`	`Any`	`Alt` chain, left-folded	
`[^abc]`	`NoneOf`	`Alt` chain over complement	alphabet minus listed
`.`	`Dot`	`Alt` chain, 98 leaves	97 under `/.../` without `s`
`\d`, `\w`, `\s`	`Escape`	`Alt` chain, 10, 63, 4 leaves	
`\D`, `\W`, `\S`	`Escape`	`Alt` chain, 88, 35, 94 leaves	

Table 7.2: Surface syntax, grouped by translation mechanism, together with the `ExtPat` each produces and the `Regex` it translates into. Leaf counts refer to the 98-character alphabet.

`ExtPat::Escape` covers `\d`, `\w`, `\s`, their capitalised forms, and escaped punctuation such as `\.`. Control, hexadecimal, and octal escapes are represented as `ExtPat::Char`. Capitalised shorthands and negated classes denote the working alphabet minus the positive set. Literals can be any Unicode `char` and become `Regex::Lit` values. Thus, `é` matches itself, but `\W` cannot match it. Shorthands expand over the working alphabet; escaped punctuation becomes one literal, so `\.` matches only a period. Within character classes, shorthands expand inline and cannot serve as range endpoints.

7.3.2 Rejected Constructs

Backreferences and lookarounds are rejected as non-regular constructs, while other regular forms, such as anchors and POSIX classes, are excluded from this work's scope (Table 7.3).

Construct	Example	Reason
Rejected at parse		
backreference	`(a)\1`, `(?P=n)`, `\1`–`\9`	not regular, no `ExtPat`
lookaround	`(?=`, `(?!`, `(?<=`, `(?<!`	not regular, no `ExtPat`
POSIX class	`[[:alpha:]]`	would be read as literal characters
Rejected at translation		
anchor	`^`, `$`	position-dependent
word boundary	`\b`, `\B`	position-dependent
bound above 1000	`a{1001}`	unrolling prohibitive
Rejected at flag check		
flag `m`	`/abc/m`	would make anchors newline-aware
flag `x`	`/a b/x`	would make whitespace insignificant

Table 7.3: Surface forms the parser refuses, grouped by the stage at which refusal happens.

7.4 Translation Sizes

The example in Figure 7.2 translates to 54 nodes. Table 7.4 shows the contribution of individual character classes. A class with n members produces n literals and $n - 1$ `Alt` nodes, for $2n - 1$ nodes in total. Negated classes are complemented against the 98-character working alphabet before expansion. The dot construct expands to the full working alphabet when `s` is set, or to the alphabet excluding newline otherwise.

Pattern	Members	`Alt` nodes	Total nodes
`\d`	10	9	19
`\D`	88	87	175
`\w`	63	62	125
`\W`	35	34	69
`\s`	4	3	7
`\S`	94	93	187
`[^a]`	97	96	193
`.`	98	97	195

Table 7.4: Node counts for alphabet expansion over the 98-character working alphabet.

Translated pattern size depends on both structural and alphabet expansion, as shown by the effects of repetition and case folding in Table 7.5. Repetition increases the size by cloning the repeated expression, while case folding replaces each letter with an `Alt` between its uppercase and lowercase forms, adding two nodes per letter.

Structural expansion only		*Including alphabet expansion*	
Pattern	Nodes	Pattern	Nodes
`a*`	2	`\d+`	40
`a?`	3	`\d{2}`	41
`/a/i`	3	`[a-z]`	51
`a+`	4	`[a-c]+\d{2}`	54
`abc`	5	`\d{2,}`	62
`a{2}`	5	`\d{2,4}`	87
`ab{2}`	7	`[a-z]+`	104
`a{2,}`	8	`\w+`	252
`a{2,4}`	15	`\S+`	376

Table 7.5: Translated size in `Regex` nodes for representative patterns.

Bounds are also a source of growth. `Regex` has no representation for `ExtPat::Bound`, so `r{m,n}` becomes m exact copies plus $n - m$ optional ones, and `r{m,}` becomes m copies plus a `Star`. Both clone the body, so size grows with the bound.

However, the implementation imposes limits to keep translated patterns manageable. Unrolling clones the body once per repetition, and `translate` recurses once per unrolled copy, so an unbounded repeat count risks a stack overflow during translation alone. Bounded repetitions whose `m` or `n` exceeds 1,000 are therefore rejected. This limits individual repetitions but not total size, since nested bounds and multiple classes compound. Dataset preparation therefore caps the translated `Regex` at 20,000 nodes and its estimated nesting depth at 2,000. The measurements taken to justify these limits can be seen in the following evaluation chapter.

8 Evaluation

This chapter evaluates the derivative and partial-derivative parsers developed in Chapters 5 and 6. Using two datasets and a common set of criteria, it compares the candidates in terms of execution time, memory consumption, and translation cost. It also assesses their agreement on membership decisions and parse-tree shape.

All measurements were taken on an AMD Ryzen 5 5600 six-core processor with 16 GB of RAM, running Pop!_OS 24.04 LTS on an AMD B550 chipset.

8.1 Candidates

A candidate is a parser p that takes a translated `Regex` r and an input `String` w, and returns either a `ParseTree` pt or no match. Such an input pair (r, w) is called an *entry*. All candidates share the same interface:

$$p :: (Regex, String) \to Option\langle ParseTree\rangle.$$

Candidates are classified into two families by their disambiguation strategy.

POSIX family.

- `deriv_std_rec`, the recursive standard derivative parser (Section 5.1.3).
- `deriv_std_loop`, the iterative standard derivative parser
- `deriv_bc`, the bit-coded derivative parser (Section 5.2.8).

Non-POSIX family.

- `pderiv_std`, the standard partial-derivative parser (Section 6.1.3).
- `pderiv_bc`, the bit-coded partial-derivative parser (Section 6.2.4).

All five parsers are expected to agree on membership, so given an entry, they must either all accept it or all reject it. They are also expected to agree on parse-tree shape *within* each family, because the parsers of one family share the same disambiguation policy and differ only in how they compute and reconstruct the tree.

For the POSIX family this means the parsers must return the same POSIX parse tree and the non-POSIX family parsers must return the same leftmost parse tree.

Across families, agreement is not required. Since the two families implement different disambiguation policies, they may select different parse trees for the same entry even when they agree on membership, as Chapter 4 shows.

8.2 Evaluation Criteria

The evaluation considers two aspects of the candidates. The first is *performance*, measured in terms of execution time, memory consumption, and translation cost. The second is *correctness*, assessed by comparing membership and parse-tree selection.

8.2.1 Measurement Setup

Each entry runs in its own worker subprocess for timing, memory, stack-depth, and parse-agreement measurements. Stable Rust has no safe way to forcibly terminate a thread [The Rust Project Developers, 2026c], so one that has not returned, e.g. from exponential derivation or from allocating towards the memory cap, cannot be stopped. A subprocess, by contrast, can be killed unconditionally, and the operating system reclaims its memory.

- *Measured*: the worker exited zero and reported a result line.
- *Timed out*: the worker exceeded the default per-entry timeout (2 s) and was killed.
- *Crashed*: the worker exited non-zero or was killed by a signal, including stack overflows and out-of-memory kills when the memory ceiling is enabled.

For timing, memory, and parse-agreement measurements, each worker runs on a custom 256 MiB stack: all candidates recurse over the pattern structure, and on deeply nested patterns the default stack would overflow before the measured input sizes are reached. This stack is a fixed allowance and not part of the measured footprint.

A timing and memory run produces two outputs per candidate and dataset: (1) a summary table of aggregate measurements; (2) a per-entry file to compare individual entries.

8.2.2 Timing

Each entry (r, w) is timed once, without warm-up run or repeated measurements. Translation is measured separately, so the wall-clock time covers only the parser call $p(r, w)$. For `deriv_std_rec`, `deriv_std_loop`, and `pderiv_std`, the entire parser call is reported as *mean*. For `deriv_bc` and `pderiv_bc`, the call is split into three: *Encode* constructs the initial annotated state before any character of w is consumed, *Derive* processes the input one character at a time and *Decode* converts the final bit-string into a parse tree.

Timing results show four statistics, aggregated differently for the two datasets:

- *Mean* is a macro-average over patterns of the real corpus, so a pattern with 10 entries and one with 2 contribute equally, a flat mean is used for the synthetic set.
- *Median*, *99th percentile*, and *maximum* are pooled over individual entries rather than averaged per pattern. The median describes a typical entry, the 99th percentile the tail of the entry distribution, and the maximum the single worst entry.

All four statistics represent lower bounds on the corresponding execution times, since timed-out and crashed entries are excluded rather than counted at their timeout values. A separate *wall* column measures the benchmark's entire loop over one source pattern. It therefore exceeds *entries* $\times$ *mean* by the per-entry process-spawn and pipe-round-trip overhead excluded from the individual timings. It captures the total cost of reproducing a pattern's results, harness included, but is not suitable for comparisons between candidates.

8.2.3 Memory

Memory is measured with a counting allocator in each worker, baselined immediately before the parser call so that only allocations made during $p(r, w)$ are counted. The allocator tracks a running sum of requested bytes and a running count of live bytes:

- *Max live peak*: the highest value of the live-byte counter within a call. It bounds the heap a single call requires, and is the measure that determines whether a parser can run at a given input length within a fixed memory budget.
- *Total allocated*: the sum of requested bytes over the call, including memory later freed. It is not a footprint, but a proxy for allocator pressure and a secondary explanation of timing, since a parser that allocates and frees frequently pays a cost even when its peak stays low.

Total allocation counts memory freed during the call, so it is always at least as large as peak. Because peak grows with the input a run processes and total with the work it performs, each is compared only where those are comparable: peak across rows that reached comparable input lengths, total across rows that completed comparable work.

The allocator sees heap allocations only, not the worker stack (Section 8.2.1), so a parser that uses substantial stack but few heap allocations may appear inexpensive in these measurements. Stack depth is evaluated separately for the two standard derivative parsers.

8.2.4 Translation

Translation depends only on the source pattern and is independent of the input, so each distinct pattern is translated once rather than once per entry. Translation runs in the main process because the frontend enforces the three following resource limits:

- *Bound cap*: bounds the numeric value of any repetition bound.
- *Depth cap*: bounds the estimated depth of the translated pattern.
- *Node cap*: bounds the node count of the result.

The caps reject a pattern before translation would recurse too deeply or unroll too large a result, so translation itself is always bounded in time and memory. A parser call has no such protection. It can run until the timeout or the memory cap kills it, which is why the caps are checked at translation and the timeout is enforced per entry.

Translation cost is measured using four quantities:

- *Time*: the wall-clock duration of the translation call.
- *Total bytes*: the total amount of memory allocated during translation.
- *Peak bytes*: the highest number of live allocated bytes during the call.
- *Nodes*: the number of nodes in the resulting `Regex`.

The counting allocator is reset before each translation and read immediately after the call returns. This excludes subsequent node counting and debug rendering from the measurement. Mean, median, 99th percentile, and maximum are calculated directly over patterns. Since each pattern contributes one measurement, no macro-averaging is needed. Rejected patterns produce no measurement, so the reported statistics are lower bounds.

8.2.5 Stack Depth

`deriv_std_rec` and `deriv_std_loop` compute identical results but store the residual sequence differently. The recursive implementation uses one call-stack frame per input character, the iterative one a `Vec`. Stack depth is measured to establish which of the two exhausts the stack first, and at what input length. Both use the operating system's default thread stack, which is 8 MiB on Linux.

The maximum surviving input length is estimated using a doubling search. Starting from a small input, the length is doubled after each successful probe until the worker fails to complete. The timeout for each probe is increased from the default 2 s to 15 s, to give slow derivatives time to exhaust the stack and cause a stack overflow. A worker can also be killed by an out-of-memory condition when a heap cap is set, as described in Section 8.2.1, but for this measurement no heap cap is applied, so a signal kill is more likely to be a stack overflow than an out-of-memory kill.

The search brackets the last length that completed against the first that did not. A crash means stack exhaustion limited the input, a timeout means derivation became too expensive first. The timeouts are kept as negative controls for patterns where derivation cost, not stack depth, is limiting.

The same doubling search is used to check the two translation caps, the *bound cap* and the *depth cap*. For the bound cap, the bound value is doubled until translation of the bound exhausts the stack. For the depth cap, the nesting depth of `(?:...)` groups is doubled until translation exhausts the stack. Bound unrolling recurses once for each unit in a bound's open part, while syntactic nesting recurses once for each level of grouping.

8.2.6 Parse Agreement

Parse agreement is evaluated on entries with a known expected membership result. Each entry is written as (r, w, e), where r is the translated `Regex`, w is the input string, and $e \in \{true, false\}$ is the expected match result. No expected parse tree is provided.

Each candidate processes the same entry, and records:

- *Timeout or crash*: if the candidate exceeded the timeout or was killed by a signal.
- *Membership*: a candidate's decision on ($e = true$) or no-match ($e = false$)
- *Parse tree*: the parse tree a candidate returned, when $e = true$.

A timeout or crash removes the candidate from the comparison for that entry. Membership is compared across all candidates. Any disagreement with e is a bug. Parse trees are compared only when $e = true$ and all candidates returned one. Agreement is required within each family, whose parsers share a disambiguation policy. Across families it is reported but not required, since the two policies may differ.

Thus the summary reports how many entries were evaluated and, for each dataset, how many timed out or crashed, how many every candidate classified correctly (*match agreement*), and of those, how many the two families parsed to the same tree (*shape agree*) or to different trees (*shape disagree*).

8.3 Datasets

The evaluation uses two datasets (Table 8.1): S2ELab, a corpus of human-authored patterns, and a synthetic set of manually authored patterns designed to stress the parsers.

Dataset	Authored	Duplicate	Rejected	Accepted	Unanchored	Fully-Anchored	Inputs
S2ELab	762	7	177	578	115	463	11 395
Synthetic	112	0	0	112	0	112	891

Table 8.1: Benchmark scope. *Authored* = *Duplicate* + *Rejected* + *Accepted*, and *Accepted* = *Full-Anchored* + *Unanchored*. Only the *Full-Anchored* patterns are benchmarked after stripping, and the *Inputs* column counts entries for those patterns only.

8.3.1 S2ELab

The dataset by Siddiq et al. [2024] contains 762 human-authored patterns. Each pattern has a natural-language description and lists of labeled matching and non-matching inputs. The number of each varies per pattern, so the corpus is not balanced between the two. The source patterns conventionally use `^` and `$` as beginning- and end-of-input anchors. A pattern surrounded by these anchors requires a full-string match, whereas a pattern without them can match a substring. As described in Section 7.3.2, the frontend does not support anchors. However, fully anchored patterns can still be used by stripping them. Since the parsers perform full-string matching, the stripped pattern accepts exactly the inputs intended by the original anchored pattern. Figure 8.1 shows this in the translation stage, as part of the data preparation that turns one example pattern into an *entry*.

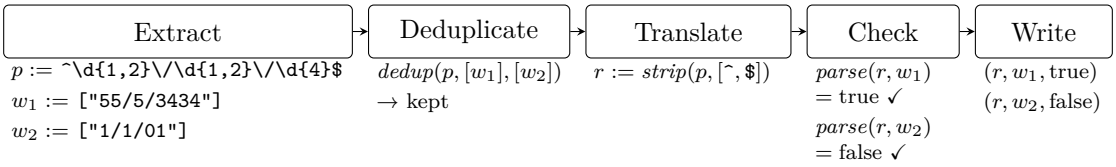


Figure 8.1: The S2ELab preparation pipeline. Extraction keeps the pattern p with its match list $[w_1]$ and non-match list $[w_2]$; *dedup* drops byte-identical patterns and inputs; $strip(p, [\hat{}, \$])$ yields the translated `Regex` r. The check calls $parse(r, w)$ on each input and keeps the pair only when the verdict agrees with its label (✓), validating the pattern's own semantics rather than real-world correctness: `"55/5/3434"` is accepted because the pattern checks digit counts, not value ranges. Surviving triples are exactly the defined entries (r, w, e) with $e \in \{true, false\}$.

The benchmarks only use the 463 fully anchored patterns after stripping, excluding the 115 unanchored ones, while 177 are rejected, with the causes summarised in Table 8.2.

Cause	S2ELab patterns
Anchor (non-strippable)	106
Lookaround	43
Word boundary	23
Backreference	3
Other (node cap, syntax errors)	2
Total	177

Table 8.2: S2ELab patterns rejected by the frontend, broken down by cause.

8.3.2 Synthetic Stress Set

The synthetic set contains 112 manually authored patterns designed to investigate how parser costs grow with input length and pattern size. It complements the real corpus, which does not exercise these cost mechanisms at comparable scales. All patterns are intended for full-string matching. Each row defines a base pattern and a base input in terms of the input length n, the pattern size p, or both, and a *sweep* varies one of these parameters over a range of values, recording a measurement at each.

A `P` row fixes the pattern and grows n, measuring cost against input size, while an `E` row fixes the input and grows p, measuring cost against pattern size. Each sweep value yields an *entry* (r, w, e). Table 8.3 shows one row of each type at two sweep values.

Row	Swept	Fixed	Description	Pattern	Input
P01	$n = 10$	—	`a` repeated 10x	`(a+)+`	[`a`^{10}]
P01	$n = 1000$	—	`a` repeated 1000x	`(a+)+`	[`a`^{1000}]
E36	$p = 100$	$n = 20$	100-way word Alt	`(?:w000001|...|w000100)+`	[`w000001`^{20}]
E36	$p = 1000$	$n = 20$	1000-way word Alt	`(?:w000001|...|w001000)+`	[`w000001`^{20}]

Table 8.3: Two rows at two sweep values each. P01 fixes the pattern and sweeps n, so the input grows. E36 sweeps p, so the alternation width grows while the input is fixed $n = 20$.

The 112 patterns are also grouped into categories according to the complexity mechanism they are designed to test (Table 8.4). Each category uses its own sweep range, chosen so that the measured cost typically increases from microseconds to a timeout or crash.

Cat.	Name	Mechanism	Example	Patterns	Entries
A	Nested quantifiers	Quantifier inside a quantifier	`(a+)+`	16	160
B	Overlapping alternation	Shared-prefix alternatives under a quantifier	`(a|aa)+`	15	150
C	Bounded rep over star	Bounded repetition of a star, unrolled	`(.*a){1,10}`	7	56
D	Anchored-failure style	Must fully consume input; near-miss tail forces full enumeration	`(a?){3}a{3}`	4	40
E	Prefix-overlap alternation	One alternative is a strict prefix of another	`(a|b|ab)*`	8	80
F	Star then failing literal	Trailing literal fails only at the last character	`(a|b)*c`	7	70
H	Adjacent unbounded groups	Consecutive unbounded groups multiply split points	`(.+)(.+)...`	4	28
I	Long input, simple pattern	Per-character cost on a very long input	`a+`	10	60
J	Large pattern, short input	Per-node cost of a very large pattern	100-word Alt.	5	22
K	Large pattern, large input	Per-node and per-character cost together	1000-word Alt.	3	30
L	Trivial pattern, trivial input	Baseline	`x`	33	195
Total				112	891

Table 8.4: The synthetic-set categories. Categories I, J, K, and L form a 2×2 matrix of pattern size against input size; the remaining seven each stress one cost mechanism. *Patterns* counts distinct hand-curated or generated pattern definitions; *Entries* counts the (pattern, input) pairs.

8.4 Performance Analysis

This section compares the candidates on both datasets in terms of execution time (8.4.2), memory use (8.4.3), and translation cost (8.4.4). Each subsection presents the real-corpus results first, followed by the synthetic results and some representative patterns with their individual measurements. The test on stack depth (8.4.1) precedes these measurements.

8.4.1 Stack Depth

The recursive and iterative forms run on the operating system's default stack of 8 MB, without a memory cap, using the doubling search described in Section 8.2.5. Both rows use `a` repeated n times as the input, with n increasing across the sweep.

Row	Pattern	Derivative step	Parser	Last n	Outcome	Elapsed
P01	`(a+)+`	expensive (nested quantifier)	`rec`	16	timeout	15.818 s
			`loop`	16	timeout	15.845 s
P30	`(\w+)(\w+)...`	moderate (product of stars)	`rec`	64	timeout	21.397 s
			`loop`	64	timeout	20.579 s
E51	`x`	cheap (fixed residual)	`rec`	32 768	crash	1.291 s
			`loop`	268 435 456	timeout	34.180 s

Table 8.5: Doubling-search stack-depth results, one group per pattern with the recursive (`rec`) and iterative (`loop`) parsers. *Derivative step* categorizes the pattern's derivative cost. P30 is ten adjacent `(\w+)` groups. *Outcome* reports what stopped the sweep: *timeout* at the 15 s per-probe cap, or *crash* on a stack overflow. *Elapsed* is the cumulative time taken for the whole sweep.

P01 and P30 are negative controls. On both, the derivative step is expensive enough that the 15 s timeout fires before either form reaches its limit, at $n = 16$ and $n = 64$ respectively. P01 fails first because its nested quantifier makes each derivative step near-exponential. P30 fails later because ten adjacent `(\w+)` groups produce a product of stars whose cost grows polynomially. In both cases the timeout, not the stack, ends the sweep, so the input never grows far enough to expose a stack ceiling. Their role is to rule out the reading that a stack limit would have been reached at small n on any pattern.

E51 isolates the stack. The pattern is the single literal `x`, which cannot match an input of `a`s, so every entry mismatches on its first character and the residual stays tiny. The derivative step therefore does not grow with the input. Both forms must keep all n intermediate derivatives alive between the forward and backward passes and differ only in where that sequence lives. The recursive parser keeps it implicitly, one derivative per stack frame, so the call stack grows with the input until it crashes between 32 768 and 65 536 characters. The iterative form keeps the same sequence explicitly, in a heap-allocated `Vec`. It reaches 268 435 456 characters and stops only by the 15 s timeout ending the sweep.

The comparison is therefore not a contest the iterative form wins on stack usage, since it has no stack growth to measure, by construction. This measurement establishes the input length at which the stack becomes the binding constraint for the recursive form. On crash resistance the iterative form is therefore preferable even where the derivative step is cheap, since it has no stack-depth limit within the sweep range. Whether it is preferable overall depends on which cost matters more for a given input, which will be examined further through the memory measurements presented later in Section 8.4.3

8.4.2 Timing

This section reports the per-candidate timing on the real corpus and the synthetic set, under the isolation, timing phase split, and averaging described in Section 8.2.2.

S2ELab

Parser	Timed out	Mean (ms)	Median (ms)	Max (ms)
`deriv_std_rec`	74	7.751	0.217	1 866.221
`deriv_std_loop`	73	7.454	0.232	1 802.287
`deriv_bc`	0	1.695	0.320	310.481
`pderiv_std`	0	0.722	0.109	124.869
`pderiv_bc`	0	0.634	0.100	108.532

Table 8.6: Corpus timing over the default scope of 463 patterns and 11 395 entries per parser.

The standard variants are substantially slower, with mean times roughly 4–4.5× that of `deriv_bc` and more than ten times those of the partial-derivative variants. The gap is wider than the means suggest, because the 73 and 74 entries time out on the standard variants and are excluded from their means. This is why the standard pair's own median looks lower but cannot be used for a fair comparison. On the median, `deriv_bc` is slower than both partial-derivative variants, a fair comparison since neither times out either. Within the partial-derivative pair, `pderiv_bc` is marginally faster, at 0.634 against `pderiv_std`'s 0.722 ms.

	Matches			Non-matches		
Parser	Total	Timeout	Time (s)	Total	Timeout	Time (s)
`deriv_std_rec`	5 797	72	74.928	5 524	2	9.812
`deriv_std_loop`	5 798	71	70.732	5 524	2	10.079
`deriv_bc`	5 869	0	14.595	5 526	0	5.067
`pderiv_std`	5 869	0	6.664	5 526	0	1.797
`pderiv_bc`	5 869	0	5.850	5 526	0	1.573

Table 8.7: Total wall time by verified outcome on the same run as Table 8.6.

For every parser, matching entries take roughly 3–8× as much total time as non-matches. The asymmetry is expected. A non-match can be rejected as soon as any character fails to match, whereas a match requires the parser to consume the whole input and construct a tree. The gap is largest for the two standard variants and smallest for the bit-coded pair, which is consistent with the standard construction spending most of its time on the matching path, where it retains the derivation sequence for the backward pass.

On matches, `deriv_std_loop` takes 70.732 s against the recursive variant's 74.928 s; their non-match times are almost identical (10.079 against 9.812 s), so the difference between the two forms is confined to the matching path, the only place the backward pass runs. The totals cover slightly different entry sets, and almost all timeouts fall on matches: 72 of `deriv_std_rec`'s 74 and 71 of `deriv_std_loop`'s 73, against none for the other three.

Synthetic Stress Set

The eleven categories (Table 8.4) are each swept once at their own default parameter values, so every entry is a single run at one point of a sweep rather than an average over repetitions. Table 8.8 reports the seven mechanism categories and Table 8.9 the 2×2 pattern-size/input-size matrix. Both group the five parsers by category, so parsers compare directly within a fixed category. Since timed-out entries are excluded from the measurements those columns understate a slower parser's cost in proportion to how many entries it loses, meaning the timeout count is the primary indicator.

Cat.	Parser	Timed out	Measured	Mean (ms)	Median (ms)	Max (ms)
A	`deriv_std_rec`	112	48	191.15	3.90	1 736.50
	`deriv_std_loop`	111	49	225.58	3.79	1 894.74
	`deriv_bc`	2	158	165.05	0.32	1 814.38
	`pderiv_std`	0	160	19.74	0.24	1 312.70
	`pderiv_bc`	0	160	15.87	0.26	825.96
B	`deriv_std_rec`	95	55	168.75	28.15	1 516.52
	`deriv_std_loop`	95	55	169.82	29.18	1 567.05
	`deriv_bc`	6	144	99.64	2.42	1 482.84
	`pderiv_std`	2	148	83.14	2.26	1 986.13
	`pderiv_bc`	0	150	98.56	1.82	1 713.12
C	`deriv_std_rec`	32	24	88.98	3.79	866.81
	`deriv_std_loop`	32	24	90.65	3.81	869.75
	`deriv_bc`	14	42	238.48	50.41	1 201.74
	`pderiv_std`	10	46	270.77	35.42	1 035.94
	`pderiv_bc`	10	46	245.32	31.47	957.84
D	`deriv_std_rec`	8	32	260.84	46.16	1 895.22
	`deriv_std_loop`	8	32	261.62	46.04	1 915.13
	`deriv_bc`	0	40	141.61	16.07	1 513.66
	`pderiv_std`	6	34	241.30	52.38	1 026.07
	`pderiv_bc`	6	34	204.20	47.87	925.94
E	`deriv_std_rec`	46	34	253.05	12.35	1 883.29
	`deriv_std_loop`	43	37	381.76	12.43	1 915.26
	`deriv_bc`	0	80	80.61	4.74	1 285.09
	`pderiv_std`	0	80	65.76	3.62	1 202.81
	`pderiv_bc`	0	80	60.00	3.69	1 023.59
F	`deriv_std_rec`	46	24	408.17	16.79	1 801.95
	`deriv_std_loop`	46	24	394.89	15.28	1 759.03
	`deriv_bc`	28	42	42.91	2.59	413.57
	`pderiv_std`	2	68	106.55	7.29	1 744.62
	`pderiv_bc`	2	68	109.19	7.68	1 790.56
H	`deriv_std_rec`	18	10	236.09	15.55	1 240.12
	`deriv_std_loop`	18	10	235.24	13.86	1 258.37
	`deriv_bc`	2	26	261.45	80.84	1 710.24
	`pderiv_std`	5	23	166.72	44.75	879.46
	`pderiv_bc`	5	23	141.85	39.31	759.49

Table 8.8: Per-parser timing over the synthetic set's default sweep for the mechanism categories A–H. *Timed out* counts entries exceeding the budget, with any crash separately shown in parentheses; *Measured* counts entries with a completed timing. *Mean*, *Median*, and *Max* are over measured entries only.

Categories A, B, E show the expected behavior. The partial-derivative variants are faster than the derivative variants, though within their own family they are nearly tied on median (0.24–0.26, 1.82–2.26, and 3.62–3.69 ms in A, B, and E). `deriv_bc` follows behind (0.32, 2.42, 4.74 ms). All three categories let the same input be consumed by many decompositions, so their residual expressions grow with each step and their size drives the cost. While `deriv_bc` simplifies after each step and keeps a single expression, the partial-derivative parsers keep a deduplicated set instead. The standard derivative parsers are much slower throughout and timeout against 111–112 of A's 160, 95 of B's 150, and 43–46 of E's 80 entries.

Category C has high timeout counts across all five parsers, from 10 to 32 of its 56 entries. Its patterns wrap a star body in a bounded repetition, so unrolling produces up to ten copies of an already ambiguous subexpression that every parser must then process within the measured call. Here the partial-derivative pair overtakes `deriv_bc` on both counts, with 10 timeouts against 14 and medians of 31.47 and 35.42 ms against 50.41, while `deriv_bc` still beats the standard derivative parsers on robustness, 14 timeouts against 32. The lower medians (3.79 and 3.81 ms) come from the 24 entries they complete and describe the easiest subset rather than a faster parser.

Category F is where `deriv_bc` loses most ground to the partial-derivative variants, timing out on 28 of its 70 entries against 2 for each. Its patterns end in a literal that fails late, so every entry must consume the whole input before that literal is checked and no parser can reject early. On completed entries `deriv_bc` appears faster than both variants, 42.91 against 106.55 and 109.19 ms on the mean and 2.59 against 7.29 and 7.68 ms on the median, but as in C these figures cover only the 42 entries it completed against 68 for each variant, so they describe the easier subset and the partial-derivative pair remains the more robust choice.

Category D is where `deriv_bc` regains the lead it lost on F, timing out on none of its 40 entries against 6 for each partial-derivative variant. Since the measured counts are close, the means compare directly, and the fewest timeouts and the lowest mean fall on the same parser, `deriv_bc`, at 141.61 against 204.20–261.62 ms. D's patterns are unrolled bounded repetitions, so consuming a prefix leaves a suffix of the unrolled chain, and different prefix lengths leave suffixes of different lengths. `deriv_bc` simplifies after each step, which rewrites equivalent residuals into a common form so that identical ones merge and the expression stays small. But the partial-derivative parsers only drop residuals that are already equal, so suffixes of different lengths remain separate elements, each stepped at every character, and the set grows with the pattern. The advantage of `deriv_bc` therefore comes from the simplification its encoding uses, not from the encoding itself.

Category H is a weaker version of D's outcome. `deriv_bc` again times out on fewer entries, 2 of its 28 against 5 for each partial-derivative variant, but here the timeouts and the median disagree. H's patterns are chains of adjacent unbounded groups, such as `(.+)(.+)`..., so the split points multiply across the group boundaries. The residual set the partial-derivative parsers carry grows with the number of groups. Each element is stepped at every character, but `deriv_bc` only carries a single simplified expression. On completed entries, however, the partial-derivative parsers are faster on both mean (141.85 and 166.72 against 261.45 ms) and median (39.31 and 44.75 against 80.84 ms).

Cat.	Parser	Timed out	Measured	Mean (ms)	Median (ms)	Max (ms)
I	`deriv_std_rec`	44 (1)	15	132.59	0.60	1 690.64
	`deriv_std_loop`	45	15	126.01	0.59	1 648.17
	`deriv_bc`	21	39	412.36	131.82	2 001.87
	`pderiv_std`	18	42	291.19	137.12	1 743.29
	`pderiv_bc`	18	42	244.71	103.34	1 874.64
J	`deriv_std_rec`	3	19	189.56	5.29	1 834.69
	`deriv_std_loop`	2	20	271.93	5.98	1 833.83
	`deriv_bc`	2	20	204.99	15.88	1 696.12
	`pderiv_std`	4	18	254.14	40.45	1 691.29
	`pderiv_bc`	4	18	214.61	35.51	1 392.01
K	`deriv_std_rec`	24	6	385.32	432.43	667.63
	`deriv_std_loop`	24	6	369.73	454.17	582.06
	`deriv_bc`	12	18	187.19	57.18	803.82
	`pderiv_std`	11	19	205.44	40.57	1 976.13
	`pderiv_bc`	10	20	183.48	35.82	1 193.98
L	`deriv_std_rec`	20	175	64.24	0.11	1 874.67
	`deriv_std_loop`	19	176	71.72	0.12	1 875.94
	`deriv_bc`	0	195	4.56	0.08	102.52
	`pderiv_std`	0	195	2.18	0.03	45.18
	`pderiv_bc`	0	195	2.14	0.03	51.74

Table 8.9: Per-parser timing over the synthetic set's default sweep for the 2×2 size matrix I–L (pattern size × input size). *Timed out* counts entries exceeding the budget, with any crash shown in parentheses separately; *Measured* counts entries with a completed timing.

Categories I, J, K, L form a 2×2 matrix of pattern size against input size. L is trivial in both and comparable, every parser measuring at least 175 of 195 entries, and its means put the partial-derivative pair at 2.18 and 2.14 ms, `deriv_bc` at 4.56 ms and the standard pair at 64.24 and 71.72 ms, so on trivial work `deriv_bc` is twice as slow as either partial-derivative variant. I grows only the input, and length alone pushes every parser past the budget on at least 18 of its 60 entries, rising to 44 for `deriv_std_rec` plus the crash its call-stack storage makes reachable (Section 8.4.1). J and K differ only in input length, since J's short input keeps per-entry work small and leaves the parsers little room to diverge, with timeouts at 2 to 4, while K repeats the same expensive steps over a long input and timeouts rise to 10 to 24. `deriv_std_rec` does not crash there, because the expensive steps time each entry out before its stack reaches the ceiling that caused the single crash in I.

Overall across all categories the standard derivative parsers are the most timeout-prone, `deriv_bc` beats them everywhere, and the partial-derivative variants beat the whole derivative family in A, B, C, F, I, and K. Though not in D, H, and J, where `deriv_bc` remains more robust, where it carries a single expression rather than a set of residuals. In D its simplification merges residuals that are equivalent but not syntactically equal, in H it does not pay the per-element cost of a set that grows with the number of groups, and in J it does not pay that cost across a wide alternation. Within the pair, the mean favours `pderiv_bc` in nine categories and the median in seven, so its advantage over `pderiv_std` lies in the slowest entries at the top of each sweep. Category J's completely inverted ranking, with the standard pair being fastest, is examined later in Table 8.13.

Representative Patterns

Six representative rows are drawn from the categories above. Four are swept past their baseline values in input length n, reaching millions of characters or thousands of repetitions, so that each row crosses from measured to failing within its own range. The other two, E37 and E38, instead sweep the pattern's own alternation width p at fixed input length, isolating compile-size cost from input length. Row selection and sweep points were set empirically as each row's cost became clear. Since the four n-sweep rows' inputs exceed the default 2 s budget, each uses a fixed 120 s deadline and an 8 GiB memory cap, so that its largest point can finish. E37 and E38 stay within the default budget and need neither override. Table 8.10 summarises how far each parser gets on them, in terms of the largest n or p it completes and the first value at which it fails.

The standard pair fails earliest on every row, by crashing rather than timing out. Since both run on the 256 MiB worker stack with the 8 GiB heap cap enabled, they have more stack headroom than in the bench of 8.4.1. They still reach the cap at the same n with nearly identical allocation. Recursion depth is therefore not the cause, but the heap is. Both keep the derivation sequence for the backward pass and its size grows with the input. With the standard pair ruled out, the remaining question is how `deriv_bc` and the partial-derivative pair compare. On P01 the pair outlasts `deriv_bc`, while on the other three all three reach the sweep maximum without timing out.

Row	Cat.	Pattern and Input	Standard `rec` & `loop`	`deriv_bc`	`pderiv` `std` & `bc`
P60	E	`(a|b|ab)*`, `a` repeated	✓ 5 000 × 20 000	✓ 10 000 000	✓ 10 000 000
P61	E	`(a|b|ab)*`, `ab` repeated	✓ 15 × 100	✓ 10 000 000	✓ 10 000 000
P09	B	`(a|aa)+`, `a` repeated	✓ 25 × 100	✓ 10 000 000	✓ 10 000 000
P01	A	`(a+)+`, `a` repeated	✓ 20 × 50	✓ 50 000 × 200 000	✓ 10 000 000
E37	J	p-way alternation, 4-letter words	✓ 10 000	✓ 10 000	✓ 10 000
E38	J	p-way alternation, numbered words	✓ 10 000	✓ 10 000	✓ 10 000

Table 8.10: Match-side crash and timeout thresholds. Each cell gives the largest swept n that completed (✓) above the first n that failed (×), and a dash means every swept n completed. P60 and P61 are the reference-paper scale rows, P09 and P01 are the nested-quantifier and overlapping-alternation rows, and E37 and E38 sweep the pattern size p rather than the input length. For P60, P61, P09, and P01, the two numbers are the last input length that completed and the first that failed. For E37 and E38, they are the last alternation width that completed and the first that failed. All six rows are drawn from the synthetic stress set. The standard variants share one column because under this test's stack cap the recursive and iterative forms reach the same limit.

P60 and P61. Figures 8.2 and 8.3 plot wall time for `(a|b|ab)*` on `a`- and `ab`-repeated input, at the scale of Sulzmann and Lu [2014]'s Figure 7(a) and 7(b). The comparison is not exact, since `deriv_bc` implements the paper's construction as described in their theory and builds a full parse tree, whereas the experiments in the paper used a variant adapted for submatching called DERIV, that records only a Kleene star's last match so it could be compared against TDFA, RE2, and C-POSIX. The candidate results are therefore not directly comparable to the paper's curves. Both inputs grow roughly linearly, and the lower panels plot the partial-derivative pair as a ratio to `deriv_bc`, which trails both across most of the sweep. `deriv_bc` holds the pattern's ambiguity in one expression and simplifies it after every character, while the partial-derivative pair keep a short list of readings and check each new one against it once, which is cheaper on both inputs.

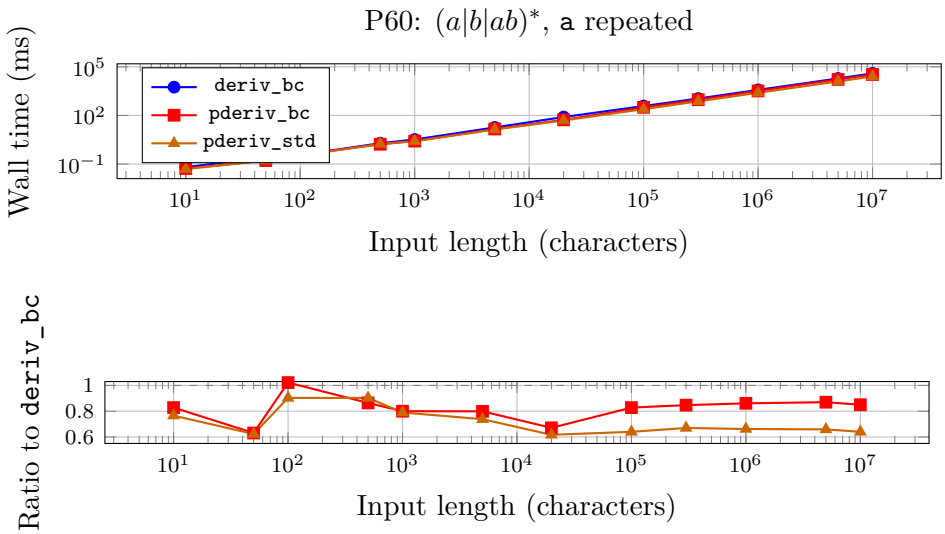


Figure 8.2: Wall time on P60, `(a|b|ab)*` against `a` repeated, to the scale of Sulzmann and Lu [2014]'s Figure 7(a). The lower panel plots the partial-derivative pair as a ratio to `deriv_bc`.

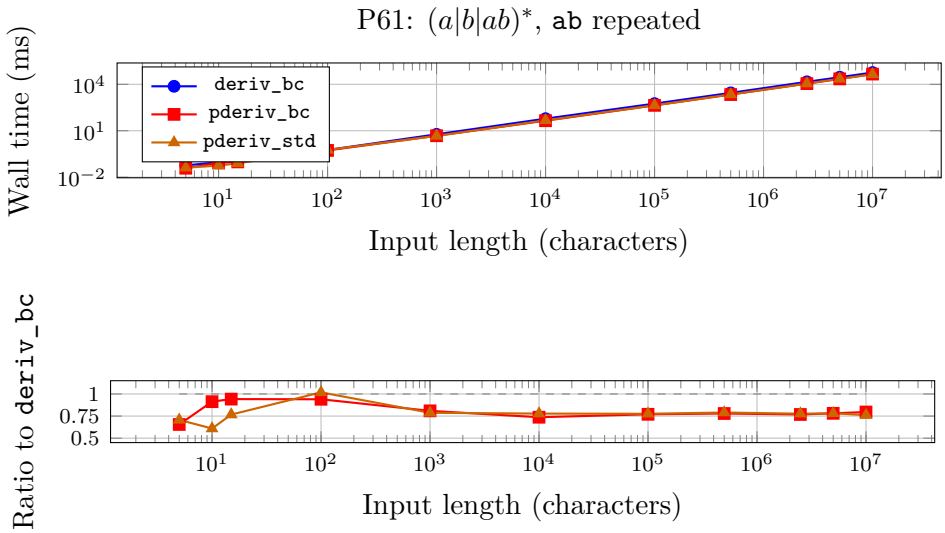


Figure 8.3: Wall time on P61, `(a|b|ab)*` against `ab` repeated, to the scale of Sulzmann and Lu [2014]'s Figure 7(b). The lower panel plots the partial-derivative pair as a ratio to `deriv_bc`.

Between the two partial-derivative parsers, `pderiv_std` is fastest on P60 across most of the sweep, reaching about 0.65 of `deriv_bc`'s time at the largest inputs, with `pderiv_bc` trailing it. On P61 the two stay close throughout and trade the lead at different points along the sweep, with neither keeping a consistent edge, unlike P60's clean ordering. This is specific to this pattern. Dataset-wide, `pderiv_bc` was shown to be faster on mean in nine categories and median in seven. `pderiv_bc` and `pderiv_std` step the same residuals and differ only in how they tag them. This gap could be attributed to how many residuals stay live, since tagging is charged per residual and the number live differs between the two inputs.

Let B denote the pattern `a|b|ab`, so the whole pattern is B^*, and let a *state* be the set of residuals that remain after a prefix of the input has been consumed. Moving to the next character means taking each residual's derivative and then dropping duplicates, keeping the first copy of any residual reached twice (6.1.2 and 6.2.3), starting with $S_0 = \{B^*\}$.

$$
\begin{aligned}
&\mathit{step}(S_0, \texttt{a}) = \{\, B^*,\quad \texttt{b} \cdot B^* \,\} =: S_1,\\
&\mathit{step}(S_1, \texttt{a}) = S_1,\\
&\mathit{step}(S_1, \texttt{b}) = \{\, B^* \,\} = S_0.
\end{aligned}
$$

The second residual in S_1 is $b \cdot B^*$. Here the `ab` branch has already used its `a` and is waiting for a `b`. When the next character is `a`, this residual cannot move, so it disappears. Meanwhile B^* produces the same two residuals again, and S_1 reproduces itself. Repeated `a`s give $S_1 \to S_1 \to S_1$, with two residuals on every step. When the next character is `b`, the waiting residual completes, both residuals become B^*, and the one copy that remains is S_0. So `ab`-repeated input alternates $S_0 \to S_1 \to S_0 \to S_1 \to \cdots$, with two residuals on every `a` and one on every `b`.

Each new residual carries a tag recording which branch produced it. `pderiv_bc` extends a bit string by one node per residual, while `pderiv_std` wraps the previous closure in another. Both allocate a small node, but the bit version also materialises the new bit slice, so `pderiv_bc` pays a real, if small, extra cost on every residual it tags. `a`-repeated input carries two residuals at every step, so that cost is paid twice per character. `ab`-repeated input drops to one on every other step, so it is paid 1.5 times on average and the gap narrows between the two partial-derivative parsers.

P01 and P09. Figures 8.4 and 8.5 plot `(a+)+` and `(a|aa)+`, category A and B's ambiguous Kleene-star pair: P01 nests a star directly inside a plus, while P09 leaves the quantifier flat and instead lets `a` and `aa` overlap, so the same substring can be consumed more than one way. So both patterns are different routes to the same kind of ambiguity. The standard variants crash earliest on both, at $n = 50$ on P01 and $n = 100$ on P09, later than either but still far earlier than the E-family rows above. `deriv_bc` times out on P01, at $n = 200\,000$, but not on P09, where it joins both partial-derivative variants in completing the full ten-million-character sweep. On P01, over the range where `deriv_bc` itself has data (up to $n = 50\,000$), the ratio starts between 0.7 and 0.9 at the smallest inputs and falls below 0.01 by $n = 50\,000$, since `deriv_bc`'s own cost grows far faster than the partial-derivative pair's as it nears its timeout. On P09, where `deriv_bc` completes the whole sweep instead of nearing a timeout, both partial-derivative variants instead stay in a roughly steady band, from about 0.38 to 0.70, throughout.

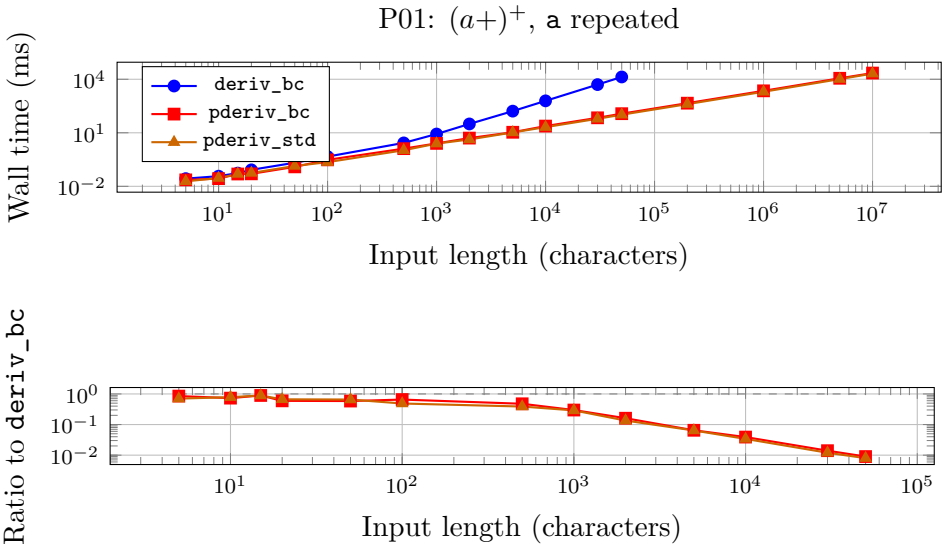


Figure 8.4: Wall time on P01, `(a+)+` against `a` repeated, every swept n on a log-log axis. The lower panel plots the partial-derivative pair's time as a ratio to `deriv_bc`.

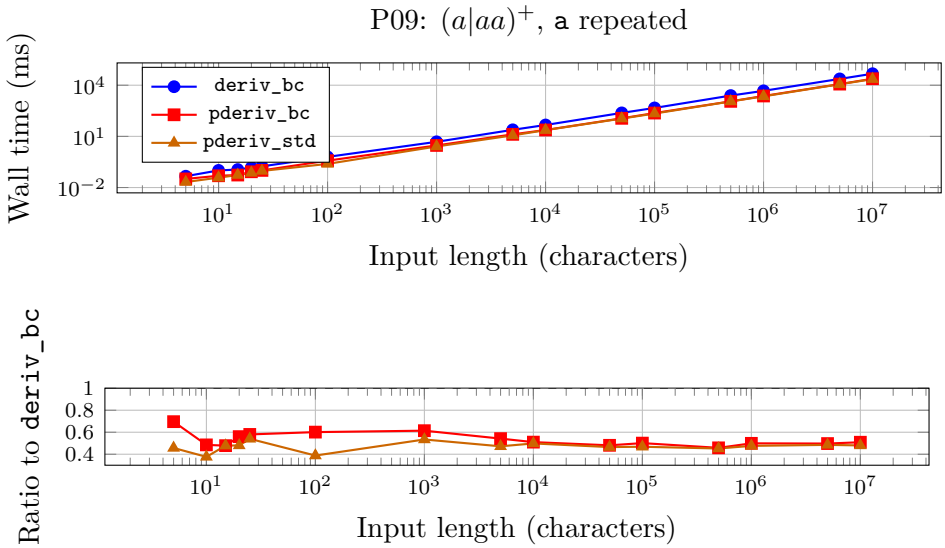


Figure 8.5: Wall time on P09, `(a|aa)+` against `a` repeated, every swept n on a log-log axis. The lower panel plots the partial-derivative pair's time as a ratio to `deriv_bc`.

The difference is what each pattern's ambiguity costs per character. P01 nests a `+` inside another `+`, so each additional `a` multiplies the number of ways the outer repetition could have been split into iterations, making every derivative step near-exponential. `deriv_bc` keeps all of these live splits in one expression and compares every pair of them after each character, so the cost grows with n, and the ratio collapses from near parity to under 0.01 before `deriv_bc` times out. P09's ambiguity is shallower, since category B's shared-prefix alternatives under one flat quantifier leave only a couple of live readings at any position, not a count tied to how many iterations have passed. The same pairwise cleanup therefore costs a bounded amount per character, holding the ratio in the 0.38–0.70 band out to ten million characters. Between the two partial-derivative variants, which avoid this cost on both rows by construction, `pderiv_std` is faster here, matching P60's ordering.

E37 and E38. The rows above sweep input length n. An E-row instead sweeps the pattern itself, isolating how `deriv_bc` and `pderiv_bc`'s three phases (Section 8.2.2) scale with pattern size rather than input length, on the matching side only. Both are drawn from category J ("large pattern, short input"), whose mechanism is the per-node cost of a very large pattern. Alternation was chosen to grow a pattern's node count by a single parameter p while leaving the input trivial. Table 8.11 reports E37, a p-way alternation of distinct four-letter words. Table 8.12 shows E38, a p-way alternation of distinct numbered placeholder words. Both sweep the same range, $p \in \{100, 500, 1\,000, 2\,000, 5\,000, 10\,000\}$, with the *node cap* (Section 8.2.4) raised from 20 000 to 200 000 for this test. The cap was sized for the real corpus, but within a p-way alternation it is too small once p gets this large. At $p = 10\,000$, E38's seven-character words each translate to 13 nodes (seven literals chained by six concatenations), and joining all 10 000 of them adds 9 999 more nodes for the alternation itself: $10\,000 \times 13 + 9\,999 = 139\,999$ nodes in total, almost seven times the default cap.

	`deriv_bc`			`pderiv_bc`		
p	Encode (ms)	Derive (ms)	Decode (ms)	Encode (ms)	Derive (ms)	Decode (ms)
100	6.49	0.60	0.01	0.04	1.74	0.00
500	0.63	4.75	0.02	0.25	34.07	0.02
1 000	2.16	12.96	0.04	0.41	138.39	0.05
2 000	2.59	41.05	0.10	1.03	603.65	0.11
5 000	6.51	215.08	0.21	2.85	4 499.90	0.21
10 000	13.44	826.40	0.48	5.10	30 845.30	0.41

Table 8.11: E37: `deriv_bc` vs. `pderiv_bc`. An E-row sweeping the pattern's alternation width p at fixed input length (`(?:aaaa|aaab|...)`, p distinct four-letter alternatives; input `"aaaa"`, matching the first alternative immediately). Encode grows only mildly with p for both parsers, and is consistently cheaper for `pderiv_bc`, whose plain tree clone does less work per node than `deriv_bc`'s annotated-state construction. Decode stays under 0.5 ms throughout for both. Derive dominates and diverges sharply: `pderiv_bc` is 2.9× `deriv_bc`'s derive time at $p = 100$, 14.7× by $p = 2\,000$, and 37.3× by $p = 10\,000$, since a partial-derivative step touches every live alternative on each character, while `deriv_bc` does not enumerate them individually.

	`deriv_bc`			`pderiv_bc`		
p	Encode (ms)	Derive (ms)	Decode (ms)	Encode (ms)	Derive (ms)	Decode (ms)
100	0.15	1.25	0.00	0.07	3.27	0.00
500	0.96	17.57	0.02	0.37	71.32	0.02
1 000	2.32	68.52	0.04	0.82	280.24	0.04
2 000	4.72	275.19	0.30	1.81	1 124.51	0.08
5 000	11.52	1 702.99	0.38	4.03	7 997.86	0.21
10 000	21.73	6 935.60	0.73	7.95	48 678.12	0.41

Table 8.12: E38: `deriv_bc` vs. `pderiv_bc`. An E-row, sweeping a p-way alternation of distinct numbered words (`w000001`, ..., `w0000`p), at fixed input length. The matching input is a single word (the first alternative). Derive grows with p for both parsers, but the divergence between them is milder and flatter than on E37: `pderiv_bc`'s derive time is 2.6× `deriv_bc`'s at $p = 100$, climbing only to 7.0× by $p = 10\,000$, against E37's 2.9× to 37.3× over the same range. Encode remains cheaper for `pderiv_bc` throughout, as on E37, and decode stays under 0.8 ms for both even at the widest point.

p	deriv_std_rec	deriv_std_loop	deriv_bc	pderiv_std	pderiv_bc
100	1.11	1.22	1.43	3.96	3.34
500	5.39	5.76	18.66	75.14	71.72
1 000	12.19	12.96	71.09	302.66	281.13
2 000	29.70	29.31	280.67	1 275.08	1 126.43
5 000	113.28	114.89	1 716.00	9 273.88	8 002.17
10 000	381.34	346.56	6 960.48	41 637.34	48 686.64

Table 8.13: Total match-side time (ms) for all five parsers on E38, same p sweep as Table 8.12. The standard pair stays cheapest throughout and grows the slowest, showing the reverse ranking.

The ranking inverts at wide p. Table 8.13 shows the mean time of all parsers on E38. Past a few thousand alternatives, the two partial-derivative parsers, fastest almost everywhere else become the slowest. By $p = 10\,000$, `deriv_bc` is roughly 18× slower than the standard pair, and the partial-derivative pair over 100× slower. The parsers blow up for different reasons, but the standard pair's own advantage here has one cause. Unlike the other three, it does not attempt to simplify the alternation at all, so it never pays for an attempt that could not have paid off anyway, since every one of the p alternatives stays genuinely distinct by construction.

```
fn dedup_first(branches: &mut Vec<ARegex>) {
    let mut i = 0;
    while i < branches.len() {
        let mut j = i + 1;
        while j < branches.len() {
            if same_shape(&branches[i], &branches[j]) {
                branches.remove(j);
            } else {
                j += 1;
            }
        }
        i += 1;
    }
}
```

Figure 8.6: `dedup_first`: the nested loop behind Figure 5.11's own `dedup_first` call. Every branch is compared against every later branch via `same_shape`, removing a match in place.

Figure 8.6's nested loop compares branch i against every later branch j. With p branches, that is roughly $p^2/2$ calls to `same_shape`, every single character. For p genuinely distinct words, `same_shape` is essentially always false, so nothing is ever removed. The full $O(p^2)$ cost is paid every character for zero branches removed. This directly implements the `nub` call in the choice case of Sulzmann and Lu [2014, sect. 3, Fig. 6]'s simplification:

$$simp\ (bs@\oplus(ri : rsi)) = bs@\oplus\Big(nub\ (filter\ (\neg\, isPhi)\ ((simp\ ri) : map\ simp\ rsi))\Big),$$

Haskell's `nub` is quadratic for the same reason. With only an `Eq` instance available, it keeps the first occurrence of each element by comparing it against every element already kept. `same_shape` is this same equality test, made insensitive to bit-code history so that identical branches merge regardless of parse path. The paper's complexity claim covers input length rather than pattern width, so this result does not contradict it.

The partial-derivative pair deduplicates differently, and more cheaply: `nub2` and `nub_tree` are both hash-based and $O(p)$. The two differ only in what carries along with each residual, a bit vector in `pderiv_bc` and an injection closure in `pderiv_std`. Since the mechanism is otherwise the same, only `pderiv_bc`'s is shown (Figure 8.7).

```
pub(super) fn step_frontier(frontier: &[(Regex, AppendBits)], x: char)
-> Vec<(Regex, AppendBits)> {
    let mut out = Vec::new();
    for (r, bits) in frontier {
        for (r_next, extra) in pderiv_bc(r, x) {
            out.push((r_next, bits.append(&extra)));
        }
    }
    nub2(out)
}

pub(super) fn nub2<B>(pairs: Vec<(Regex, B)>) -> Vec<(Regex, B)> {
    let mut seen: HashSet<Regex> = HashSet::with_capacity(pairs.len());
    let mut out = Vec::with_capacity(pairs.len());
    for (r, bs) in pairs {
        if seen.insert(r.clone()) {
            out.push((r, bs));
        }
    }
    out
}
```

Figure 8.7: `pderiv_bc`'s own `step_frontier` and `nub2`: each live residual steps through its own `pderiv_bc` call, then the combined results are deduplicated once via a `HashSet` keyed on the residual alone, implementing the rule of Sulzmann and Lu [2012], Definition 8: keep only the first occurrence of each residual across the whole combined result.

The loop in `step_frontier` is the real bottleneck. For every character, it calls `pderiv_bc` separately, once per live residual, rather than walking the whole pattern as one combined tree the way `deriv_bc` and the standard pair both do. `nub2` only runs once that loop has already finished, so however cheaply it merges the results, the p separate calls that produced them were already paid for. On E38, the words share the prefix `w0000...`, so many residuals stay alive, undistinguished from each other, for several characters before diverging. Each of those extra characters repeats the same once-per-residual loop, multiplying this cost across several steps rather than paying it once. That is consistent with E38 climbing more slowly than E37 (Tables 8.11 and 8.12). The standard pair pays neither cost. No cross-branch comparison and no per-residual call.

The improvement for `deriv_bc` would be replacing the pairwise `same_shape` scan with a hash keyed on bit-code-independent structure, which is the same approach `nub2` and `nub_tree` already take. This would make this dedup $O(p)$ without changing what it computes. The partial-derivative pair's own bottleneck has no equally direct fix, since it is the once-per-residual loop itself, not the comparison after it, that costs time. Batching that loop's work across all live residuals in one pass would require restructuring how they are stored, not just replacing a comparison.

8.4.3 Memory

Memory is reported per candidate on both datasets, using the isolation, counting-allocator setup, and averaging method described in Section 8.2.3.

S2ELab

Parser	Total allocated	Avg/entry	Max live peak
`deriv_std_rec`	29.19 GB	2.67 MB	582.4 MB
`deriv_std_loop`	29.96 GB	2.76 MB	618.2 MB
`deriv_bc`	15.94 GB	1.37 MB	5.1 MB
`pderiv_std`	5.61 GB	0.48 MB	2.5 MB
`pderiv_bc`	4.74 GB	0.41 MB	3.8 MB

Table 8.14: Heap allocation over the default scope of 463 patterns and 11 395 entries. *Total allocated* counts every allocation, including memory later freed, and measures allocator pressure. *Avg/entry* divides that total by the parser's own measured-entry count, since total alone scales with how many entries a parser completes and the standard variants exclude their timeouts. *Max live peak* is the highest live-byte high-water mark within a single call and measures the footprint a deployment would see. Total and peak are different quantities: total is cumulative and can exceed the live peak, because freed memory is counted in the former but not the latter. Timed-out entries are excluded from all three, so each is a lower bound on the standard variants' true figure.

The largest difference is in peak allocation. The standard variants reach 582.4 and 618.2 MB, compared with 5.1 MB for `deriv_bc`, a factor of 114–121. The standard parser retains its derivation sequence for the backward pass, whereas `deriv_bc` stores a flat bit vector. The difference in total allocation is smaller, at roughly 1.8–1.9×.

The partial-derivative pair shows a different result from the growth curves. `pderiv_std` has the lower peak (2.5 against 3.8 MB), reversing the pair's timing ranking on the same corpus. At these input lengths the frontier is already small, so bit-tracking adds overhead. The growth argument concerns how the variants scale, not which uses less memory on short inputs.

Growth on canonical expressions. Two demos drive the parsers on two canonical expressions and report live-state size per input character. For the two standard forms the figure is the node count of the current residual. For the two partial-derivative forms it is the total node count across the frontier, after deduplicating repeated residuals the same way the parser's own frontier-stepping does.

	a^* on a^n			$(a+(b+ab))^*$ on $(ab)^n$		
Parser	n=1	n=8	n=16	n=1	n=8	n=16
`deriv_std`	4	39	79	16	539	9179
`deriv_bc`	2	2	2	12	8	8
`pderiv_std`	2	2	2	18	8	8
`pderiv_bc`	2	2	2	18	8	8

Table 8.15: Live-state size per parser at three input lengths.

On `a`, the `deriv_std` state grows linearly with n, while the other three remain bounded. Its two-pass structure retains every intermediate expression until the backward pass, making state size depend on input length. On `(a+(b+ab))`, `deriv_std` reaches 9 179 nodes at $n = 16$, while `deriv_bc` remains between 8 and 12. Shape-based simplification collapses the repeated branches in this pattern. The partial-derivative variants also remain bounded, oscillating between 8 and 18 nodes. Their state is the deduplicated frontier of distinct residuals, rather than a single expression.

Synthetic Stress Set

Cat.	`std_rec/loop`	`deriv_bc`	`pderiv_std`	`pderiv_bc`
A	553.6	12.0	90.6	10.7
B	503.3	7.3	17.9	8.9
C	334.7	32.8	9.8	14.3
D	638.8	2.3	18.9	5.7
E	672.9	21.7	40.4	15.7
F	529.0	1.0	24.7	13.0
H	423.1	2.1	3.8	1.9

Table 8.16: Peak live MB over the full swept entry set per category: the highest live-byte high-water mark (Section 8.2.3) reached by any entry of that category, timed-out and crashed entries excluded. `std_rec` and `std_loop` are merged into one column, agreeing to within 0.1 MB on every category. Tables 8.17 and 8.18 report peak live memory for the representative rows the same way; cumulative allocation for one of them is discussed in prose below.

The standard pair's peak is at least an order of magnitude above every other parser in six of the seven categories, consistent with retaining the full derivation sequence regardless of which category's mechanism is in play. The exception is A, where the gap narrows to 6.1× – the same category where, as below, `pderiv_std`'s own peak is unusually high. Among the other three, `pderiv_std` has the highest peak in six of the seven categories (A, B, D, E, F, H), sometimes by a wide margin – 90.6 MB against `pderiv_bc`'s 10.7 MB and `deriv_bc`'s 12.0 MB on A. Its injection closures grow one layer per residual kept alive, where `pderiv_bc`'s bit vectors stay flat regardless of how many residuals survive. `deriv_bc` is the exception only on C, where its retained history for the backward pass costs more than either partial-derivative parser's frontier: 32.8 MB against 9.8 and 14.3 MB. The margin by which `pderiv_std` leads is itself uneven: narrowest on H, where all three stay within 1.9 MB of each other (1.9 to 3.8), and widest on A, where its peak is roughly eight times either of the other two (90.6 against 12.0 and 10.7). This is the same asymmetry noted for total allocation on the real corpus (Section 8.5.3): `pderiv_std`'s closures grow with every live residual, so the more adversarial a pattern's mechanism, the more that cost compounds relative to `pderiv_bc`'s flat bit vectors. These peaks are measured only over each category's *completed* entries, the same caveat the Timing subsection raises for its own tables: a category where one parser times out on most of its entries, such as A's 70% share for the standard pair, reports a peak drawn from a smaller and easier surviving subset than a category where it completes nearly everything. The standard pair's own peak also moves far less in relative terms across categories than the other three do: it

stays within a factor of two, from 335 to 673 MB, where `pderiv_std` ranges from 1.9 to 90.6 MB, a factor of close to fifty, and `pderiv_bc` from 1.9 to 15.7 MB, a factor of roughly eight.

Representative Patterns

The four representative rows are reported by mechanism family. Tables 8.17 and 8.18 give each row's peak live-byte high-water mark on any single entry, against the largest swept input length the parser completed. The two standard forms are reported once per row, since their peaks agree to within 0.01% of each other on every row measured.

Row	Parser	Max input length	Max live peak
P60	`deriv_std_rec/loop`	5 000	5.70 GB
P60	`deriv_bc`	10 000 000	2.64 GB
P60	`pderiv_std`	10 000 000	2.48 GB
P60	`pderiv_bc`	10 000 000	1.64 GB
P61	`deriv_std_rec/loop`	15	140.0 MB
P61	`deriv_bc`	10 000 000	3.40 GB
P61	`pderiv_std`	10 000 000	4.97 GB
P61	`pderiv_bc`	10 000 000	3.27 GB

Table 8.17: Peak live memory across the two E-family representative rows, against the largest swept input length each parser completed. Peak is the live high-water mark within a single call. The standard variants complete only the smallest swept inputs, so their peak is not directly comparable with the survivors' peaks and is excluded from the comparison in the text.

Row	Parser	Max input length	Max live peak
P01	`deriv_std_rec/loop`	20	805.3 MB
P01	`deriv_bc`	50 000	7.9 MB
P01	`pderiv_std`	10 000 000	1.68 GB
P01	`pderiv_bc`	10 000 000	1.14 GB
P09	`deriv_std_rec/loop`	25	284.3 MB
P09	`deriv_bc`	10 000 000	1.69 GB
P09	`pderiv_std`	10 000 000	1.92 GB
P09	`pderiv_bc`	10 000 000	1.40 GB

Table 8.18: Peak live memory across the two nested-quantifier representative rows, against the largest swept input length each parser completed. *Max live peak* is the live high-water mark within a single call.

The tables report peak live memory against the input length each parser actually completed. The standard variants fail at lengths in the tens to thousands, while the other three reach millions, so the standard variants' peaks are not directly comparable with the survivors' and are excluded from the comparisons below.

Among the survivors, `pderiv_bc` has the lowest peak on both E-family rows, though the two values diverge at matching input length, ten million characters each: 1.64 GB on P60 against 3.27 GB on P61. `deriv_bc` is lower than `pderiv_std` on P61 (3.40 against 4.97 GB) but higher on P60 (2.64 against 2.48 GB). `deriv_bc`'s own peak on P01 stays

low at 7.9 MB, since its timeout threshold keeps the derivative sequence short before it collapses to a single residual, consistent with its relatively small set of live candidates.

Within the partial-derivative pair, bit-coding reduces peak allocation on every representative row. This holds across the E family and on P01 and P09. On P09, for example, `pderiv_bc` peaks at 1.40 GB, compared with 1.92 GB for `pderiv_std`. Its memory advantage is therefore more consistent than its timing advantage.

Cumulative allocation is a separate matter. On P60, `deriv_bc` allocates 169.26 GB in total over the sweep while holding at most 2.64 GB live, because short-lived state is allocated and released at every step. This total exceeds the 15.94 GB allocated across all 11 395 real-corpus entries, showing the scale the synthetic set reaches; it is allocator churn, not a footprint.

8.4.4 Translation

Translation is reported per distinct pattern on both datasets, using the in-process setup and averaging method described in Section 8.2.4. Since translation cost depends on the pattern, results are counted per pattern rather than per entry.

S2ELab

Patterns	Mean (ms)	Max (ms)	Mean nodes	Max nodes
463	0.017	0.570	502.5	18 707

Table 8.19: Translation cost and output size over the 463 distinct patterns in the default scope.

Translation is inexpensive on the real corpus, with a mean of 0.017 ms. The largest pattern, at 18 707 translated nodes, also translates in under a millisecond. No real-corpus pattern exceeds the 20 000-node cap.

Synthetic Stress Set

Cat.	Name	Rows swept	Rejected	Mean (ms)	Max nodes
A	Nested quantifiers	16	0	0.001	84
B	Overlapping alternation	15	0	0.003	389
C	Bounded rep over star	7	0	0.104	9 881
D	Anchored-failure style	20	0	0.005	905
E	Prefix-overlap alternation	8	0	0.002	40
F	Alternation-star + literal	7	0	0.006	756
H	Many adjacent stars/groups	4	0	0.054	3 889
I	Trivial pattern, large input	10	0	0.024	4 001
J	Large pattern, trivial input	11	0	0.382	15 999
K	Large pattern, large input	3	0	0.103	8 400
L	Trivial pattern, trivial input	33	0	0.002	388

Table 8.20: Translation cost over the synthetic set's baseline values. *Rejected* counts rows whose translated result exceeds the node cap at the swept pattern size and therefore produce no measurement. Categories J and K each omit the rows that would exceed the cap at every entry of their sweep, since those rows carry no comparison signal.

Category J has the highest translation cost in the table, with a mean of 0.382 ms and 15 999 maximum nodes – unsurprising, since its patterns are deliberately the largest the

set sweeps. Category C is next, with a mean of 0.104 ms and 9 881 maximum nodes, consistent with its use of bound unrolling to grow pattern size instead. K's maximum nodes (8 400) rank third, behind J and C, but its mean (0.103 ms) is close to C's despite that gap; like J, neither rejects any row, since both are swept only over pattern sizes that stay under the cap.

Translation Caps

The three translation caps in Section 8.2.4 are based on the doubling search described there. Table 8.21 shows the crash points of the recursions underlying the bound and depth caps on the default 8 MiB stack, alongside the enforced caps.

Mechanism	Survives to	Crashes between	Cap
Bound unrolling	51 200	51 200–102 400	1 000
Syntactic nesting	6 400	6 400–12 800	2 000

Table 8.21: The crash point of each mechanism on the default 8 MiB stack, alongside the cap the frontend enforces. The caps sit far below the crash points, for the reasons in the text.

The depth estimate adds one level per group. In category J, the E39 row uses $p \in \{100, 500, 1000, 2000\}$ sequential capturing groups; at $p = 1000$ and $p = 2000$, this estimate exceeds the depth cap, rejecting those two entries (Section 8.4.5). The real corpus is unaffected: running `dataset_prepare` on S2ELab gives 578 accepted patterns, 572 usable patterns, and 13 957 inputs.

The caps are deliberately set below the measured crash points. The crash test uses one bound with a trivial one-node body, whereas real patterns can nest bounds and unroll multi-node subtrees. This increases both recursion depth and frame size. The measurements therefore give a safety reference, not a safe operating limit, and the caps remain well below it.

Real patterns remain well below the caps. Of the 762 authored patterns, 714 pass the frontend grammar check alone, before deduplication, anchor-shape checks, translation, or cap checks. This is an earlier filter than the 578 accepted patterns in Table 8.1, not a directly comparable count. Of the 714 patterns, 442 contain a bounded repetition. The largest bound is 67, with a median of 2 and a 99th percentile of 16.

The node cap is set by coverage rather than a recursion threshold. The largest real-corpus pattern translates to 18 707 nodes, so the 20 000-node cap covers every pattern in this evaluation.

8.4.5 Parse Agreement

Parse agreement is reported per dataset using the method in Section 8.2.6. Since shape agreement is meaningful only when membership agrees, both are reported together. The S2ELab count of 11 395 entries uses the same 463-pattern, full-match-only scope as Timing and Memory (Section 8.3.1), not the full prepared corpus of 572 patterns and 13 957 inputs.

Dataset	Entries	Timeouts	Crashes	Compared	Match agr.	Shape agr.	Shape dis.
S2ELab	11 395	73	0	5 798	100.0%	99.8% (5 784)	0.2% (14)
Synthetic	885	451	1	206	100.0%	89.8% (185)	10.2% (21)

Table 8.22: Parse agreement over both datasets. *Compared* is the number of expected matches where no candidate timed out or crashed and both families were internally consistent, so a single POSIX tree and a single non-POSIX tree exist to compare. The two shape columns are counted over *Compared*, not over *Entries*.

All completed entries show membership agreement across the candidates. No membership errors were observed. The 73 S2ELab timeouts indicate patterns that the slower candidates could not finish within the budget. The synthetic set's 451 timeouts and single crash are expected outcomes of its deliberately difficult inputs.

The real corpus has 14 shape disagreements, or 0.2% of compared matches. Each family is internally consistent on these entries, so the differences arise between the POSIX and non-POSIX policies. The synthetic set has 21 disagreements among 206 compared matches, or 10.2%. The category breakdown shows where these disagreements occur.

Category	Pattern	Entries	Timeouts	Compared	Shape agr.	Shape dis.
A	`(a+)+`	160	111	28	100.0% (28)	0.0% (0)
B	`(a|aa)+`	150	95	28	35.7% (10)	64.3% (18)
C	`(.*a){1,10}`	56	32	12	100.0% (12)	0.0% (0)
D	`(a?){N}a{N}`	40	12	14	100.0% (14)	0.0% (0)
E	`(a|b|ab)+`	80	45	16	87.5% (14)	12.5% (2)
F	`([ab]+)*c`	70	46	12	91.7% (11)	8.3% (1)
H	`(\w+\s{0,1})*`	28	18	4	100.0% (4)	0.0% (0)
I	`a+`	60	44	3	100.0% (3)	0.0% (0)
J	`(?:a{1,10}){1,100}`	22	4	9	100.0% (9)	0.0% (0)
K	`(?:ab|cd|ef|gh)+`	30	24	3	100.0% (3)	0.0% (0)
L	`[a-z]+`	189	20	77	100.0% (77)	0.0% (0)

Table 8.23: Parse agreement by category over the synthetic set. *Pattern* is one representative row per category; Table 8.4 names the mechanism each category stresses. Every completed entry had every candidate return the expected result, so the membership column is omitted. Category L excludes row E51 (Table 8.5), whose match example is deliberately a non-match by construction and so does not belong in a membership-based check; this leaves L's entries 6 short of the 195 test cases Table 8.4 counts for the category.

Category B accounts for 18 of the 21 disagreements, or 64.3% of its 28 compared matches. Its alternatives share a prefix under a quantifier, creating ambiguity between possible decompositions. Categories E and F contribute two and one disagreements, respectively. All other categories show none. The disagreements therefore concentrate in the categories designed to create overlapping alternatives.

Representative disagreements

S2ELab. The loosely-formatted-number pattern

```
^[-+]?([0-9]{1,3}[,]?)?([0-9]{3}[,]?)*[.]?[0-9]*$
```

on input `"9000"` produces two trees that flatten to the same word, shown in Figure 8.8. Every constructor name is as it appears in the parser output, with two abbreviations for

space. `<9>` stands for `Right(Char('9'))` and `<0>` for the nine-deep `Left` injection the digit-class encoding produces around `Char('0')`.

```
POSIX (deriv_bc):
  Pair(Pair(Pair(Pair(Right(Empty),
    Left(Pair(Pair(Pair(Empty, <9>),
               Left(Empty)), Right(Empty)))),
      Star([Pair(Pair(Pair(Pair(Empty, <0>), <0>), <0>),
                 Right(Empty))])),
    Right(Empty)), Star([]))

Non-POSIX (pderiv_bc):
  Pair(Pair(Pair(Pair(Right(Empty),
    Left(Pair(Pair(Pair(Empty, <9>),
               Right(Pair(<0>, Right(Pair(<0>, Empty))))),
          Right(Empty)))),
      Star([])), Right(Empty)), Star([<0>]))
```

Figure 8.8: The two parse trees for the loosely-formatted-number pattern on `"9000"`: the POSIX tree (`deriv_bc`) against the non-POSIX tree (`pderiv_bc`).

The trees differ in how they use the optional copies of the one-to-three-digit group. The POSIX tree takes `Left(Empty)` for both, so the group matches only `"9"`. The following `([0-9]{3}[ ,]?)` then matches `"000"` in one iteration. The non-POSIX tree takes `Right(Pair(<0>, Right(Pair(<0>, Empty))))` for both, so the group matches `"900"`. The star is empty, and the trailing `[0-9]` matches the final `"0"`.

Synthetic. Row P61, the pattern `(a|b|ab)*` on input `"ab"`, is the reference paper's running example, and here it appears at the smallest input that makes its ambiguity visible, shown in Figure 8.9.

```
POSIX (deriv_bc):
  Star([Right(Pair(Char('a'), Char('b')))])

Non-POSIX (pderiv_bc):
  Star([Left(Char('a')), Right(Left(Char('b')))])
```

Figure 8.9: The two parse trees for `(a|b|ab)*` on `"ab"`, the reference paper's running example of POSIX-versus-greedy disagreement.

The POSIX tree matches `"ab"` in one iteration through the right branch of the alternation. The non-POSIX tree uses two iterations, matching `"a"` and then `"b"` through the left branch. Both flatten to `"ab"`, so the disagreement is about parse structure rather than the matched word.

8.5 Discussion of Results

8.5.1 Cost of the standard construction

The two standard derivative parsers are consistently the slowest and use the most memory. Their two-pass structure stores every intermediate expression until the backward pass

completes. On `a` input, their state grows linearly with n, while the other three variants remain bounded in the measured range.

On the full corpus, the standard variants have similar mean times (7.751 ms and 7.454 ms), but their matching totals differ. `deriv_std_loop` takes 70.732 s, compared with 74.928 s for `deriv_std_rec`. Their non-match totals are almost identical, suggesting that iteration mainly reduces the cost of matching. On the representative rows, both variants reach the memory cap at the same input lengths and have similar allocation figures. This points to their shared retention of intermediate derivatives rather than recursion depth. The separate stack-depth test isolates the other failure mode. Without a memory cap, `deriv_std_loop` survives roughly 26 times longer in wall time before the doubling search's own per-probe cap ends it. Thus, the limiting factor depends on whether derivation cost or stack depth grows first.

8.5.2 Cost of bit-coding

For the standard-derivative family, bit-coding performs better on the measured corpus. `deriv_bc` has a mean time of 1.695 ms, roughly 4–4.5× below the standard variants, and a peak allocation of 5.1 MB, about 114–121× lower. Its forward pass retains information needed for single-pass reconstruction rather than simplifying every redundant branch. The decode step adds little time, staying under 0.1 ms even at $p = 2\,000$ swept alternatives, while encode grows with pattern size alone, from 0.63 ms at $p = 500$ to 2.59 ms at $p = 2\,000$ (Table 8.11). At far larger p, this same row is also where `deriv_bc` stops being faster than the standard pair at all, discussed below.

The representative sweeps extend this result. `deriv_bc` completes three of the four rows (P60, P61, P09), while both standard variants crash on all four. It reaches the full ten-million-character range on all three. Cumulative allocation can still be high, reaching 169.26 GB on P60, although peak allocation is 2.64 GB. This distinction between cumulative allocation and peak live bytes is why both are reported (Section 8.2.3).

8.5.3 Cost of partial derivatives

The partial-derivative parsers are the fastest and use the least memory on the S2ELab corpus. They represent the residual as a set of states rather than one combined expression, limiting the alternation growth seen in the standard derivative parser.

Within the pair, bit-coding does not consistently help or hurt. On the full corpus, `pderiv_bc` is marginally faster (0.634 against `pderiv_std`'s 0.722 ms mean) and has lower total allocation (4.74 against `pderiv_std`'s 5.61 GB), though `pderiv_std` still has the lower peak (2.5 against 3.8 MB). This asymmetry follows from how the two parsers apply the same collapse (Section 6.1.1): `pderiv_std`'s nullable-`Seq` branch builds a differently-shaped injection closure on every step where r_1's own derivative is `Eps`, regardless of whether a duplicate residual was actually waiting to be caught, while `pderiv_bc`'s side of the same collapse only removes a wrapper, with no added cost on a step where nothing needed collapsing. On the real corpus, where that collision is rare, this shows up as a small net loss for `pderiv_std` and a small net gain for `pderiv_bc`. The ranking changes again on the representative rows: `pderiv_std` is faster on P60, yet `pderiv_bc` still has the lower peak allocation there (1.64 against 2.48 GB) – the two axes do not move together on this row. The peak-memory ranking generalizes cleanly: `pderiv_bc` has the lower peak on

every one of the other three rows too (1.14 against 1.68 GB on P01; 1.40 against 1.92 GB on P09; 3.27 against 4.97 GB on P61), so its lower-peak advantage holds across all four representative rows without exception. The speed ranking does not generalize the same way: `pderiv_std` is faster on P01 and P09, matching P60's pattern, while on P61 the two are close enough to trade the lead across the sweep without a consistent winner, unlike P60's clean ordering. The results suggest that the relative cost of bit-coding depends on the pattern and input, not only on input length – consistently so for peak memory, where it always helps, but not for speed, where it helps on some patterns and not others.

The two bit-coded parsers' phase costs diverge sharply once the pattern itself grows wide. On a p-way alternation of equal-length words (Table 8.11), `pderiv_bc`'s derive time grows from 2.9× `deriv_bc`'s at $p = 100$ to 37.3× at $p = 10\,000$ on matching input, since a partial-derivative step touches every live alternative on each character, while `deriv_bc`'s symbolic derivative does not enumerate them individually – though both lose to the standard pair at this scale, as Section 8.4.2 shows. The gap narrows sharply on non-matching input: rejecting `"zzzz"` still means checking every one of the p alternatives, just without the per-character cost a full match needs, so both parsers' non-match time keeps growing with p but far more slowly than on the matching side – `deriv_bc` reaches 33.79 ms and `pderiv_bc` 8.96 ms at $p = 10\,000$, making `pderiv_bc` the faster of the two on this side despite trailing badly on the matching one. `pderiv_bc`'s own encode step is, however, consistently cheaper than `deriv_bc`'s, since cloning the pattern tree does less work per node than building `deriv_bc`'s annotated state.

Both partial-derivative variants produce non-POSIX trees. This is a difference in parsing policy, not a correctness defect, and is why the two families are compared separately in Section 8.4.5.

8.5.4 Threats to validity

Corpus scope. S2ELab represents one domain and its conventions. Other corpora may have different acceptance rates and timing profiles. Timing, Memory, and Parse Agreement use the 463-pattern default scope (Section 8.3.1). Including the remaining 115 patterns with `BENCH_MATCH_INTENT=both` could change the absolute results and potentially the ranking of candidates.

The synthetic set. Its 112 rows isolate individual complexity mechanisms at controlled scales rather than representing patterns written by typical users. Most categories vary one parameter at a time, so combinations of stress mechanisms are not directly tested.

Agreement subset. The *Compared* subset includes expected matches completed by every candidate, with both families internally consistent. It excludes 71 of 5 869 S2ELab matches (about 1%) and 237 of 443 synthetic matches (about 53%). The synthetic result is therefore based on a much smaller subset. For alternation-heavy categories, the excluded cases may also contain more ambiguity because shared prefixes can increase both derivation cost and the number of possible parses. This need not apply to categories such as I, where timeouts arise from input length alone.

Measurement model. The memory benchmarks measure heap allocations, not total process memory. Timing results are specific to the machine used and may vary with its CPU and cache. A recorded *crash* means that the process was killed by a signal, not that stack overflow was confirmed (Section 8.2.1). The separate stack-depth test removes

the memory cap and provides direct evidence of stack overflow. The representative-row crashes are instead consistent with the memory costs of retaining intermediate derivatives.

9 Related Work and Conclusion

Sulzmann and Lu observed that many POSIX regular expression implementations were incorrect. To address this problem, they proposed a method based on Brzozowski's derivatives [Brzozowski, 1964], which constructs a full parse tree according to the longest-leftmost disambiguation policy. They also provided a proof of correctness for their approach. In separate work, they investigated submatching, an area in which existing approaches predominantly rely on greedy disambiguation. Greedy matching is closely tied to the structure of the regular expression and can be implemented efficiently by translating the expression into a nondeterministic finite automaton (NFA). Examples include the general parsing approach of Frisch and Cardelli [2004] and Laurikari's tagged NFAs Laurikari [2000] for submatching. Applying these NFA-based approaches to POSIX semantics, however, requires additional bookkeeping to select the longest-leftmost result, as observed by Sulzmann and Lu [2012]. Instead, their approach to submatching uses Antimirov's partial derivatives directly, resulting in a non-POSIX counterpart to their full-parse-tree construction [Sulzmann and Lu, 2012, Antimirov, 1996].

While the original papers provided Haskell implementations, their practical realisation in a systems programming language had not been investigated. This thesis addressed this gap by implementing all four parser variants in Rust and empirically evaluating their performance and memory consumption on the S2ELab corpus [Siddiq et al., 2024] and a synthetic stress-test dataset.

Across the datasets, the implementations had the following performance ordering, from the slowest to the fastest parser: `deriv_std_rec`, `deriv_std_loop`, `deriv_bc`, `pderiv_std`, and `pderiv_bc`. In particular, `deriv_bc` was four to five times faster than the standard derivative parser while using roughly two orders of magnitude less peak memory. Partial derivatives proved more efficient for submatching on realistic inputs, whereas bit-coding substantially benefited the full-derivative parser but made little difference for partial derivatives. The relative performance nevertheless depends on the input, particularly on the growth of the residual set, so neither family was shown to be uniformly superior.

Beyond derivative-based approaches, Laurikari [2000] attach submatch information to automaton transitions, as in Kuklewicz's `regex-tdfa` [Kuklewicz, 2026]. It is one of three tools Sulzmann and Lu [2014] benchmark against in their own paper, not with the general construction presented there but with a second, adapted Haskell implementation that records only the last match of each Kleene star. The bit-coded parsers here implement that general construction directly, without this adaptation. Production engines such as RE2 [Google LLC, 2024] and Rust's `regex` [The Rust Regex Project Developers, 2024] compile to automata for linear-time matching but do not build the full parse trees this thesis targets.

For future work, three directions follow from these results. First, `deriv_bc` still trails the partial-derivative variants on several large-scale patterns, partly due to repeatedly simplifying a shared expression against itself rather than stepping independent residuals. Sulzmann and Lu report memory problems in their bit-coded implementation and propose computing the POSIX path separately before constructing the parse tree with a space-efficient tree builder [Sulzmann and Lu, 2014]. A related improvement for `deriv_bc` would be to defer bit construction until decoding. Second, the supported language could be extended to include operations currently rejected by the frontend, such as anchors, bounded lookaround, and backreferences, as discussed in Chapter 7. Lu's partial-derivative implementation [Lu and Sulzmann, 2012] provides a useful reference for such extensions. Additionally, the current trace instrumentation could be developed into a proper diagnostic tool identifying which subexpressions are responsible for a match or timeout, helping to localise remaining performance problems or providing verbose information on where and why a given input failed to match.

Bibliography

Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.

Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4): 481–494, 1964. doi: 10.1145/321239.321249.

Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). https://swtch.com/~rsc/regexp/regexp1.html, 2007. Accessed: 2026-08-05.

Alain Frisch and Luca Cardelli. Greedy regular expression matching. In *Automata, Languages and Programming (ICALP 2004)*, volume 3142, pages 618–629. Springer, 2004. doi: 10.1007/978-3-540-27836-8_53.

Google LLC. RE2: A fast, safe, POSIX-compatible regular expression library. https://github.com/google/re2, 2024. Accessed: 2026-08-05.

Chris Kuklewicz. regex-tdfa. Hackage, 2026. URL https://hackage.haskell.org/package/regex-tdfa. Accessed: 2026-09-11.

Ville Laurikari. NFAs with tagged transitions, their conversion to deterministic automata and application to regular expressions. In *Proceedings of the 7th International Symposium on String Processing and Information Retrieval (SPIRE 2000)*, 2000.

Kenny Zhuo Ming Lu and Martin Sulzmann. regex-pderiv. https://github.com/luzhuomi/regex-pderiv, 2012. Accessed: 2026-11-10.

Mohammed Latif Siddiq, Jiahao Zhang, Lindsay Roney, and Joanna C. S. Santos. RegexEval dataset. https://github.com/s2e-lab/RegexEval, 2024.

Martin Sulzmann. Programming paradigms: Regular expressions. https://sulzmann.github.io/ProgrammingParadigms/pp-regular-expressions.html, 2026. Accessed: 2026-08-08.

Martin Sulzmann and Kenny Zhuo Ming Lu. Regular expression sub-matching using partial derivatives. In *PPDP*, pages 79–90, 2012. doi: 10.1145/2370776.2370788.

Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

The Rust Project Developers. Using `Box<T>` to point to data on the heap. The Rust Programming Language, 2026a. URL https://doc.rust-lang.org/book/ch15-01-box.html. Accessed: 2026-09-11.

The Rust Project Developers. Types. The Rust Reference, 2026b. URL https://doc.rust-lang.org/reference/types.html. Accessed: 2026-09-11.

The Rust Project Developers. `JoinHandle` in `std::thread`. Rust Standard Library, 2026c. URL https://doc.rust-lang.org/std/thread/struct.JoinHandle.html. Accessed: 2026-11-07.

The Rust Regex Project Developers. regex — An implementation of regular expressions for Rust. https://github.com/rust-lang/regex, 2024. Accessed: 2026-08-05.